Group 4 - L09_Diffusion_student_notebook_ITAI_2376 (1)(1)

October 10, 2026

1 Creating Numbers/images with AI: A Hands-on Diffusion Model Exercise

1.1 Introduction

In this assignment, you'll learn how to create an AI model that can generate realistic images from scratch using a powerful technique called 'diffusion'. Think of it like teaching AI to draw by first learning how images get blurry and then learning to make them clear again.

1.1.1 What We'll Build

- A diffusion model capable of generating realistic images
- For most students: An AI that generates handwritten digits (0-9) using the MNIST dataset
- For students with more computational resources: Options to work with more complex datasets
- Visual demonstrations of how random noise gradually transforms into clear, recognizable images
- By the end, your AI should create images realistic enough for another AI to recognize them

1.1.2 Dataset Options

This lab offers flexibility based on your available computational resources:

- Standard Option (Free Colab): We'll primarily use the MNIST handwritten digit dataset, which works well with limited GPU memory and completes training in a reasonable time frame. Most examples and code in this notebook are optimized for MNIST.
- Advanced Option: If you have access to more powerful GPUs (either through Colab Pro/Pro+ or your own hardware), you can experiment with more complex datasets like Fashion-MNIST, CIFAR-10, or even face generation. You'll need to adapt the model architecture, hyperparameters, and evaluation metrics accordingly.

1.1.3 Resource Requirements

- Basic MNIST: Works with free Colab GPUs (2-4GB VRAM), ~30 minutes training
- Fashion-MNIST: Similar requirements to MNIST CIFAR-10: Requires more memory (8-12GB VRAM) and longer training (~2 hours)
- Higher resolution images: Requires substantial GPU resources and several hours of training

1.1.4 Before You Start

1. Make sure you're running this in Google Colab or another environment with GPU access
2. Go to 'Runtime' → 'Change runtime type' and select 'GPU' as your hardware accelerator
3. Each code cell has comments explaining what it does
4. Don't worry if you don't understand every detail - focus on the big picture!
5. If working with larger datasets, monitor your GPU memory usage carefully

The concepts you learn with MNIST will scale to more complex datasets, so even if you're using the basic option, you'll gain valuable knowledge about generative AI that applies to more advanced applications.

1.2 Step 1: Setting Up Our Tools

First, let's install and import all the tools we need. Run this cell and wait for it to complete.

[ ]:
```
# Step 1: Install required packages
%pip install einops
print("Package installation complete.")

# Step 2: Import libraries
# --- Core PyTorch libraries ---
import torch  # Main deep learning framework
import torch.nn.functional as F  # Neural network functions like activation␣
↪functions
import torch.nn as nn  # Neural network building blocks (layers)
from torch.optim import Adam  # Optimization algorithm for training

# --- Data handling ---
from torch.utils.data import Dataset, DataLoader  # For organizing and loading␣
↪our data
import torchvision  # Library for computer vision datasets and models
import torchvision.transforms as transforms  # For preprocessing images

# --- Tensor manipulation ---
import random  # For random operations
from einops.layers.torch import Rearrange  # For reshaping tensors in neural␣
↪networks
from einops import rearrange  # For elegant tensor reshaping operations
import numpy as np  # For numerical operations on arrays

# --- System utilities ---
import os  # For operating system interactions (used for CPU count)

# --- Visualization tools ---
import matplotlib.pyplot as plt  # For plotting images and graphs
from PIL import Image  # For image processing
from torchvision.utils import save_image, make_grid  # For saving and␣
↪displaying image grids
```

```
# Step 3: Set up device (GPU or CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"We'll be using: {device}")

# Check if we're actually using GPU (for students to verify)
if device.type == "cuda":
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
    print(f"GPU memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:
 ↪.2f} GB")
else:
    print("Note: Training will be much slower on CPU. Consider using Google␣
 ↪Colab with GPU enabled.")
```

```
Requirement already satisfied: einops in /usr/local/lib/python3.13/dist-packages
(0.8.2)
Package installation complete.
We'll be using: cuda
GPU name: Tesla T4
GPU memory: 15.64 GB
```

1.2.1 REPRODUCIBILITY AND DEVICE SETUP

[ ]:
```
# Step 4: Set random seeds for reproducibility
# Diffusion models are sensitive to initialization, so reproducible results␣
 ↪help with debugging
SEED = 42  # Universal seed value for reproducibility. 42 is a not a unviversal␣
 ↪or required value. *** JAMES DELGADO ***
torch.manual_seed(SEED)           # PyTorch random number generator.
np.random.seed(SEED)              # NumPy random number generator
random.seed(SEED)                 # Python's built-in random number generator

print(f"Random seeds set to {SEED} for reproducible results")

# Configure CUDA for GPU operations if available
if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)        # GPU random number generator
    torch.cuda.manual_seed_all(SEED)    # All GPUs random number generator

    # Ensure deterministic GPU operations
    # Note: This slightly reduces performance but ensures results are␣
 ↪reproducible
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

    try:
        # Check available GPU memory
```

```
        gpu_memory = torch.cuda.get_device_properties(0).total_memory / 1e9  # Convert to GB
        print(f"Available GPU Memory: {gpu_memory:.1f} GB")

        # Add recommendation based on memory
        if gpu_memory < 4:
            print("Warning: Low GPU memory. Consider reducing batch size if you encounter OOM errors.")
    except Exception as e:
        print(f"Could not check GPU memory: {e}")
else:
    print("No GPU detected. Training will be much slower on CPU.")
    print("If you're using Colab, go to Runtime > Change runtime type and select GPU.")
```

```
Random seeds set to 42 for reproducible results
Available GPU Memory: 15.6 GB
```

1.3 Step 2: Choosing Your Dataset

You have several options for this exercise, depending on your computer's capabilities:

1.3.1 Option 1: MNIST (Basic - Works on Free Colab)

- Content: Handwritten digits (0-9)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if**: You're using free Colab or have a basic GPU

1.3.2 Option 2: Fashion-MNIST (Intermediate)

- Content: Clothing items (shirts, shoes, etc.)
- Image size: 28x28 pixels, Grayscale
- Training samples: 60,000
- Memory needed: ~2GB GPU
- Training time: ~15-30 minutes on Colab
- **Choose this if**: You want more interesting images but have limited GPU

1.3.3 Option 3: CIFAR-10 (Advanced)

- Content: Real-world objects (cars, animals, etc.)
- Image size: 32x32 pixels, Color (RGB)
- Training samples: 50,000
- Memory needed: ~4GB GPU
- Training time: ~1-2 hours on Colab
- **Choose this if**: You have Colab Pro or a good local GPU (8GB+ memory)

1.3.4 Option 4: CelebA (Expert)

- Content: Celebrity face images
- Image size: 64x64 pixels, Color (RGB)
- Training samples: 200,000
- Memory needed: ~8GB GPU
- Training time: ~3-4 hours on Colab
- **Choose this if**: You have excellent GPU (12GB+ memory)

To use your chosen dataset, uncomment its section in the code below and make sure all others are commented out.

[ ]:
```
#================================================================================
# SECTION 2: DATASET SELECTION AND CONFIGURATION
#================================================================================
# STUDENT INSTRUCTIONS:
# 1. Choose ONE dataset option based on your available GPU memory
# 2. Uncomment ONLY ONE dataset section below
# 3. Make sure all other dataset sections remain commented out

#----------------------------------------
# OPTION 1: MNIST (Basic - 2GB GPU)
#----------------------------------------
# Recommended for: Free Colab or basic GPU
# Memory needed: ~2GB GPU
# Training time: ~15-30 minutes

    # --- LEARNING COMMENTS ADDED: JAMES DELGADO ---

# MNIST images are 28 pixels tall and 28 pixels wide.
# This variable records the expected size; it does not resize images.
IMG_SIZE = 28

# IMG_CH means image channels.
# Grayscale needs one channel: one brightness value per pixel.
IMG_CH = 1

# A class is a category. MNIST has ten digit categories: 0 through 9.
# Later, the class label will tell the model which digit to generate.
N_CLASSES = 10

# A batch is a group of examples processed together.
# During training, we will use up to 64 images for each weight update.
# Smaller batches generally need less working memory.
BATCH_SIZE = 64


# An epoch is one complete pass through the training subset.
```

```
# We plan to give the model 30 passes to keep adjusting its weights.
# Unlike image size and class count, 64 and 30 are adjustable
# training choices supplied by the notebook.
EPOCHS = 30


# This defines the preparation applied when an image is
# retrieved. Compose runs the steps in order: ToTensor converts the
# image into a numerical tensor and divides pixel values by 255,
# changing their range from 0-255 to 0-1. Normalize then calculates
# (value - 0.5) / 0.5, changing that range to -1 through 1.
# This gives the model a consistent input scale while preserving
# the digit's shape and leaving its label unchanged.
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])


# --- END LEARNING COMMENTS James Delgado ---

# Your code to load the MNIST dataset
# Hint: Use torchvision.datasets.MNIST with root='./data', train=True,
#       transform=transform, and download=True
# Then print a success message

# Enter your code here:

    # --- BEGIN STUDENT WORK: KING OWUSU ---

# I chose MNIST because its small grayscale images make the computing
# requirements manageable, and the digits are easy to inspect visually.
# This section was missing the code that creates the dataset object.
# Later cells need this object to check the data and organize batches.

dataset = torchvision.datasets.MNIST(
    root="./data",        # Store the files in the colab working invironment.
    train=True,           # Select the training examples: this does not train a␣
 ↪model.
    transform=transform,  # Apple the prepartion steps defined above when␣
 ↪retreiving images.
    download=True         # Download missing files and reuse files already␣
 ↪downloaded.
)

# Check the number of examples instead of assuming the download worked.
# The : formatting below adds a thousands separator for readability.
```

```
print(f"MNIST loaded successfully: {len(dataset):,} images.")
print("Cell edited by James Delgado")

# Verified result: the cell reported 60,000 images, matching the expected
# size of the MNIST training portion. Model training has not started yet.

# --- END STUDENT WORK: KING OWUSU ---


#-----------------------------------------
# OPTION 2: Fashion-MNIST (Intermediate - 2GB GPU)
#-----------------------------------------
# Uncomment this section to use Fashion-MNIST instead
"""
IMG_SIZE = 28
IMG_CH = 1
N_CLASSES = 10
BATCH_SIZE = 64
EPOCHS = 30

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Your code to load the Fashion-MNIST dataset
# Hint: Very similar to MNIST but use torchvision.datasets.FashionMNIST

# Enter your code here:

"""

#-----------------------------------------
# OPTION 3: CIFAR-10 (Advanced - 4GB+ GPU)
#-----------------------------------------
# Uncomment this section to use CIFAR-10 instead
"""
IMG_SIZE = 32
IMG_CH = 3
N_CLASSES = 10
BATCH_SIZE = 32  # Reduced batch size for memory
EPOCHS = 50      # More epochs for complex data

# Your code to create the transform and load CIFAR-10
# Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.
 ↪5, 0.5, 0.5))
# Then load torchvision.datasets.CIFAR10
```

```
# Enter your code here:

"""
```

```
MNIST loaded successfully: 60,000 images.
Cell edited by James Delgado
```

[ ]:
```
'\nIMG_SIZE = 32\nIMG_CH = 3\nN_CLASSES = 10\nBATCH_SIZE = 32  # Reduced batch size for memory\nEPOCHS = 50      # More epochs for complex data\n\n# Your code to create the transform and load CIFAR-10\n# Hint: Use transforms.Normalize with RGB means and stds ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))\n# Then load torchvision.datasets.CIFAR10\n\n# Enter your code here:\n\n'
```

[ ]:
```
#Validating Dataset Selection
#Let's add code to validate that a dataset was selected
# and check if your GPU has enough memory:

# Validate dataset selection
if 'dataset' not in locals():
    raise ValueError("""
      ERROR: No dataset selected! Please uncomment exactly one dataset option.
    Available options:
    1. MNIST (Basic) - 2GB GPU
    2. Fashion-MNIST (Intermediate) - 2GB GPU
    3. CIFAR-10 (Advanced) - 4GB+ GPU
    4. CelebA (Expert) - 8GB+ GPU
    """)

# Your code to validate GPU memory requirements
# Hint: Check torch.cuda.is_available() and use torch.cuda.
 ↪get_device_properties(0).total_memory
# to get available GPU memory, then compare with dataset requirements

# Enter your code here:

    # --- BEGIN STUDENT WORK: JAMES DELGADO ---

# The check above confirms that a variable named dataset exists.
# I also display its type and size so I can check what was loaded.
print(
    f"Dataset confirmed: {type(dataset).__name__}, "
    f"{len(dataset):,} images."
)

# I use the notebook's approximate 2 GB recommendation for MNIST.
# Actual training memory use also depends on the model and batch size.
required_gpu_memory_gb = 2.0
```

```
# Check for a GPU before requesting its properties.
# This allows the cell to run while I am preparing data on CPU.
if torch.cuda.is_available():

    # Device 0 means the first GPU.
    gpu_properties = torch.cuda.get_device_properties(0)

    # PyTorch reports memory in bytes. Divide by one billion
    # to express the total capacity in gigabytes (GB).
    total_gpu_memory_gb = gpu_properties.total_memory / 1e9

    print(f"Total GPU memory: {total_gpu_memory_gb:.2f} GB")
    print(f"Approximate MNIST recommendation: {required_gpu_memory_gb:.1f} GB")

    # This compares total capacity with the assignment's estimate.
    # It does not guarantee that enough memory is currently free.
    if total_gpu_memory_gb >= required_gpu_memory_gb:
        print("GPU capacity meets the notebook's MNIST recommendation.")
    else:
        print("Warning: GPU capacity is below the MNIST recommendation.")

else:
    # A GPU is not required for the data preparation we are doing now.
    # We plan to switch to the T4 GPU before testing model training.
    print("CPU mode: GPU memory check skipped.")
    print("Data preparation can continue.")

# --- END STUDENT WORK: JAMES DELGADO ---
```

```
Dataset confirmed: MNIST, 60,000 images.
Total GPU memory: 15.64 GB
Approximate MNIST recommendation: 2.0 GB
GPU capacity meets the notebook's MNIST recommendation.
```

[ ]:
```
#Dataset Properties and Data Loaders
#Now let's examine our dataset
#and set up the data loaders:

# Your code to check sample batch properties
# Hint: Get a sample batch using next(iter(DataLoader(dataset, batch_size=1)))
# Then print information about the dataset shape, type, and value ranges

# Enter your code here:

    # --- BEGIN STUDENT WORK: JAMES DELGADO ---
```

```
# I inspect one example to check that the earlier image preparation
# produced the expected dimensions and pixel values.
# This batch size of 1 is only for inspection; BATCH_SIZE remains 64.
# shuffle=False keeps the examples in their original order.
sample_loader = DataLoader(dataset, batch_size=1, shuffle=False)

# iter() prepares to move through the batches, and next() retrieves
# the first one. Images and their matching labels are returned together.
sample_images, sample_labels = next(iter(sample_loader))

# Check how the image numbers are organized and stored.
print("Image batch shape:", sample_images.shape)
print("Image data type:", sample_images.dtype)

# Our earlier normalization should put pixel values between -1 and 1.
# min() and max() check this sample's smallest and largest values.
# item() extracts a single number from a tensor for printing.
print(
    f"Sample pixel range: {sample_images.min().item():.3f} "
    f"to {sample_images.max().item():.3f}"
)

# The label should remain an integer identifying the digit.
# It comes from the dataset, not from a model prediction.
print("Label batch shape:", sample_labels.shape)
print("Label data type:", sample_labels.dtype)
print("Digit label:", sample_labels.item())

# Visually compare the image with its supplied label.
# [0, 0] selects the first image and its single grayscale channel.
# vmin and vmax match the brightness range used in our preparation.
plt.imshow(sample_images[0, 0], cmap="gray", vmin=-1, vmax=1)
plt.title(f"MNIST digit label: {sample_labels.item()}")
plt.axis("off")
plt.show()

# Verified result: the sample image has shape [1, 1, 28, 28],
# uses float32, and has pixel values from -1 to 1.
# Its integer label is 5, matching the displayed handwriting.

    # --- END STUDENT WORK: KING OWUSU ---

#=============================================================================
# SECTION 3: DATASET SPLITTING AND DATALOADER CONFIGURATION
#=============================================================================
# Create train-validation split
```

```
# Your code to create a train-validation split (80% train, 20% validation)
# Hint: Use random_split() with appropriate train_size and val_size
# Be sure to use a fixed generator for reproducibility

# Enter your code here:

    # --- BEGIN STUDENT WORK: JAMES DELGADO ---

# Import the function that randomly divides a dataset into subsets.
from torch.utils.data import random_split

# Use 80% of the examples for training.
# int() expresses the count as a whole number.
train_size = int(0.8 * len(dataset))

# Give all remaining examples to validation.
# Subtracting ensures that every example belongs to one of the subsets.
val_size = len(dataset) - train_size

# Create a separate random-number generator for this split.
# Resetting it to our seed lets us repeat the same division without
# depending on random numbers used elsewhere in the notebook.
split_generator = torch.Generator().manual_seed(SEED)

# Randomly assign examples to two non-overlapping subsets.
# Each image stays paired with its original label.
train_dataset, val_dataset = random_split(
    dataset,
    [train_size, val_size],
    generator=split_generator
)

# Check the actual subset sizes returned by the split.
print(f"Training examples: {len(train_dataset):,}")
print(f"Validation examples: {len(val_dataset):,}")

    # --- END STUDENT WORK: JAMES DELGADO ---


# Your code to create dataloaders for training and validation
# Hint: Use DataLoader with batch_size=BATCH_SIZE, appropriate shuffle settings,
# and num_workers based on available CPU cores

# Enter your code here:

    # --- BEGIN STUDENT WORK: KING OWUSU ---
```

```
# Check the reported CPU count and use at most two loading workers.
# The fallback of 1 handles cases where the CPU count is unavailable.
# I start with a small worker count to limit loading overhead.
cpu_count = os.cpu_count() or 1
num_workers = min(2, cpu_count)

# Give training shuffles their own seeded random-number generator.
# This helps repeat the sequence of shuffles across fresh runs.
# The generator advances, so each epoch can have a different order.
train_generator = torch.Generator().manual_seed(SEED)

# Deliver training examples in shuffled batches of 64.
# Each image remains paired with its label during shuffling.
train_dataloader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=num_workers,
    generator=train_generator,
    drop_last=False
)

# Deliver validation examples in a consistent order.
# drop_last=False keeps the final batch even if it has fewer
# than 64 examples, so all validation examples are included.
val_dataloader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=num_workers,
    drop_last=False
)

# Check how many batches each loader will provide.
print(f"Data-loading workers: {num_workers}")
print(f"Training batches per epoch: {len(train_dataloader)}")
print(f"Validation batches per pass: {len(val_dataloader)}")

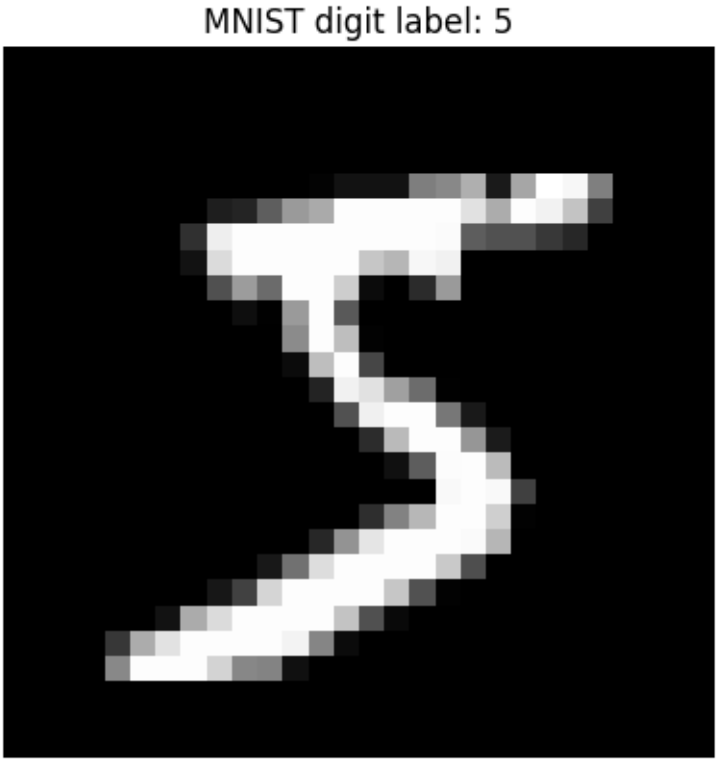
# Retrieve one training batch to verify that the loader works.
# This inspects the data; it does not update any model weights.
batch_images, batch_labels = next(iter(train_dataloader))
print("Training image batch shape:", batch_images.shape)
print("Training label batch shape:", batch_labels.shape)

# Verified result: the loaders report 750 training batches and
# 188 validation batches, using two data-loading workers.
# Retrieving a training batch succeeded: images have shape
```

```
# [64, 1, 28, 28], with 64 corresponding labels.

    # --- END STUDENT WORK: KING OWUSU ---
```

```
Image batch shape: torch.Size([1, 1, 28, 28])
Image data type: torch.float32
Sample pixel range: -1.000 to 1.000
Label batch shape: torch.Size([1])
Label data type: torch.int64
Digit label: 5
```




```
Training examples: 48,000
Validation examples: 12,000
Data-loading workers: 2
Training batches per epoch: 750
Validation batches per pass: 188
Training image batch shape: torch.Size([64, 1, 28, 28])
Training label batch shape: torch.Size([64])
```

1.4 Step 3: Building Our Model Components

Now we'll create the building blocks of our AI model. Think of these like LEGO pieces that we'll put together to make our number generator:

- GELUConvBlock: The basic building block that processes images
- DownBlock: Makes images smaller while finding important features
- UpBlock: Makes images bigger again while keeping the important features
- Other blocks: Help the model understand time and what number to generate

1.4.1 Learning notes: James Delgado

The previous section prepared batches of 64 images, each with one grayscale channel and 28×28 pixels. This section builds the model components that will process those images.

The components have different jobs:

- **GELUConvBlock:** Combines convolution, normalization, and an activation function. Convolution looks for local patterns; normalization adjusts the scale of intermediate values; the activation function allows more complex relationships to be learned. GELU means Gaussian Error Linear Unit.
- **DownBlock:** Reduces the height and width of feature maps. Feature maps are grids of numbers representing patterns extracted from the image. Smaller grids help the network combine information over larger parts of the original image.
- **UpBlock:** Increases the height and width of feature maps again. Connections to earlier layers help retain spatial details as the model returns to the original image resolution.
- **Time and class blocks:** Supply information about the diffusion step and the requested digit. "Time" here means a step in the noise-adding or noise-removal process, not clock time.

These pieces will form a U-Net, named for its shrinking and expanding structure.

For this assignment, the model will receive a noisy image, a diffusion step, and a digit label. Its training job is to predict the noise in the image. Later, a separate sampling process will use those predictions to gradually turn random noise into a new digit image.

The next code cell defines GELUConvBlock. Defining this reusable component establishes its operations; its weights will be learned during training after we assemble the model.

Learning notes: King Owusu

The output has the same shape as the input because the model predicts the noise that was added to each pixel.

Why the blocks are built this way

- **Two ways to shrink an image.** `RearrangePoolBlock` moves each 2×2 patch of pixels into channels ($28 \times 28 \times$ C becomes $14 \times 14 \times 4$C) and then uses a GELUConvBlock to compress 4C channels back to C. Unlike max pooling, nothing is thrown away at the rearranging step; the convolution learns what to keep.
- **Why padding matters.** Every GELUConvBlock uses a 3×3 filter with padding 1, so it never changes height or width. Only the rearrange step (down) and the transposed convolution (up) change the size, which keeps the shapes easy to track.
- **Why GroupNorm instead of BatchNorm.** GroupNorm normalizes within groups of channels for each image separately, so it does not depend on batch size. We use 8 groups, and 32, 64 and 128 channels all divide evenly by 8.

- **How a skip connection fits in.** In `UpBlock.forward`, the decoder features and the saved encoder features are joined with `torch.cat` along the channel dimension, doubling the channels (128 + 128 = 256 in Up 1). That is why the transposed convolution takes `2 * in_chs` input channels. The transposed convolution (kernel 2, stride 2) then doubles height and width and reduces the channels.
- **Where the conditioning enters.** The time embedding (what step we are on) and the class embedding (which digit) are both added at the 7×7 bottleneck, where each has 128 channels so it can be added directly to the feature map.

[ ]:
```python
# Basic building block that processes images
class GELUConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, group_size):
        """
        Creates a block with convolution, normalization, and activation

        Args:
            in_ch (int): Number of input channels
            out_ch (int): Number of output channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Check that group_size is compatible with out_ch
        if out_ch % group_size != 0:
            print(f"Warning: out_ch ({out_ch}) is not divisible by group_size ({group_size})")
            # Adjust group_size to be compatible
            group_size = min(group_size, out_ch)
            while out_ch % group_size != 0:
                group_size -= 1
            print(f"Adjusted group_size to {group_size}")

        # Your code to create layers for the block
        # Hint: Use nn.Conv2d, nn.GroupNorm, and nn.GELU activation
        # Then combine them using nn.Sequential

        # Enter your code here:

        # --- BEGIN STUDENT WORK: KING OWUSU ---

        # I use Sequential to apply the layers in this order:
        # convolution, group normalization, then GELU activation.
        # self.model stores these layers as part of this block.
        self.model = nn.Sequential(

            # A 3-by-3 filter moves one position at a time (stride=1).
            # Padding adds a one-pixel border of zeros so this
```

```
            # convolution preserves the image's height and width.
            # out_ch controls how many feature maps it produces.
            nn.Conv2d(
                in_channels=in_ch,
                out_channels=out_ch,
                kernel_size=3,
                stride=1,
                padding=1
            ),

            # Normalize the convolution's output channels in groups.
            # Here, group_size means NUMBER of groups.
            # The instructor's check above ensures out_ch divides
            # evenly into this number of groups.
            nn.GroupNorm(
                num_groups=group_size,
                num_channels=out_ch
            ),

            # GELU changes feature values through a nonlinear function.
            # It keeps the tensor's shape unchanged.
            nn.GELU()
        )

        # --- END STUDENT WORK: KING OWUSU ---

    def forward(self, x):
        # Your code for the forward pass
        # Hint: Simply pass the input through the model

        # Enter your code here:

        # --- BEGIN STUDENT WORK: KING OWUSU ---

        # x is the incoming batch of images or feature maps.
        # Pass it through our three layers and return the result
        # so the next part of the network can use it.
        return self.model(x)

        # --- END STUDENT WORK: KING OWUSU ---
```

[ ]:
```
# Rearranges pixels to downsample the image (2x reduction in spatial dimensions)
class RearrangePoolBlock(nn.Module):
    def __init__(self, in_chs, group_size):
        """
        Downsamples the spatial dimensions by 2x while preserving information
```

```
        Args:
            in_chs (int): Number of input channels
            group_size (int): Number of groups for GroupNorm
        """
        super().__init__()

        # Your code to create the rearrange operation and convolution
        # Hint: Use Rearrange from einops.layers.torch to reshape pixels
        # Then add a GELUConvBlock to process the rearranged tensor

        # Enter your code here:

            # --- BEGIN STUDENT WORK: KING OWUSU ---

        # Move the four positions in each 2-by-2 patch into channels.
        # This halves height and width while multiplying channels by 4.
        # Rearrangement keeps all values; it changes their organization.
        # Both input height and width must be divisible by 2.
        #
        # b = batch size, c = input channels
        # h, w = height and width AFTER rearranging
        # p1, p2 = patch height and width, both set to 2
        self.rearrange = Rearrange(
            "b c (h p1) (w p2) -> b (c p1 p2) h w",
            p1=2,
            p2=2
        )

        # Reuse the processing block from our previous cell.
        # It receives 4 times the original number of channels
        # and learns to combine them back into in_chs channels.
        # Its padding keeps the new, smaller height and width.
        self.conv = GELUConvBlock(
            in_ch=4 * in_chs,
            out_ch=in_chs,
            group_size=group_size
        )

            # --- END STUDENT WORK: KING OWUSU ---

    def forward(self, x):
        # Your code for the forward pass
        # Hint: Apply rearrange to downsample, then apply convolution

        # Enter your code here:

        # --- BEGIN STUDENT WORK: KING OWUSU ---
```

```
        # First, move spatial positions into the channel dimension.
        x = self.rearrange(x)

        # Then process and combine those channels using our
        # convolution, group normalization, and GELU activation.
        x = self.conv(x)

        # Send the smaller feature maps to the next network component.
        return x

        # --- END STUDENT WORK: KING OWUSU ---
```

[ ]:
```
# Let's implement the downsampling block for our U-Net architecture:
class DownBlock(nn.Module):
    """
    Downsampling block for encoding path in U-Net architecture.

    This block:
    1. Processes input features with two convolutional blocks
    2. Downsamples spatial dimensions by 2x using pixel rearrangement

    Args:
        in_chs (int): Number of input channels
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__()

        # JAMES DELGADO - LEARNING NOTES:
        # This block combines the components we defined earlier.
        # The first GELUConvBlock produces out_chs feature maps.
        # The second processes those features without changing shape.
        # RearrangePoolBlock then halves height and width.
        # It also contains its own GELUConvBlock, which combines
        # the rearranged channels back into out_chs channels.
        layers = [
            GELUConvBlock(in_chs, out_chs, group_size),
            GELUConvBlock(out_chs, out_chs, group_size),
            RearrangePoolBlock(out_chs, group_size)
        ]

        # JAMES DELGADO - LEARNING NOTES:
        # layers is a Python list containing our three components.
        # *layers unpacks that list into separate arguments.
        # Sequential connects the components in their listed order:
```

```
        # each component's output becomes the next component's input.
        self.model = nn.Sequential(*layers)

        # JAMES DELGADO - LEARNING NOTES:
        # This message appears when a DownBlock instance is created.
        # Defining the class alone does not execute __init__.
        print(f"Created DownBlock: in_chs={in_chs}, out_chs={out_chs},␣
↪spatial_reduction=2x")

    def forward(self, x):
        """
        Forward pass through the DownBlock.

        Args:
            x (torch.Tensor): Input tensor of shape [B, in_chs, H, W]

        Returns:
            torch.Tensor: Output tensor of shape [B, out_chs, H/2, W/2]
        """

        # JAMES DELGADO - LEARNING NOTES:
        # Pass the incoming batch through all three components.
        # Return the smaller feature maps for the next network stage.
        # Height and width must both be even for the rearrangement.
        return self.model(x)
```

[ ]:
```
#Now let's implement the upsampling block for our U-Net architecture:
class UpBlock(nn.Module):
    """
    Upsampling block for decoding path in U-Net architecture.

    This block:
    1. Takes features from the decoding path and corresponding skip connection
    2. Concatenates them along the channel dimension
    3. Upsamples spatial dimensions by 2x using transposed convolution
    4. Processes features through multiple convolutional blocks

    Args:
        in_chs (int): Number of input channels from the previous layer
        out_chs (int): Number of output channels
        group_size (int): Number of groups for GroupNorm
    """
    def __init__(self, in_chs, out_chs, group_size):
        super().__init__()

        # Your code to create the upsampling operation
        # Hint: Use nn.ConvTranspose2d with kernel_size=2 and stride=2
```

```
        # Note that the input channels will be 2 * in_chs due to concatenation

        # Enter your code here:

                # --- BEGIN STUDENT WORK: JAMES DELGADO ---

        # Joining x and skip doubles the number of input channels.
        # This layer learns to enlarge the feature maps, doubling
        # height and width while producing out_chs channels.
        self.up = nn.ConvTranspose2d(
            in_channels=2 * in_chs,
            out_channels=out_chs,
            kernel_size=2,
            stride=2
        )

        # --- END STUDENT WORK: JAMES DELGADO ---

        # Your code to create the convolutional blocks
        # Hint: Use multiple GELUConvBlocks in sequence

        # Enter your code here:

                # --- BEGIN STUDENT WORK: JAMES DELGADO ---

        # Reuse two GELUConvBlocks to process the enlarged features.
        # Both preserve the number of channels, height, and width.
        # Sequential passes the first block's output into the second.
        self.conv = nn.Sequential(
            GELUConvBlock(out_chs, out_chs, group_size),
            GELUConvBlock(out_chs, out_chs, group_size)
        )

        # --- END STUDENT WORK: JAMES DELGADO ---



        # Log the configuration for debugging
        print(f"Created UpBlock: in_chs={in_chs}, out_chs={out_chs},␣
↪spatial_increase=2x")

    def forward(self, x, skip):
        """
        Forward pass through the UpBlock.

        Args:
            x (torch.Tensor): Input tensor from previous layer [B, in_chs, H, W]
```

```
            skip (torch.Tensor): Skip connection tensor from encoder [B,␣
↪in_chs, 2H, 2W]

        Returns:
            torch.Tensor: Output tensor with shape [B, out_chs, 2H, 2W]
        """
        # Your code for the forward pass
        # Hint: Concatenate x and skip, then upsample and process

        # Enter your code here:

                # --- BEGIN STUDENT WORK: JAMES DELGADO ---

        # x and skip must have matching batch size, height, and width.
        # dim=1 joins their channels because our tensor dimensions
        # are [batch, channels, height, width], numbered from zero.
        x = torch.cat((x, skip), dim=1)

        # Double height and width using the upsampling layer.
        x = self.up(x)

        # Process the enlarged features and return the result.
        x = self.conv(x)
        return x

        # --- END STUDENT WORK: JAMES DELGADO ---
```

[ ]:
```
# Here we implement the time embedding block for our U-Net architecture:
# Helps the model understand time steps in diffusion process
class SinusoidalPositionEmbedBlock(nn.Module):
    """
    Creates sinusoidal embeddings for time steps in diffusion process.

    This embedding scheme is adapted from the Transformer architecture and
    provides a unique representation for each time step that preserves
    relative distance information.

    Args:
        dim (int): Embedding dimension
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, time):
        """
        Computes sinusoidal embeddings for given time steps.
```

```
        Args:
            time (torch.Tensor): Time steps tensor of shape [batch_size]

        Returns:
            torch.Tensor: Time embeddings of shape [batch_size, dim]
        """
        device = time.device
        half_dim = self.dim // 2
        embeddings = torch.log(torch.tensor(10000.0, device=device)) /␣
↪(half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) *␣
↪-embeddings)
        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings
```

[ ]:
```
# Helps the model understand which digit to generate (class conditioning)
class EmbedBlock(nn.Module):
    """
    Creates learned embeddings for class conditioning in diffusion models.

    Transforms floating-point one-hot class vectors into embeddings
    that can be added to feature maps.

    Args:
        input_dim (int): Input dimension (typically number of classes)
        emb_dim (int): Output embedding dimension
    """
    def __init__(self, input_dim, emb_dim):
        super(EmbedBlock, self).__init__()
        self.input_dim = input_dim

        # Your code to create the embedding layers
        # Hint: Use nn.Linear layers with a GELU activation, followed by
        # nn.Unflatten to reshape for broadcasting with feature maps

        # Enter your code here:

        # --- BEGIN STUDENT WORK: JAMES DELGADO ---

        # Convert each class vector into a learned embedding.
        # Linear layers use trainable weights and biases.
        # GELU adds a nonlinear transformation between the two layers.
        self.model = nn.Sequential(
            nn.Linear(input_dim, emb_dim),
            nn.GELU(),
```

```
            nn.Linear(emb_dim, emb_dim),

            # Reshape [batch_size, emb_dim] into
            # [batch_size, emb_dim, 1, 1].
            # This changes the shape without changing the values.
            nn.Unflatten(dim=1, unflattened_size=(emb_dim, 1, 1))
        )

        # --- END STUDENT WORK: JAMES DELGADO ---

    def forward(self, x):
        """
        Computes class embeddings from floating-point one-hot vectors.

        Args:
            x (torch.Tensor): Class vectors [batch_size, input_dim]

        Returns:
            torch.Tensor: Class embeddings [batch_size, emb_dim, 1, 1]
        """

        # JAMES DELGADO - LEARNING NOTES:
        # CORRECTION: Reshaping does not convert integer digit labels
        # into one-hot vectors. That conversion must happen earlier.
        # -1 asks PyTorch to infer the batch dimension from the
        # number of values, keeping input_dim values in each row.
        x = x.view(-1, self.input_dim)

        # Apply the embedding layers and return the reshaped result.
        return self.model(x)
```

[ ]:
```
# Main U-Net model that puts everything together
class UNet(nn.Module):
    """
    U-Net architecture for diffusion models with time and class conditioning.

    This architecture uses:
    1. A downsampling path that reduces spatial dimensions
    2. Middle processing blocks
    3. An upsampling path that restores spatial dimensions
    4. Skip connections carrying saved encoder features to the decoder

    The model is conditioned on:
    - Time step: position in the diffusion process
    - Class label: the digit we want to generate

    Args:
```

```
        T (int): Number of diffusion time steps
        img_ch (int): Number of image channels
        img_size (int): Height and width of the square input image
        down_chs (list or tuple): Channels at each spatial level
        t_embed_dim (int): Size of the initial sinusoidal time embedding
        c_embed_dim (int): Size of the input class vector, 10 for MNIST
    """
    def __init__(self, T, img_ch, img_size, down_chs, t_embed_dim, c_embed_dim):
        super().__init__()

        # --- BEGIN STUDENT WORK: JAMES DELGADO ---

        # Store the number of diffusion steps as model configuration.
        # The noise schedule itself is created outside this class.
        self.T = T

        # Use 8 groups for GroupNorm.
        # Our planned channel counts (32, 64, 128) all divide by 8.
        group_size = 8

        # The last channel count belongs to the smallest feature maps.
        # Both conditioning embeddings must match this channel count.
        middle_chs = down_chs[-1]

        # Each transition between channel levels halves image size.
        # Three levels require divisibility by 2**2 = 4.
        reduction_factor = 2 ** (len(down_chs) - 1)
        if img_size % reduction_factor != 0:
            raise ValueError(
                f"Image size {img_size} must be divisible by "
                f"{reduction_factor} for these downsampling levels."
            )

        # Our sinusoidal block divides its output between sine and cosine.
        if t_embed_dim < 4 or t_embed_dim % 2 != 0:
            raise ValueError("t_embed_dim must be even and at least 4.")

        # --- END STUDENT WORK: JAMES DELGADO ---

        # Your code to create the time embedding
        # Hint: Use SinusoidalPositionEmbedBlock, nn.Linear, and nn.GELU in␣
↪sequence

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # First represent the timestep with fixed sine/cosine values.
```

```
        # Then learn a transformation to match the middle feature channels.
        # With our settings: [B] -> [B, 8] -> [B, 128].
        self.time_embed = nn.Sequential(
            SinusoidalPositionEmbedBlock(t_embed_dim),
            nn.Linear(t_embed_dim, middle_chs),
            nn.GELU()
        )

        # Your code to create the class embedding
        # Hint: Use the EmbedBlock class you defined earlier

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # c_embed_dim is the INPUT class-vector size: 10 for MNIST.
        # EmbedBlock produces [B, middle_chs, 1, 1] for addition later.
        self.class_embed = EmbedBlock(c_embed_dim, middle_chs)

        # Your code to create the initial convolution
        # Hint: Use GELUConvBlock to process the input image

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Convert the image channel into our first set of feature maps.
        # For MNIST: [B, 1, 28, 28] -> [B, 32, 28, 28].
        self.initial_conv = GELUConvBlock(
            img_ch, down_chs[0], group_size
        )

        # Your code to create the downsampling path
        # Hint: Use nn.ModuleList with DownBlock for each level

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Create one DownBlock for each pair of neighboring levels.
        # For (32, 64, 128), the pairs are 32->64 and 64->128.
        # ModuleList registers these blocks as parts of our model.
        self.down_blocks = nn.ModuleList()

        for i in range(len(down_chs) - 1):
            self.down_blocks.append(
                DownBlock(down_chs[i], down_chs[i + 1], group_size)
            )

        # Your code to create the middle blocks
```

```
        # Hint: Use GELUConvBlock twice to process features at lowest resolution

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Process the smallest feature maps without shrinking them further.
        # For our configuration, both blocks keep [B, 128, 7, 7].
        self.middle = nn.Sequential(
            GELUConvBlock(middle_chs, middle_chs, group_size),
            GELUConvBlock(middle_chs, middle_chs, group_size)
        )

        # Your code to create the upsampling path
        # Hint: Use nn.ModuleList with UpBlock for each level (in reverse order)

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Walk backward through the channel levels: 128->64, then 64->32.
        # range starts at the last index and decreases by 1, stopping before 0.
        self.up_blocks = nn.ModuleList()

        for i in range(len(down_chs) - 1, 0, -1):
            self.up_blocks.append(
                UpBlock(down_chs[i], down_chs[i - 1], group_size)
            )

        # Your code to create the final convolution
        # Hint: Use nn.Conv2d to project back to the original image channels

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # A 1-by-1 convolution combines the feature channels at each position.
        # For MNIST, 32 features become one predicted noise value per pixel.
        # No final activation restricts the range of predicted noise values.
        self.final_conv = nn.Conv2d(
            in_channels=down_chs[0],
            out_channels=img_ch,
            kernel_size=1
        )

        print(f"Created UNet with {len(down_chs)} scale levels")
        print(f"Channel dimensions: {down_chs}")

    def forward(self, x, t, c, c_mask):
        """
```

```
        Forward pass through the U-Net.

        Args:
            x (torch.Tensor): Noisy images [B, img_ch, H, W]
            t (torch.Tensor): Diffusion timestep indices [B]
            c (torch.Tensor): Floating-point one-hot labels [B, c_embed_dim]
            c_mask (torch.Tensor): Class-conditioning mask [B, 1];
                                   1 keeps the class embedding, 0 removes it

        Returns:
            torch.Tensor: Predicted noise [B, img_ch, H, W]
        """

        # Your code for the time embedding
        # Hint: Process the time steps through the time embedding module

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Represent each timestep, then add two spatial dimensions.
        # [B, middle_chs] becomes [B, middle_chs, 1, 1].
        time_embedding = self.time_embed(t.float())
        time_embedding = time_embedding[:, :, None, None]

        # Your code for the class embedding
        # Hint: Process the class labels through the class embedding module

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # The training code supplies one-hot labels.
        # EmbedBlock already returns [B, middle_chs, 1, 1].
        class_embedding = self.class_embed(c.float())

        # Reshape the per-image mask to apply across embedding channels.
        # Multiplying by 1 keeps the embedding; multiplying by 0 removes it.
        mask = c_mask.reshape(x.shape[0], 1, 1, 1).float()
        class_embedding = class_embedding * mask

        # Your code for the initial feature extraction
        # Hint: Apply initial convolution to the input

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Begin converting noisy pixel values into image features.
        x = self.initial_conv(x)
```

```
        # Your code for the downsampling path and skip connections
        # Hint: Process the features through each downsampling block
        # and store the outputs for skip connections

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Save each output AFTER downsampling.
        # These saved tensors will match the decoder's input sizes
        # when our UpBlocks concatenate before upsampling.
        skips = []

        for down_block in self.down_blocks:
            x = down_block(x)
            skips.append(x)

        # Your code for the middle processing and conditioning
        # Hint: Process features through middle blocks, then add time and class␣
↪embeddings

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Process image features at the smallest spatial size.
        x = self.middle(x)

        # Add information about the timestep and requested digit.
        # The 1-by-1 embeddings broadcast across height and width.
        x = x + time_embedding + class_embedding

        # Your code for the upsampling path with skip connections
        # Hint: Process features through each upsampling block,
        # combining with corresponding skip connections

        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # pop() retrieves and removes the last saved tensor first.
        # This pairs the smallest encoder features with the first UpBlock.
        # Each UpBlock joins matching features, then doubles their size.
        for up_block in self.up_blocks:
            skip = skips.pop()
            x = up_block(x, skip)

        # Your code for the final projection
        # Hint: Apply the final convolution to get output in image space
```

```
        # Enter your code here:

        # JAMES DELGADO - STUDENT WORK:
        # Return one predicted noise value per input pixel and channel.
        # The output shape matches the noisy image's input shape.
        return self.final_conv(x)
```

1.5 Step 4: Setting Up The Diffusion Process

Now we'll create the process of adding and removing noise from images. Think of it like: 1. Adding fog: Slowly making the image more and more blurry until you can't see it 2. Removing fog: Teaching the AI to gradually make the image clearer 3. Controlling the process: Making sure we can generate specific numbers we want

[ ]:
```
# Set up the noise schedule
n_steps = 100  # How many steps to go from clear image to noise
beta_start = 0.0001  # Starting noise level (small)
beta_end = 0.02      # Ending noise level (larger)

# Create schedule of gradually increasing noise levels
beta = torch.linspace(beta_start, beta_end, n_steps).to(device)

# Calculate important values used in diffusion equations
alpha = 1 - beta  # Portion of original image to keep at each step
alpha_bar = torch.cumprod(alpha, dim=0)  # Cumulative product of alphas
sqrt_alpha_bar = torch.sqrt(alpha_bar)  # For scaling the original image
sqrt_one_minus_alpha_bar = torch.sqrt(1 - alpha_bar)  # For scaling the noise
```

[ ]:
```
# Function to add noise to images (forward diffusion process)
def add_noise(x_0, t):
    """
    Add noise to images according to the forward diffusion process.

    The formula is: x_t = √( _bar_t) * x_0 + √(1- _bar_t) *
    where   is random noise and _bar_t is the cumulative product of (1- ).

    Args:
        x_0 (torch.Tensor): Original clean image [B, C, H, W]
        t (torch.Tensor): Timestep indices indicating noise level [B]

    Returns:
        tuple: (noisy_image, noise_added)
            - noisy_image is the image with noise added
            - noise_added is the actual noise that was added (for training)
    """
    # Create random Gaussian noise with same shape as image
```

```
    noise = torch.randn_like(x_0)

    # Get noise schedule values for the specified timesteps
    # Reshape to allow broadcasting with image dimensions
    sqrt_alpha_bar_t = sqrt_alpha_bar[t].reshape(-1, 1, 1, 1)
    sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1,␣
↪1)

    # Apply the forward diffusion equation:
    # Mixture of original image (scaled down) and noise (scaled up)     # Your␣
↪code to apply the forward diffusion equation
    # Hint: Mix the original image and noise according to the noise schedule

    # Enter your code here:

        # --- BEGIN STUDENT WORK: JAMES DELGADO ---

    # Scale the clean image and random noise using the coefficients
    # selected from our noise schedule, then add them together.
    # Each image can have its own timestep and therefore its own
    # pair of coefficients.
    x_t = (
        sqrt_alpha_bar_t * x_0
        + sqrt_one_minus_alpha_bar_t * noise
    )

    # Keep noise unchanged because it is the training target.
    # The scaled noise contribution is already included in x_t.
    # Do not clip x_t: noisy values can extend outside [-1, 1].

    # --- END STUDENT WORK: JAMES DELGADO ---




    return x_t, noise
```

[ ]:
```
# Function to remove noise from images (reverse diffusion process)
@torch.no_grad()
def remove_noise(x_t, t, model, c, c_mask):
    """
    Remove noise from images using the learned reverse diffusion process.
    """
    # Predict the noise in the image using our model
    predicted_noise = model(x_t, t, c, c_mask)

    # Get noise schedule values for the current timestep
```

```
    alpha_t = alpha[t].reshape(-1, 1, 1, 1)
    alpha_bar_t = alpha_bar[t].reshape(-1, 1, 1, 1)
    beta_t = beta[t].reshape(-1, 1, 1, 1)

    # Define the missing scale factor from the global schedule
    sqrt_one_minus_alpha_bar_t = sqrt_one_minus_alpha_bar[t].reshape(-1, 1, 1,␣
↪1)

    # Special case: if we're at the first timestep (t=0), we're done
    if t[0] == 0:
        return x_t
    else:
        # Calculate the mean of the denoised distribution
        mean = (1 / torch.sqrt(alpha_t)) * (
            x_t - (beta_t / sqrt_one_minus_alpha_bar_t) * predicted_noise
        )

        # Add a small amount of random noise (variance depends on timestep)
        noise = torch.randn_like(x_t)

        # Return the partially denoised image with a bit of new random noise
        return mean + torch.sqrt(beta_t) * noise
```

[ ]:
```
# Visualization function to inspect images at different noise levels
def show_noise_progression(image, num_steps=5):
    """
    Display a clean image and noisy versions at selected timesteps.

    Args:
        image (torch.Tensor): Original clean image [C, H, W]
        num_steps (int): Total panels, including the original image
    """

    # JAMES DELGADO - STUDENT WORK:
    # We need at least one original panel and one noisy panel.
    if num_steps < 2:
        raise ValueError("Use at least 2 panels.")

    plt.figure(figsize=(15, 3))

    # Show original image
    plt.subplot(1, num_steps, 1)

    if IMG_CH == 1:
        # JAMES DELGADO - CORRECTION:
        # Keep the same brightness scale across all grayscale panels.
        # These display limits do not change the underlying tensor.
```

```
        plt.imshow(image[0].cpu(), cmap="gray", vmin=-1, vmax=1)
    else:
        # JAMES DELGADO - LEARNING NOTES:
        # Convert [C, H, W] to [H, W, C] for displaying color images.
        # Our preprocessing used [-1, 1], so rescale for display.
        # Clipping affects this display copy only.
        img = image.permute(1, 2, 0).cpu()
        img = ((img + 1) / 2).clamp(0, 1)
        plt.imshow(img)

    plt.title("Original")
    plt.axis("off")

    # Show noisy versions at selected timesteps
    for i in range(1, num_steps):

        # JAMES DELGADO - CORRECTION:
        # Indexing starts at 0, so the final timestep is n_steps - 1.
        # Dividing by num_steps - 1 ensures the last panel reaches it.
        t_idx = int((i / (num_steps - 1)) * (n_steps - 1))
        t = torch.tensor([t_idx], dtype=torch.long, device=image.device)

        # JAMES DELGADO - LEARNING NOTES:
        # unsqueeze(0) adds a batch dimension:
        # [C, H, W] becomes [1, C, H, W].
        # Each call starts from the same clean image and draws fresh noise.
        # _ means we are ignoring the returned training-target noise here.
        noisy_image, _ = add_noise(image.unsqueeze(0), t)

        plt.subplot(1, num_steps, i + 1)

        if IMG_CH == 1:
            # Select the first image and its grayscale channel.
            # Use the same display range as the original image.
            plt.imshow(
                noisy_image[0, 0].cpu(),
                cmap="gray",
                vmin=-1,
                vmax=1
            )
        else:
            img = noisy_image[0].permute(1, 2, 0).cpu()
            img = ((img + 1) / 2).clamp(0, 1)
            plt.imshow(img)

        # JAMES DELGADO - CORRECTION:
        # A timestep identifies a schedule position, not a noise percentage.
```

```
        plt.title(f"Timestep {t_idx}")
        plt.axis("off")

    plt.tight_layout()
    plt.show()


# Get one batch, then select its first image.
sample_batch = next(iter(train_dataloader))
sample_image = sample_batch[0][0].to(device)

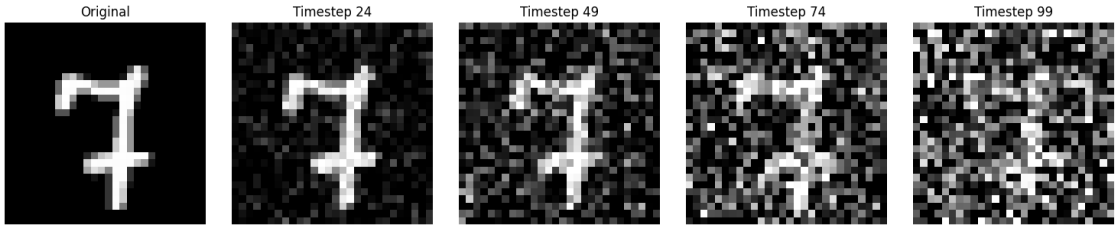
# Calling the function produces the plot.
show_noise_progression(sample_image)

# --- BEGIN STUDENT WORK: JAMES DELGADO ---

# Inspect the schedule's final multipliers.
# These tell us how strongly the clean image and noise contribute
# at the final timestep. They are not percentages.
print(f"Final clean-image multiplier: {sqrt_alpha_bar[-1].item():.4f}")
print(f"Final noise multiplier: {sqrt_one_minus_alpha_bar[-1].item():.4f}")

# --- END STUDENT WORK: JAMES DELGADO ---


# Student Activity: Try different noise schedules
# Keep this optional experiment commented out for now.
#
# Creating these alternative variables alone does not change add_noise:
# that function uses sqrt_alpha_bar and sqrt_one_minus_alpha_bar.
#
# beta_alt = torch.linspace(beta_start, beta_end, n_steps)**2
# alpha_alt = 1 - beta_alt
# alpha_bar_alt = torch.cumprod(alpha_alt, dim=0)
# How would this affect the diffusion process?
```




```
Final clean-image multiplier: 0.6030
Final noise multiplier: 0.7978
```

1.6 Step 5: Training Our Model

Now we'll teach our AI to generate images. This process: 1. Takes a clear image 2. Adds random noise to it 3. Asks our AI to predict what noise was added 4. Helps our AI learn from its mistakes

This will take a while, but we'll see progress as it learns!

[ ]:
```
# Create our model and move it to GPU if available
model = UNet(
    T=n_steps,                  # Number of diffusion time steps
    img_ch=IMG_CH,              # Number of channels in our images (1 for␣
 ↪grayscale, 3 for RGB)
    img_size=IMG_SIZE,          # Size of input images (28 for MNIST, 32 for␣
 ↪CIFAR-10)
    down_chs=(32, 64, 128),     # Channel dimensions for each downsampling level
    t_embed_dim=8,              # Dimension for time step embeddings
    c_embed_dim=N_CLASSES       # Number of classes for conditioning
).to(device)

# Print model summary
print(f"\n{'='*50}")
print(f"MODEL ARCHITECTURE SUMMARY")
print(f"{'='*50}")
print(f"Input resolution: {IMG_SIZE}x{IMG_SIZE}")
print(f"Input channels: {IMG_CH}")
print(f"Time steps: {n_steps}")
print(f"Condition classes: {N_CLASSES}")
print(f"GPU acceleration: {'Yes' if device.type == 'cuda' else 'No'}")

# Validate model parameters and estimate memory requirements
# Hint: Create functions to count parameters and estimate memory usage

# Enter your code here:

# --- BEGIN STUDENT WORK: JAMES DELGADO ---

def inspect_model_size(model):
    # numel() counts individual numbers in each parameter tensor.
    # requires_grad identifies parameters eligible for learning.
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(
        p.numel() for p in model.parameters() if p.requires_grad
    )

    # element_size() gives the bytes used by each stored number.
    parameter_bytes = sum(
        p.numel() * p.element_size() for p in model.parameters()
    )
```

```
    trainable_bytes = sum(
        p.numel() * p.element_size()
        for p in model.parameters() if p.requires_grad
    )

    # Estimate storage for parameters, gradients, and Adam's two
    # running statistics, assuming their data types match the parameters.
    # This is NOT an estimate of total GPU memory required for training.
    estimated_bytes = parameter_bytes + 3 * trainable_bytes

    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")
    print(f"Parameter storage: {parameter_bytes / 1e6:.2f} MB")
    print(
        "Estimated parameters + gradients + Adam statistics: "
        f"{estimated_bytes / 1e6:.2f} MB"
    )
    print("Estimate excludes activations, temporary tensors, and runtime␣
 ↪overhead.")
    print(f"Actual model device: {next(model.parameters()).device}")


inspect_model_size(model)

# --- END STUDENT WORK: JAMES DELGADO ---

# Your code to verify data ranges and integrity
# Hint: Create functions to check data ranges in training and validation data

# Enter your code here:

# --- BEGIN STUDENT WORK: ESTHER ---

def inspect_data_batch(dataloader, name):
    # Inspect one batch, not every example in the dataset.
    images, labels = next(iter(dataloader))

    minimum = images.min().item()
    maximum = images.max().item()
    finite = torch.isfinite(images).all().item()

    print(f"\n{name} batch:")
    print(f"Image shape: {tuple(images.shape)}")
    print(f"Image data type: {images.dtype}")
    print(f"Pixel range: {minimum:.3f} to {maximum:.3f}")
    print(f"Label shape: {tuple(labels.shape)}")
    print(f"Label data type: {labels.dtype}")
```

```
    print(f"All image values finite: {finite}")

    # Our clean images should match the configuration and contain
    # valid floating-point values in the normalized range.
    if images.ndim != 4 or tuple(images.shape[1:]) != (
        IMG_CH, IMG_SIZE, IMG_SIZE
    ):
        raise ValueError(f"{name}: unexpected image shape.")

    if images.dtype != torch.float32 or not finite:
        raise ValueError(f"{name}: invalid image data type or values.")

    # This range check applies to CLEAN images.
    # Noisy images are allowed to extend outside [-1, 1].
    if minimum < -1.0001 or maximum > 1.0001:
        raise ValueError(f"{name}: clean images are outside the expected range.
 ↪")

    if labels.shape != (images.shape[0],) or labels.dtype != torch.int64:
        raise ValueError(f"{name}: unexpected label shape or data type.")

    if labels.min().item() < 0 or labels.max().item() >= N_CLASSES:
        raise ValueError(f"{name}: a label is outside the class range.")

    return images, labels


check_images, check_labels = inspect_data_batch(
    train_dataloader, "Training"
)
inspect_data_batch(val_dataloader, "Validation")

# Use two images to check the full network at two noise levels:
# the first and last timesteps in our schedule.
test_images = check_images[:2].to(device)
test_t = torch.tensor(
    [0, n_steps - 1], dtype=torch.long, device=device
)

# Convert integer digit labels into the floating-point one-hot
# vectors expected by our class embedding.
test_classes = F.one_hot(
    check_labels[:2], num_classes=N_CLASSES
).float().to(device)

# A value of 1 keeps class conditioning for each image.
test_mask = torch.ones(2, 1, device=device)
```

```
# no_grad() avoids storing the calculations needed for backpropagation.
# This check makes predictions but does not update model weights.
with torch.no_grad():
    test_noisy_images, _ = add_noise(test_images, test_t)
    test_prediction = model(
        test_noisy_images, test_t, test_classes, test_mask
    )

# We need one predicted noise value per input pixel and channel.
if test_prediction.shape != test_images.shape:
    raise ValueError("The model output shape does not match the input.")

if not torch.isfinite(test_prediction).all().item():
    raise ValueError("The model produced invalid numerical values.")

print(
    "\nForward-pass check passed. Output shape:",
    tuple(test_prediction.shape)
)

# --- END STUDENT WORK: ESTHER ---


# Set up the optimizer with parameters tuned for diffusion models
# Note: Lower learning rates tend to work better for diffusion models
initial_lr = 0.001  # Starting learning rate
weight_decay = 1e-5  # L2 regularization to prevent overfitting

optimizer = Adam(
    model.parameters(),
    lr=initial_lr,
    weight_decay=weight_decay
)

# Learning rate scheduler to reduce LR when validation loss plateaus
# This helps fine-tune the model toward the end of training

# --- BEGIN STUDENT WORK: ESTHER ---

# The scheduler monitors validation loss during training.
# If improvement stalls beyond its patience, it halves the learning rate.
# For example: 0.001 * 0.5 = 0.0005.
#
# CORRECTION: Removed verbose=True for compatibility with current PyTorch.
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
```

```
    mode="min",       # Lower validation loss is better.
    factor=0.5,       # Multiply the current learning rate by one-half.
    patience=5,       # Allow 5 epochs without sufficient improvement.
    min_lr=1e-6       # Do not reduce the learning rate below 0.000001.
)

# --- END STUDENT WORK: ESTHER ---

# STUDENT EXPERIMENT:
# Try different channel configurations and see how they affect:
# 1. Model size (parameter count)
# 2. Training time
# 3. Generated image quality
#
# Suggestions:
# - Smaller: down_chs=(16, 32, 64)
# - Larger: down_chs=(64, 128, 256, 512)
```

```
Created DownBlock: in_chs=32, out_chs=64, spatial_reduction=2x
Created DownBlock: in_chs=64, out_chs=128, spatial_reduction=2x
Created UpBlock: in_chs=128, out_chs=64, spatial_increase=2x
Created UpBlock: in_chs=64, out_chs=32, spatial_increase=2x
Created UNet with 3 scale levels
Channel dimensions: (32, 64, 128)

==================================================
MODEL ARCHITECTURE SUMMARY
==================================================
Input resolution: 28x28
Input channels: 1
Time steps: 100
Condition classes: 10
GPU acceleration: Yes
Total parameters: 1,505,409
Trainable parameters: 1,505,409
Parameter storage: 6.02 MB
Estimated parameters + gradients + Adam statistics: 24.09 MB
Estimate excludes activations, temporary tensors, and runtime overhead.
Actual model device: cuda:0

Training batch:
Image shape: (64, 1, 28, 28)
Image data type: torch.float32
Pixel range: -1.000 to 1.000
Label shape: (64,)
Label data type: torch.int64
All image values finite: True
```

```
Validation batch:
Image shape: (64, 1, 28, 28)
Image data type: torch.float32
Pixel range: -1.000 to 1.000
Label shape: (64,)
Label data type: torch.int64
All image values finite: True

Forward-pass check passed. Output shape: (2, 1, 28, 28)
```

[ ]:
```python
# Define helper functions needed for training and evaluation
def validate_model_parameters(model):
    """
    Counts model parameters and estimates memory usage.
    """
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.
 ↪requires_grad)

    print(f"Total parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")

    # Estimate memory requirements (very approximate)
    param_memory = total_params * 4 / (1024 ** 2)  # MB for params (float32)
    grad_memory = trainable_params * 4 / (1024 ** 2)  # MB for gradients
    buffer_memory = param_memory * 2  # Optimizer state, forward activations,␣
 ↪etc.

    print(f"Estimated GPU memory usage: {param_memory + grad_memory +␣
 ↪buffer_memory:.1f} MB")

# Define helper functions for verifying data ranges
def verify_data_range(dataloader, name="Dataset"):
    """
    Verifies the range and integrity of the data.
    """
    batch = next(iter(dataloader))[0]
    print(f"\n{name} range check:")
    print(f"Shape: {batch.shape}")
    print(f"Data type: {batch.dtype}")
    print(f"Min value: {batch.min().item():.2f}")
    print(f"Max value: {batch.max().item():.2f}")
    print(f"Contains NaN: {torch.isnan(batch).any().item()}")
    print(f"Contains Inf: {torch.isinf(batch).any().item()}")

# Define helper functions for generating samples during training
def generate_samples(model, n_samples=10):
```

```
    """
    Generates sample images using the model for visualization during training.
    """
    model.eval()
    with torch.no_grad():
        # Generate digits 0-9 for visualization
        samples = []
        for digit in range(min(n_samples, 10)):
            # Start with random noise
            x = torch.randn(1, IMG_CH, IMG_SIZE, IMG_SIZE).to(device)

            # Set up conditioning for the digit
            c = torch.tensor([digit]).to(device)
            c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)
            c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

            # Remove noise step by step
            for t in range(n_steps-1, -1, -1):
                t_batch = torch.full((1,), t).to(device)
                x = remove_noise(x, t_batch, model, c_one_hot, c_mask)

            samples.append(x)

        # Combine samples and display
        samples = torch.cat(samples, dim=0)
        grid = make_grid(samples, nrow=min(n_samples, 5), normalize=True)

        plt.figure(figsize=(10, 4))

        # Display based on channel configuration
        if IMG_CH == 1:
            plt.imshow(grid[0].cpu(), cmap='gray')
        else:
            plt.imshow(grid.permute(1, 2, 0).cpu())

        plt.axis('off')
        plt.title('Generated Samples')
        plt.show()

# Define helper functions for safely saving models
def safe_save_model(model, path, optimizer=None, epoch=None, best_loss=None):
    """
    Safely saves model with error handling and backup.
    """
    try:
        # Create a dictionary with all the elements to save
        save_dict = {
```

```
            'model_state_dict': model.state_dict(),
        }

        # Add optional elements if provided
        if optimizer is not None:
            save_dict['optimizer_state_dict'] = optimizer.state_dict()
        if epoch is not None:
            save_dict['epoch'] = epoch
        if best_loss is not None:
            save_dict['best_loss'] = best_loss

        # Create a backup of previous checkpoint if it exists
        if os.path.exists(path):
            backup_path = path + '.backup'
            try:
                os.replace(path, backup_path)
                print(f"Created backup at {backup_path}")
            except Exception as e:
                print(f"Warning: Could not create backup - {e}")

        # Save the new checkpoint
        torch.save(save_dict, path)
        print(f"Model successfully saved to {path}")

    except Exception as e:
        print(f"Error saving model: {e}")
        print("Attempting emergency save...")

        try:
            emergency_path = path + '.emergency'
            torch.save(model.state_dict(), emergency_path)
            print(f"Emergency save successful: {emergency_path}")
        except:
            print("Emergency save failed. Could not save model.")
```

[ ]:
```
#  Implementation of the training step function
def train_step(x, c):
    """
    Performs a single training step for the diffusion model.

    This function:
    1. Prepares class conditioning
    2. Samples random timesteps for each image
    3. Adds corresponding noise to the images
    4. Asks the model to predict the noise
    5. Calculates the loss between predicted and actual noise
```

```
    Args:
        x (torch.Tensor): Batch of clean images [batch_size, channels, height,␣
↪width]
        c (torch.Tensor): Batch of class labels [batch_size]

    Returns:
        torch.Tensor: Mean squared error loss value
    """
    # Convert number labels to one-hot encoding for class conditioning
    # Example: Label 3 -> [0, 0, 0, 1, 0, 0, 0, 0, 0, 0] for MNIST
    c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

    # Create conditioning mask (all ones for standard training)
    # This would be used for classifier-free guidance if implemented
    c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

    # Pick random timesteps for each image in the batch
    # Different timesteps allow the model to learn the entire diffusion process
    t = torch.randint(0, n_steps, (x.shape[0],)).to(device)

    # Add noise to images according to the forward diffusion process
    # This simulates images at different stages of the diffusion process
    # Hint: Use the add_noise function you defined earlier

    # Enter your code here:

        # --- BEGIN STUDENT WORK: JAMES DELGADO ---

    # Create noisy images and keep the original noise as our
    # known target for evaluating the model's prediction.
    x_t, noise = add_noise(x, t)

    # --- END STUDENT WORK: JAMES DELGADO ---

    # The model tries to predict the exact noise that was added
    # This is the core learning objective of diffusion models
    predicted_noise = model(x_t, t, c_one_hot, c_mask)

    # Calculate loss: how accurately did the model predict the noise?
    # MSE loss works well for image-based diffusion models
    # Hint: Use F.mse_loss to compare predicted and actual noise

    # Enter your code here:

        # --- BEGIN STUDENT WORK: JAMES DELGADO ---

    # Mean Squared Error measures the average squared difference
```

```
    # between the model's predicted noise and our known noise.
    loss = F.mse_loss(predicted_noise, noise)

    # Return the loss tensor so the training loop can use it
    # to calculate gradients and update the model's weights.
    return loss

    # --- END STUDENT WORK: JAMES DELGADO ---
```

[ ]:
```
# Implementation of the main training loop

# Training configuration
early_stopping_patience = 10  # Number of epochs without improvement before␣
 ↪stopping
gradient_clip_value = 1.0     # Maximum gradient norm for stability
display_frequency = 100       # How often to show progress (in steps)
generate_frequency = 500      # How often to generate samples (in steps)

# Progress tracking variables
best_loss = float('inf')
train_losses = []
val_losses = []
no_improve_epochs = 0

# Training loop
print("\n" + "=" * 50)
print("STARTING TRAINING")
print("=" * 50)
model.train()

# Wrap the training loop in a try-except block for better error handling:
# Your code for the training loop
# Hint: Use a try-except block for better error handling
# Process each epoch and each batch, with validation after each epoch

# Enter your code here:

# JAMES DELGADO - STUDENT WORK:
# Activate the try block around the instructor's existing training loop.
# The matching except blocks below handle interruptions and errors.
try:
    for epoch in range(EPOCHS):
        print(f"\nEpoch {epoch + 1}/{EPOCHS}")
        print("-" * 20)

        # Training phase
        model.train()
```

```
        epoch_losses = []

        # Process each batch
        for step, (images, labels) in enumerate(train_dataloader):
            images = images.to(device)
            labels = labels.to(device)

            # Training step

            # JAMES DELGADO - LEARNING NOTE:
            # Clear old gradients before calculating this batch's gradients.
            optimizer.zero_grad()

            # JAMES DELGADO - LEARNING NOTE:
            # Our train_step function returns the noise-prediction error.
            loss = train_step(images, labels)

            # JAMES DELGADO - LEARNING NOTE:
            # Backpropagation calculates how each parameter affects the loss.
            loss.backward()

            # Add gradient clipping for stability
            torch.nn.utils.clip_grad_norm_(
                model.parameters(),
                max_norm=gradient_clip_value
            )

            # JAMES DELGADO - LEARNING NOTE:
            # The optimizer uses the gradients to change the model's weights.
            optimizer.step()

            # Store a plain number for progress tracking and plotting.
            epoch_losses.append(loss.item())

            # Show progress at regular intervals
            if step % display_frequency == 0:
                print(
                    f"  Step {step}/{len(train_dataloader)}, "
                    f"Loss: {loss.item():.4f}"
                )

                # Generate samples less frequently to save time
                if step % generate_frequency == 0 and step > 0:
                    print("  Generating samples...")
                    generate_samples(model, n_samples=5)

        # End of epoch - calculate average training loss
```

```
        avg_train_loss = sum(epoch_losses) / len(epoch_losses)
        train_losses.append(avg_train_loss)

        print(
            f"\nTraining - Epoch {epoch + 1} "
            f"average loss: {avg_train_loss:.4f}"
        )

        # Validation phase
        model.eval()
        val_epoch_losses = []
        print("Running validation...")

        # JAMES DELGADO - LEARNING NOTE:
        # Validation measures prediction error on held-out images.
        # We disable gradient tracking and do not update the weights.
        with torch.no_grad():
            for val_images, val_labels in val_dataloader:
                val_images = val_images.to(device)
                val_labels = val_labels.to(device)

                # Calculate validation loss
                val_loss = train_step(val_images, val_labels)
                val_epoch_losses.append(val_loss.item())

        # Calculate average validation loss
        avg_val_loss = sum(val_epoch_losses) / len(val_epoch_losses)
        val_losses.append(avg_val_loss)

        print(
            f"Validation - Epoch {epoch + 1} "
            f"average loss: {avg_val_loss:.4f}"
        )

        # Learning rate scheduling based on validation loss
        scheduler.step(avg_val_loss)
        current_lr = optimizer.param_groups[0]['lr']
        print(f"Learning rate: {current_lr:.6f}")

        # Generate samples after epochs 1, 3, 5, etc., and the final epoch
        if epoch % 2 == 0 or epoch == EPOCHS - 1:
            print("\nGenerating samples for visual progress check...")
            generate_samples(model, n_samples=10)

        # Save best model based on validation loss
        if avg_val_loss < best_loss:
            best_loss = avg_val_loss
```

```
            # Use safe_save_model instead of just saving state_dict
            safe_save_model(
                model,
                'best_diffusion_model.pt',
                optimizer,
                epoch,
                best_loss
            )

            print(f"  New best model saved! (Val Loss: {best_loss:.4f})")
            no_improve_epochs = 0
        else:
            no_improve_epochs += 1
            print(
                f"No improvement for "
                f"{no_improve_epochs}/{early_stopping_patience} epochs"
            )

        # Early stopping
        if no_improve_epochs >= early_stopping_patience:
            print(
                "\nEarly stopping triggered! "
                "No improvement in validation loss."
            )
            break

        # Plot loss curves every few epochs
        if epoch % 5 == 0 or epoch == EPOCHS - 1:
            plt.figure(figsize=(10, 5))
            plt.plot(train_losses, label='Training Loss')
            plt.plot(val_losses, label='Validation Loss')
            plt.xlabel('Epoch')
            plt.ylabel('Loss')
            plt.title('Training and Validation Loss')
            plt.legend()
            plt.grid(True)
            plt.show()

# --- BEGIN STUDENT WORK: ESTHER ---

except KeyboardInterrupt:
    # Handle a manual interruption, such as pressing the stop button.
    print("\nTraining interrupted before normal completion.")

    # Stop execution so the cell does not report training as complete.
    raise
```

```
except Exception as error:
    # Display the error type and message to help identify the problem.
    print(
        f"\nTraining stopped because of "
        f"{type(error).__name__}: {error}"
    )

    # Preserve the original traceback for debugging and stop execution.
    raise

# --- END STUDENT WORK: ESTHER ---


# Final wrap-up
print("\n" + "=" * 50)
print("TRAINING COMPLETE")
print("=" * 50)
print(f"Best validation loss: {best_loss:.4f}")

# Generate final samples
print("Generating final samples...")
generate_samples(model, n_samples=10)

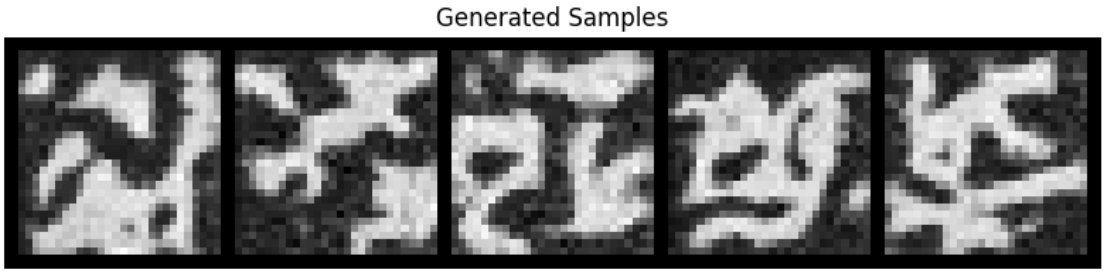
# Display final loss curves
plt.figure(figsize=(12, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

# Clean up memory
torch.cuda.empty_cache()
```

```
==================================================
STARTING TRAINING
==================================================

Epoch 1/30
--------------------
  Step 0/750, Loss: 1.1503
  Step 100/750, Loss: 0.1011
  Step 200/750, Loss: 0.1095
```

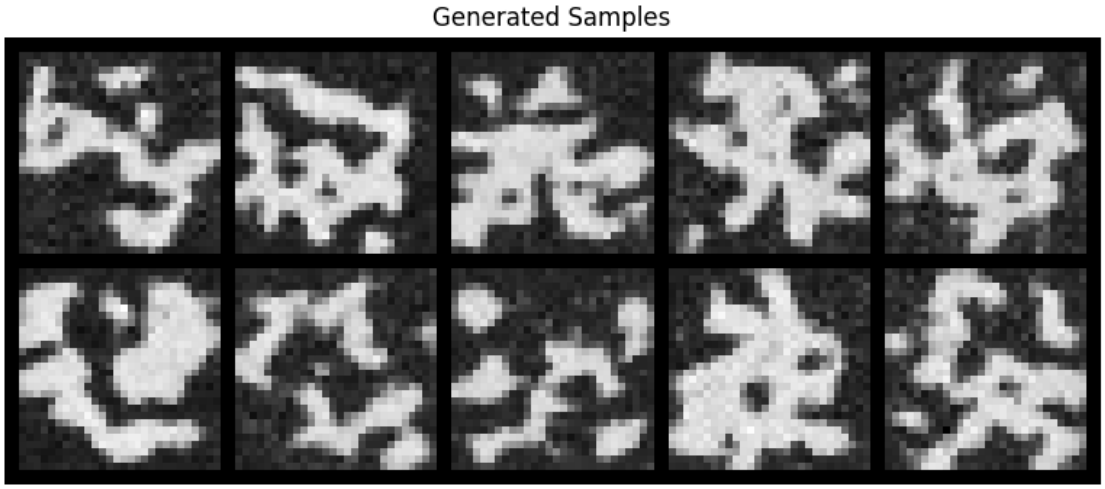
```
  Step 300/750, Loss: 0.0894
  Step 400/750, Loss: 0.0816
  Step 500/750, Loss: 0.0859
  Generating samples…
```



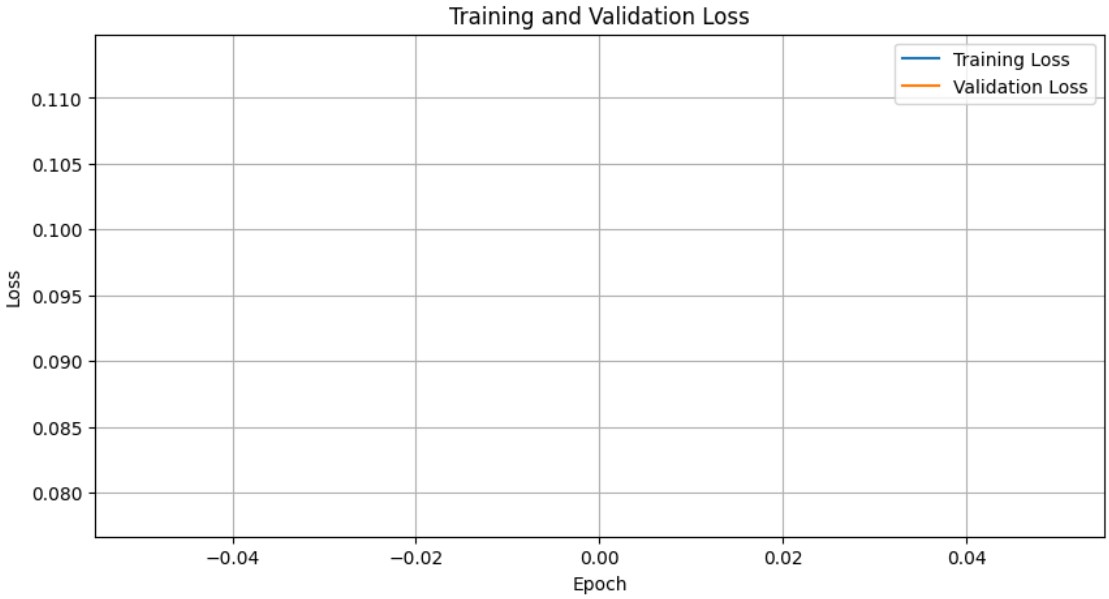

```
  Step 600/750, Loss: 0.0761
  Step 700/750, Loss: 0.0737

Training - Epoch 1 average loss: 0.1131
Running validation…
Validation - Epoch 1 average loss: 0.0784
Learning rate: 0.001000

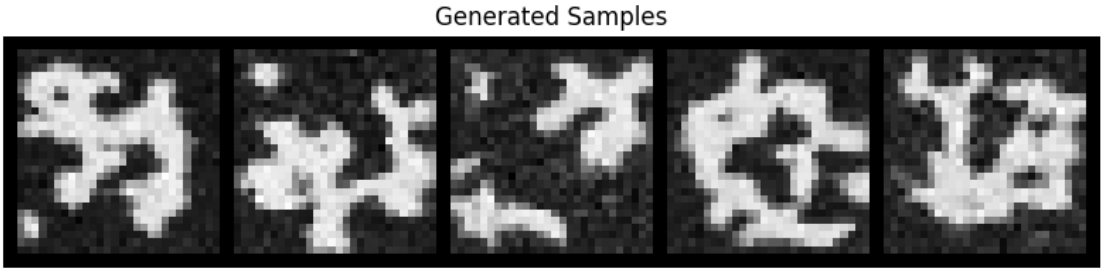
Generating samples for visual progress check…
```




```
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0784)
```




```
Epoch 2/30
--------------------
  Step 0/750, Loss: 0.0944
  Step 100/750, Loss: 0.0817
  Step 200/750, Loss: 0.0863
  Step 300/750, Loss: 0.0765
  Step 400/750, Loss: 0.0854
  Step 500/750, Loss: 0.0723
  Generating samples…
```

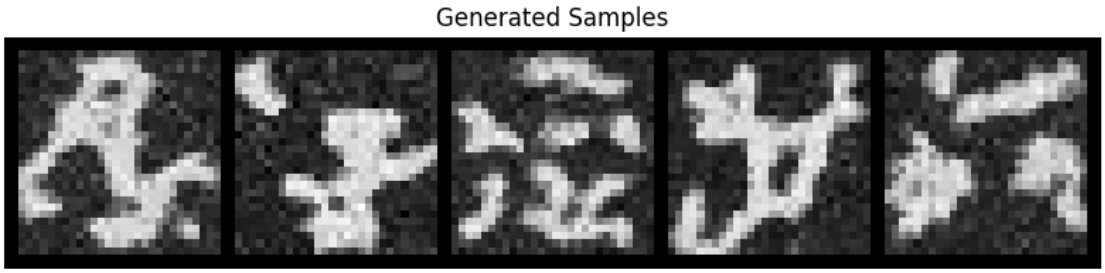



```
  Step 600/750, Loss: 0.0820
  Step 700/750, Loss: 0.0742

Training - Epoch 2 average loss: 0.0758
Running validation…
```

```
Validation - Epoch 2 average loss: 0.0734
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0734)

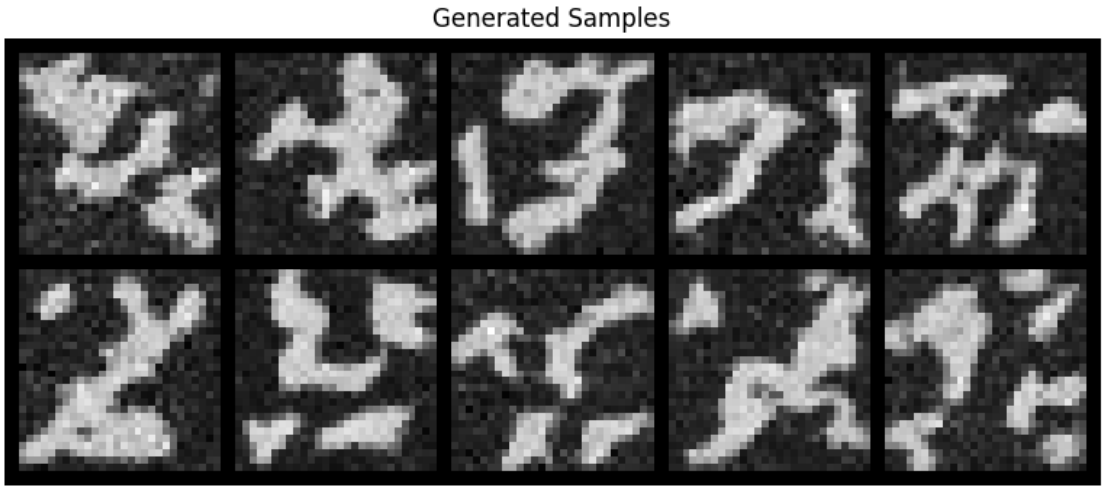
Epoch 3/30
--------------------
  Step 0/750, Loss: 0.0764
  Step 100/750, Loss: 0.0701
  Step 200/750, Loss: 0.0667
  Step 300/750, Loss: 0.0649
  Step 400/750, Loss: 0.0676
  Step 500/750, Loss: 0.0723
  Generating samples…
```




```
  Step 600/750, Loss: 0.0694
  Step 700/750, Loss: 0.0684

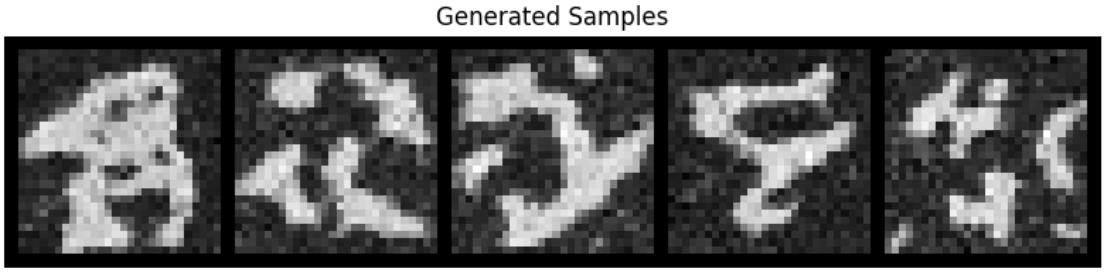
Training - Epoch 3 average loss: 0.0714
Running validation…
Validation - Epoch 3 average loss: 0.0723
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0723)

Epoch 4/30
--------------------
  Step 0/750, Loss: 0.0783
  Step 100/750, Loss: 0.0731
  Step 200/750, Loss: 0.0686
  Step 300/750, Loss: 0.0784
  Step 400/750, Loss: 0.0666
  Step 500/750, Loss: 0.0681
  Generating samples…
```

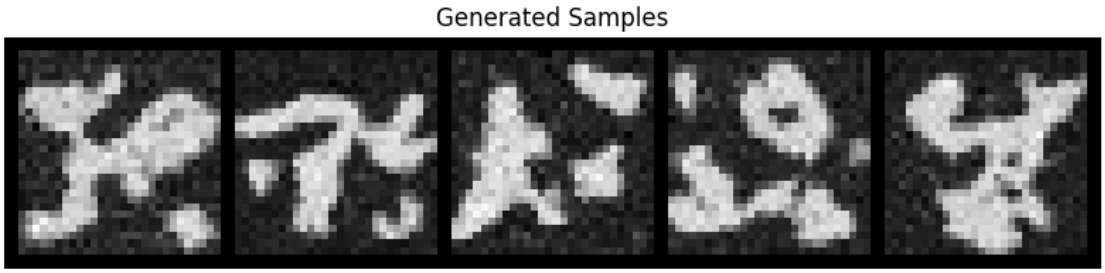



```
  Step 600/750, Loss: 0.0671
  Step 700/750, Loss: 0.0607

Training - Epoch 4 average loss: 0.0687
Running validation…
```

```
Validation - Epoch 4 average loss: 0.0673
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0673)

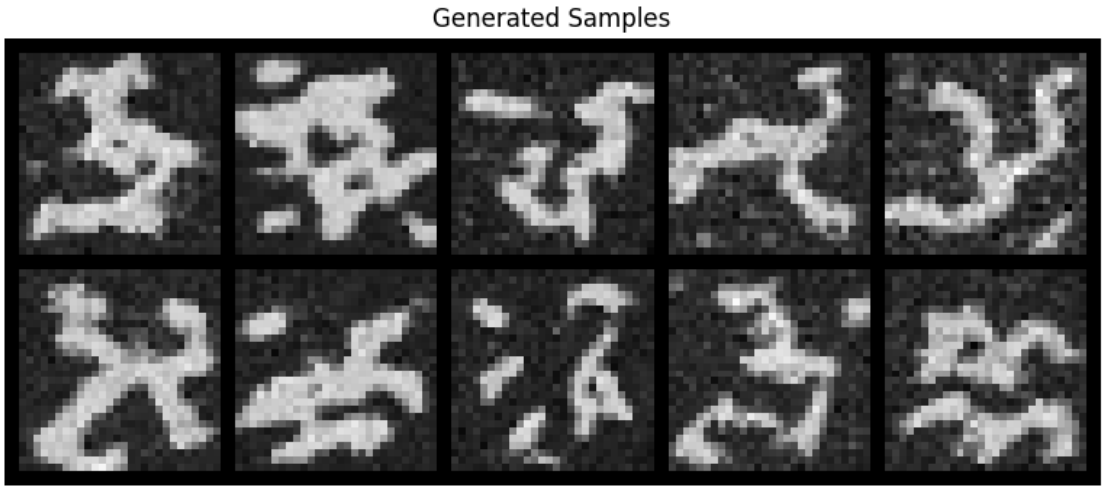
Epoch 5/30
--------------------
  Step 0/750, Loss: 0.0684
  Step 100/750, Loss: 0.0642
  Step 200/750, Loss: 0.0636
  Step 300/750, Loss: 0.0615
  Step 400/750, Loss: 0.0736
  Step 500/750, Loss: 0.0673
  Generating samples…
```




```
  Step 600/750, Loss: 0.0597
  Step 700/750, Loss: 0.0660

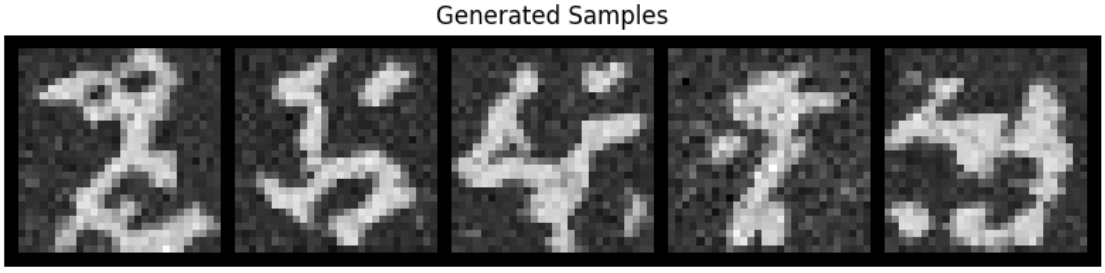
Training - Epoch 5 average loss: 0.0673
Running validation…
Validation - Epoch 5 average loss: 0.0686
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
No improvement for 1/10 epochs

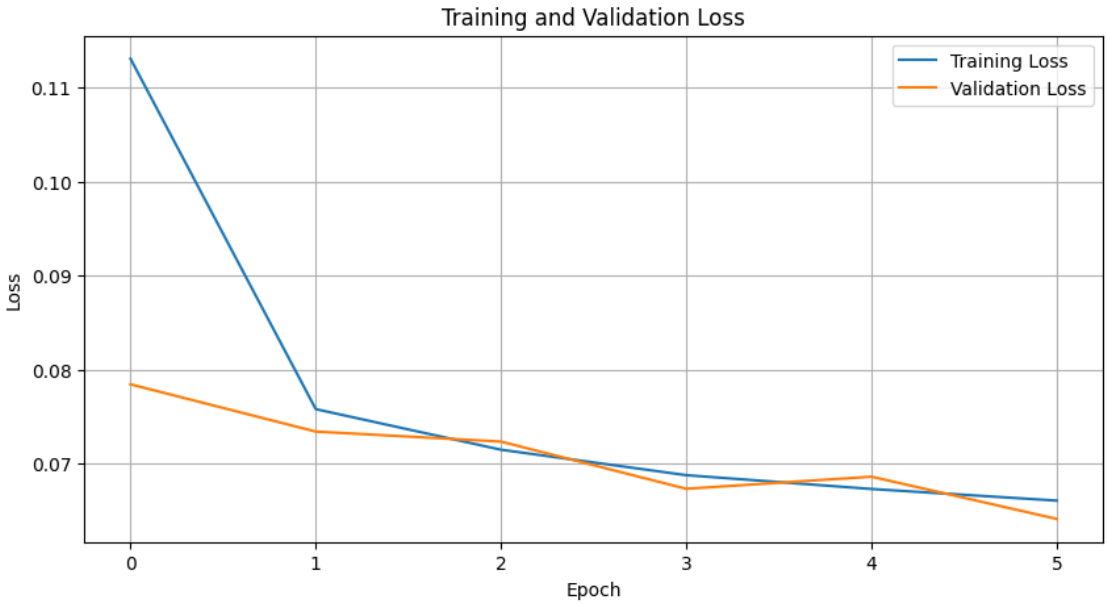
Epoch 6/30
--------------------
  Step 0/750, Loss: 0.0749
  Step 100/750, Loss: 0.0645
  Step 200/750, Loss: 0.0649
  Step 300/750, Loss: 0.0640
  Step 400/750, Loss: 0.0638
  Step 500/750, Loss: 0.0574
  Generating samples…
```



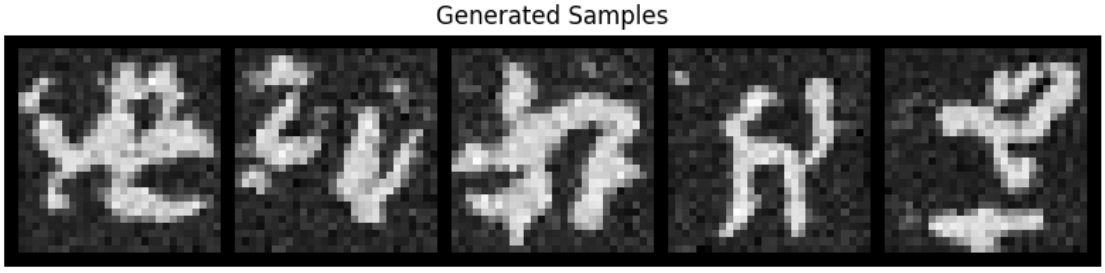

```
  Step 600/750, Loss: 0.0627
  Step 700/750, Loss: 0.0573

Training - Epoch 6 average loss: 0.0660
Running validation…
Validation - Epoch 6 average loss: 0.0641
Learning rate: 0.001000
```

```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0641)
```



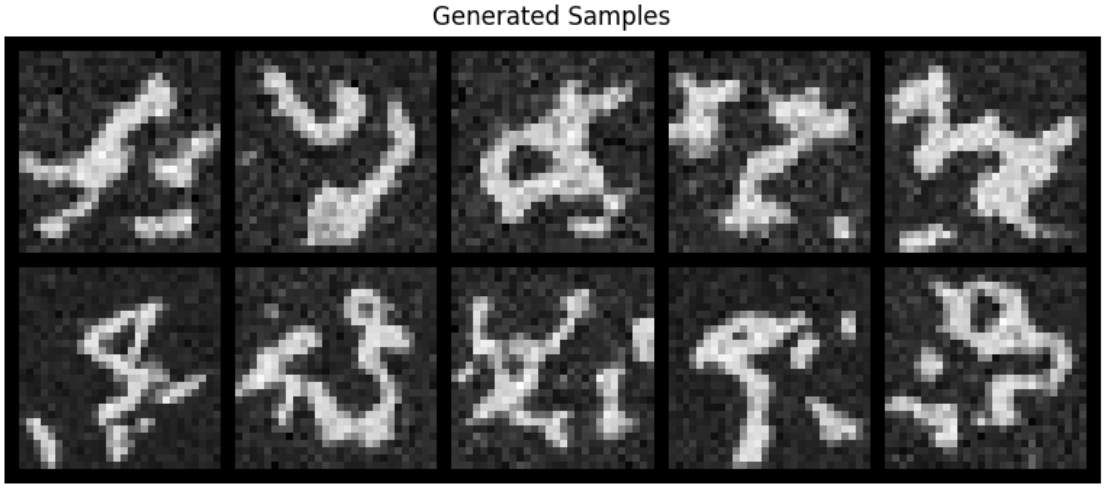

```
Epoch 7/30
--------------------
  Step 0/750, Loss: 0.0733
  Step 100/750, Loss: 0.0660
  Step 200/750, Loss: 0.0596
  Step 300/750, Loss: 0.0591
  Step 400/750, Loss: 0.0637
  Step 500/750, Loss: 0.0734
  Generating samples…
```




```
  Step 600/750, Loss: 0.0659
  Step 700/750, Loss: 0.0600
```

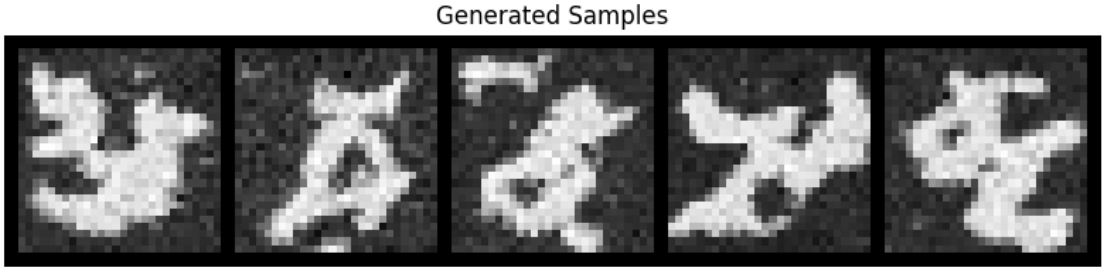
```
Training - Epoch 7 average loss: 0.0648
Running validation…
Validation - Epoch 7 average loss: 0.0639
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0639)

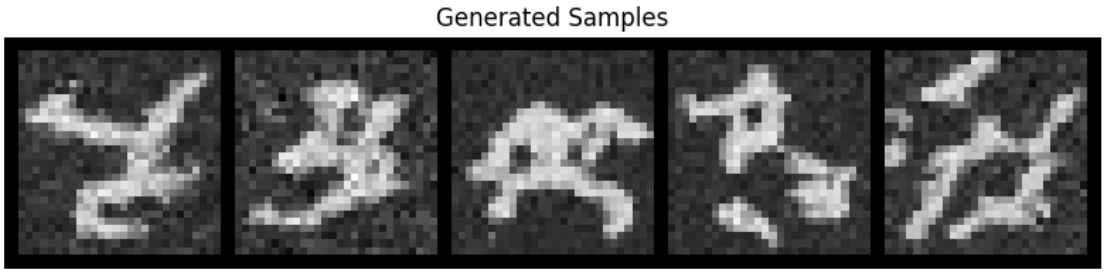
Epoch 8/30
--------------------
  Step 0/750, Loss: 0.0675
  Step 100/750, Loss: 0.0566
  Step 200/750, Loss: 0.0607
  Step 300/750, Loss: 0.0660
  Step 400/750, Loss: 0.0640
  Step 500/750, Loss: 0.0577
  Generating samples…
```



```
  Step 600/750, Loss: 0.0659
  Step 700/750, Loss: 0.0616

Training - Epoch 8 average loss: 0.0636
Running validation…
Validation - Epoch 8 average loss: 0.0649
Learning rate: 0.001000
No improvement for 1/10 epochs

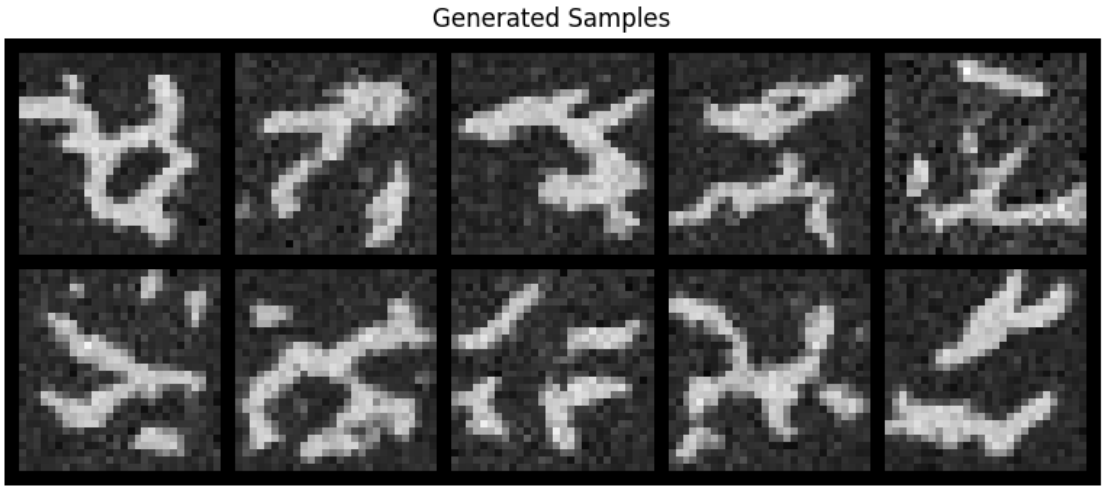
Epoch 9/30
--------------------
  Step 0/750, Loss: 0.0589
  Step 100/750, Loss: 0.0631
  Step 200/750, Loss: 0.0566
  Step 300/750, Loss: 0.0690
  Step 400/750, Loss: 0.0718
  Step 500/750, Loss: 0.0595
  Generating samples…
```




```
  Step 600/750, Loss: 0.0569
  Step 700/750, Loss: 0.0700

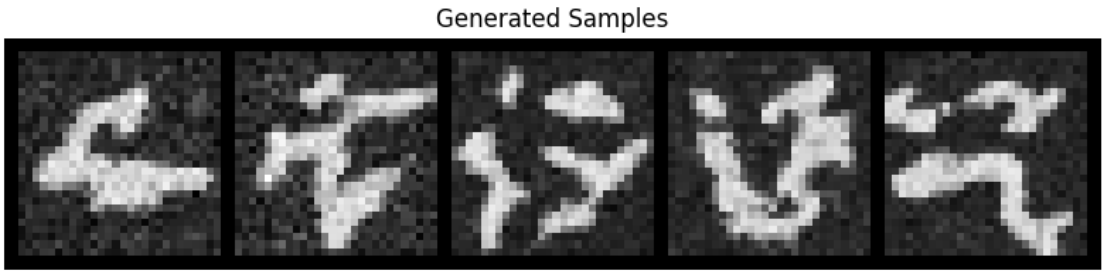
Training - Epoch 9 average loss: 0.0630
Running validation…
Validation - Epoch 9 average loss: 0.0628
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0628)

Epoch 10/30
--------------------
  Step 0/750, Loss: 0.0666
  Step 100/750, Loss: 0.0636
  Step 200/750, Loss: 0.0585
  Step 300/750, Loss: 0.0610
  Step 400/750, Loss: 0.0596
  Step 500/750, Loss: 0.0616
  Generating samples…
```

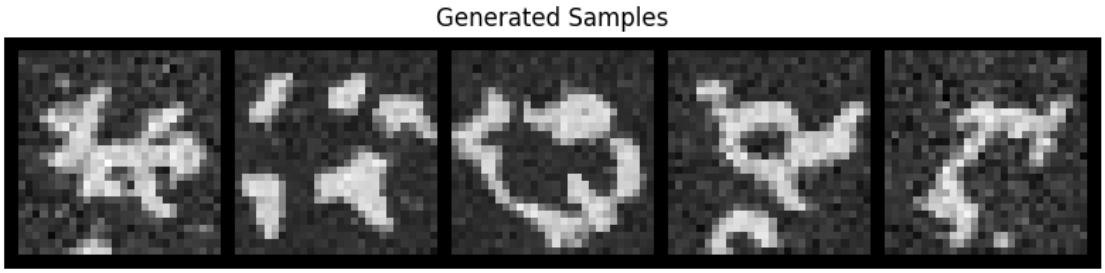



```
  Step 600/750, Loss: 0.0625
  Step 700/750, Loss: 0.0614

Training - Epoch 10 average loss: 0.0624
Running validation…
```

```
Validation - Epoch 10 average loss: 0.0614
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0614)

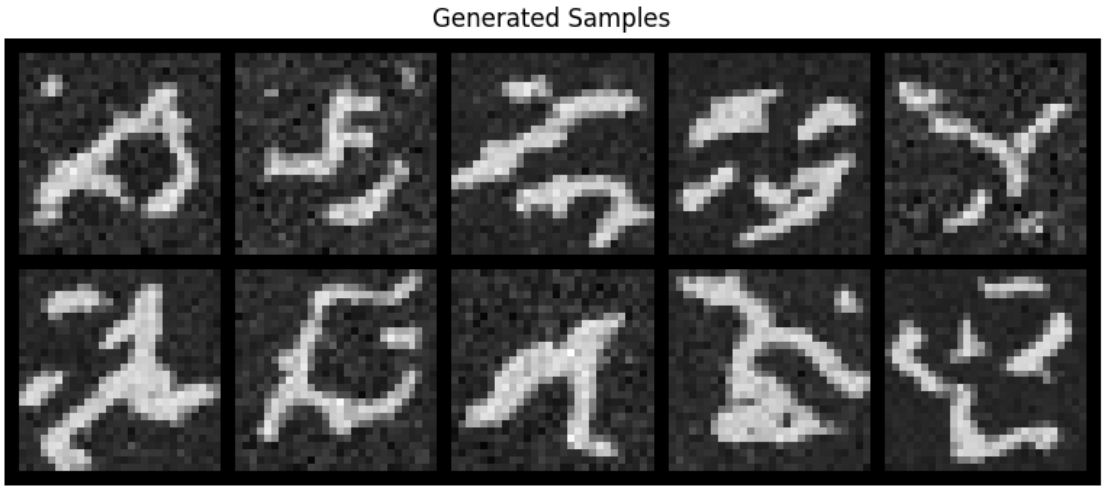
Epoch 11/30
--------------------
  Step 0/750, Loss: 0.0625
  Step 100/750, Loss: 0.0589
  Step 200/750, Loss: 0.0608
  Step 300/750, Loss: 0.0636
  Step 400/750, Loss: 0.0561
  Step 500/750, Loss: 0.0680
  Generating samples…
```




```
  Step 600/750, Loss: 0.0652
  Step 700/750, Loss: 0.0629

Training - Epoch 11 average loss: 0.0615
Running validation…
Validation - Epoch 11 average loss: 0.0610
Learning rate: 0.001000

Generating samples for visual progress check…
```

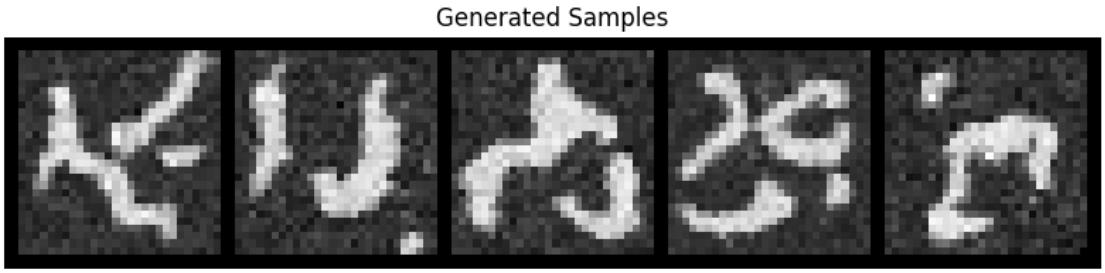



```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0610)
```




```
Epoch 12/30
--------------------
  Step 0/750, Loss: 0.0663
  Step 100/750, Loss: 0.0601
  Step 200/750, Loss: 0.0673
```

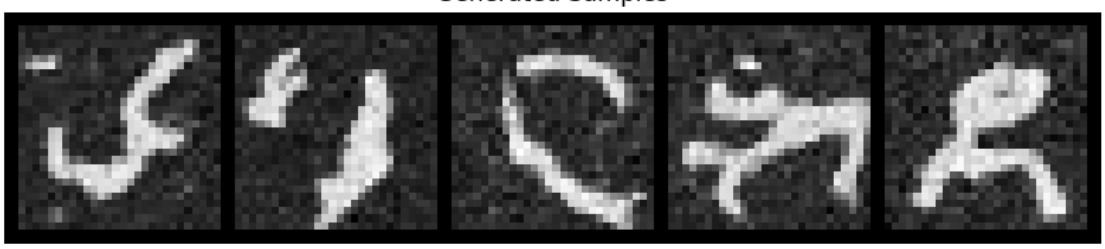
```
  Step 300/750, Loss: 0.0583
  Step 400/750, Loss: 0.0631
  Step 500/750, Loss: 0.0570
  Generating samples…
```




```
  Step 600/750, Loss: 0.0588
  Step 700/750, Loss: 0.0677

Training - Epoch 12 average loss: 0.0614
Running validation…
Validation - Epoch 12 average loss: 0.0599
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0599)

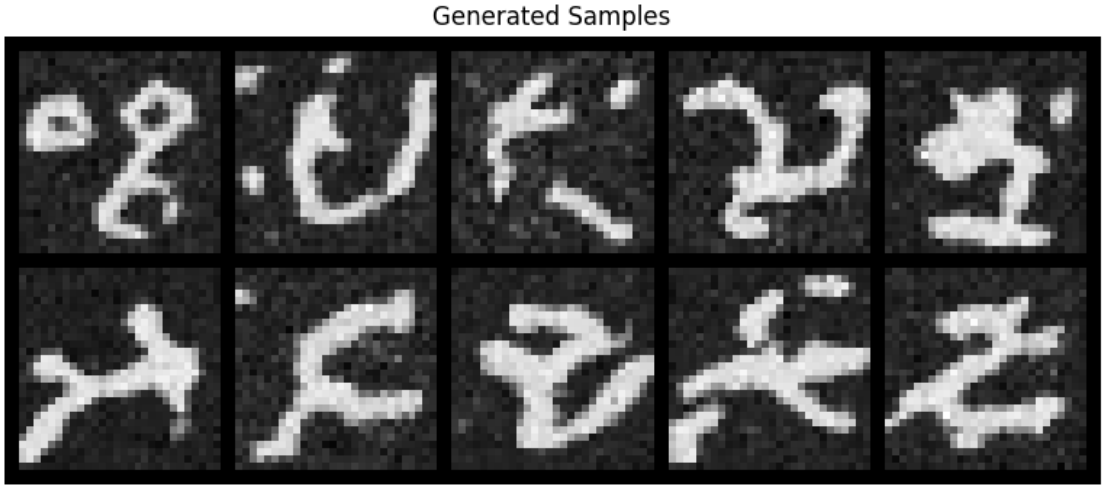
Epoch 13/30
--------------------
  Step 0/750, Loss: 0.0551
  Step 100/750, Loss: 0.0677
  Step 200/750, Loss: 0.0584
  Step 300/750, Loss: 0.0533
  Step 400/750, Loss: 0.0686
  Step 500/750, Loss: 0.0639
  Generating samples…
```



```
  Step 600/750, Loss: 0.0593
  Step 700/750, Loss: 0.0634

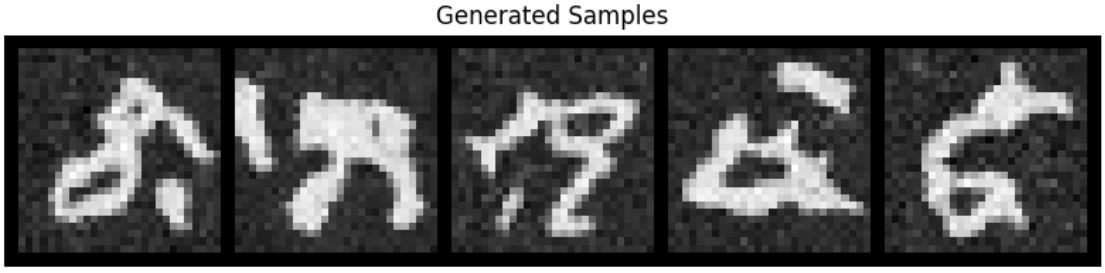
Training - Epoch 13 average loss: 0.0607
Running validation…
Validation - Epoch 13 average loss: 0.0601
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
No improvement for 1/10 epochs

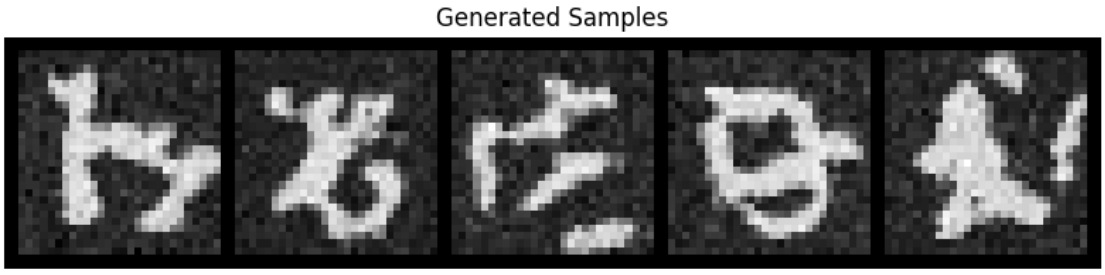
Epoch 14/30
--------------------
  Step 0/750, Loss: 0.0633
  Step 100/750, Loss: 0.0509
  Step 200/750, Loss: 0.0595
  Step 300/750, Loss: 0.0583
  Step 400/750, Loss: 0.0648
  Step 500/750, Loss: 0.0655
  Generating samples…
```



```
  Step 600/750, Loss: 0.0592
  Step 700/750, Loss: 0.0605

Training - Epoch 14 average loss: 0.0602
Running validation…
Validation - Epoch 14 average loss: 0.0600
Learning rate: 0.001000
No improvement for 2/10 epochs

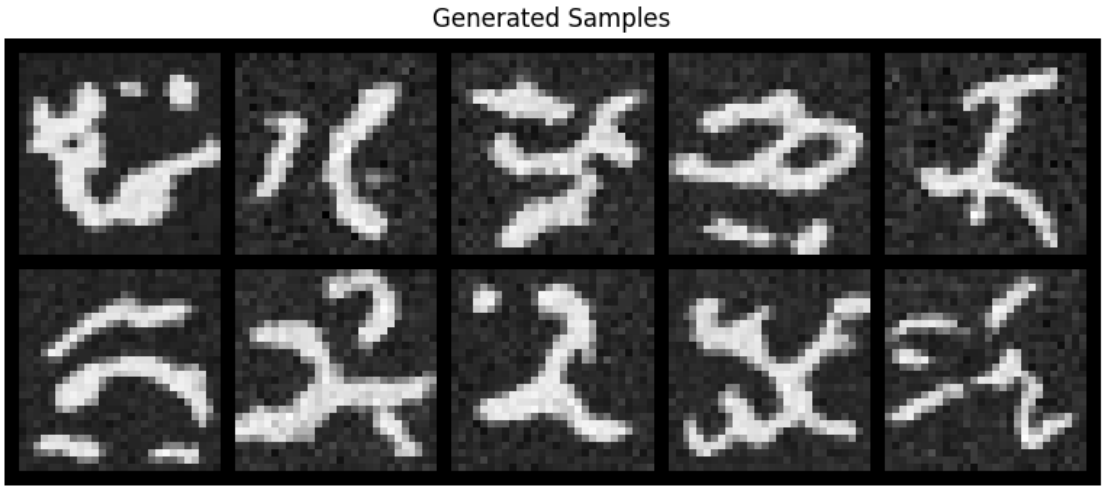
Epoch 15/30
--------------------
  Step 0/750, Loss: 0.0575
  Step 100/750, Loss: 0.0577
  Step 200/750, Loss: 0.0581
  Step 300/750, Loss: 0.0641
  Step 400/750, Loss: 0.0633
  Step 500/750, Loss: 0.0505
  Generating samples…
```




```
  Step 600/750, Loss: 0.0554
  Step 700/750, Loss: 0.0626

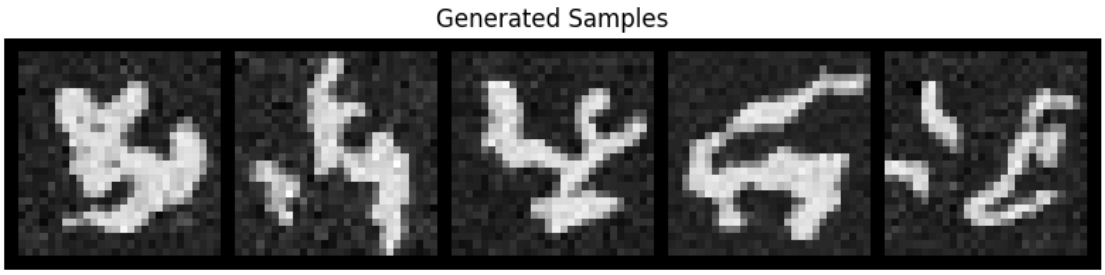
Training - Epoch 15 average loss: 0.0601
Running validation…
Validation - Epoch 15 average loss: 0.0596
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0596)

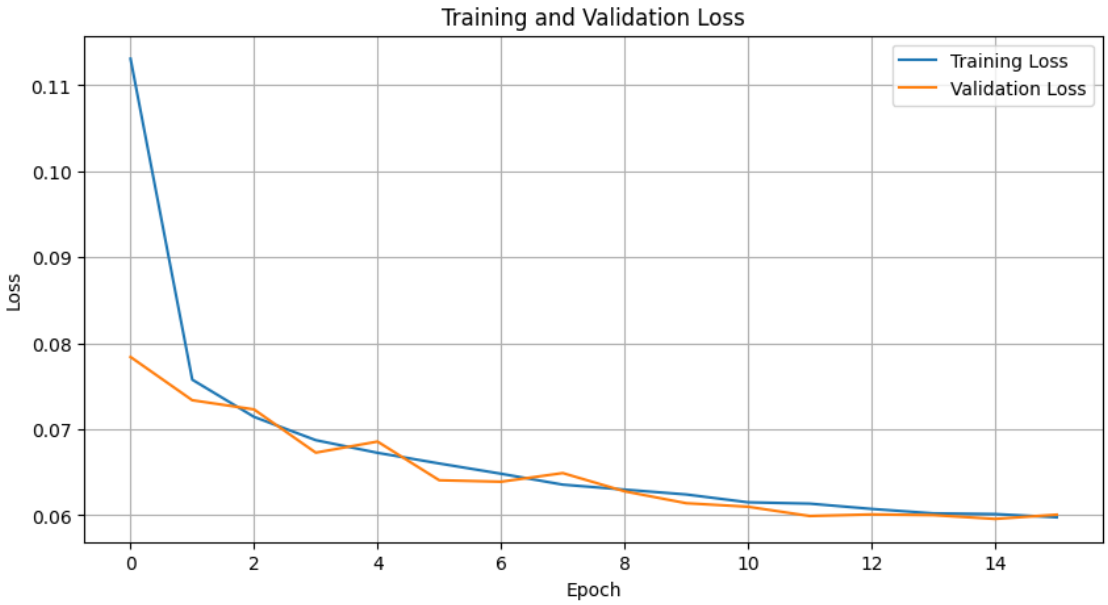
Epoch 16/30
--------------------
  Step 0/750, Loss: 0.0644
  Step 100/750, Loss: 0.0569
  Step 200/750, Loss: 0.0624
  Step 300/750, Loss: 0.0672
  Step 400/750, Loss: 0.0602
  Step 500/750, Loss: 0.0545
  Generating samples…
```



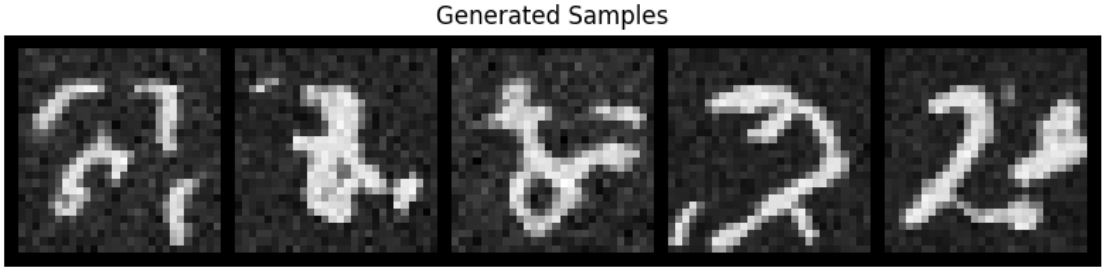

```
  Step 600/750, Loss: 0.0697
  Step 700/750, Loss: 0.0547

Training - Epoch 16 average loss: 0.0598
Running validation…
```

```
Validation - Epoch 16 average loss: 0.0601
Learning rate: 0.001000
No improvement for 1/10 epochs
```



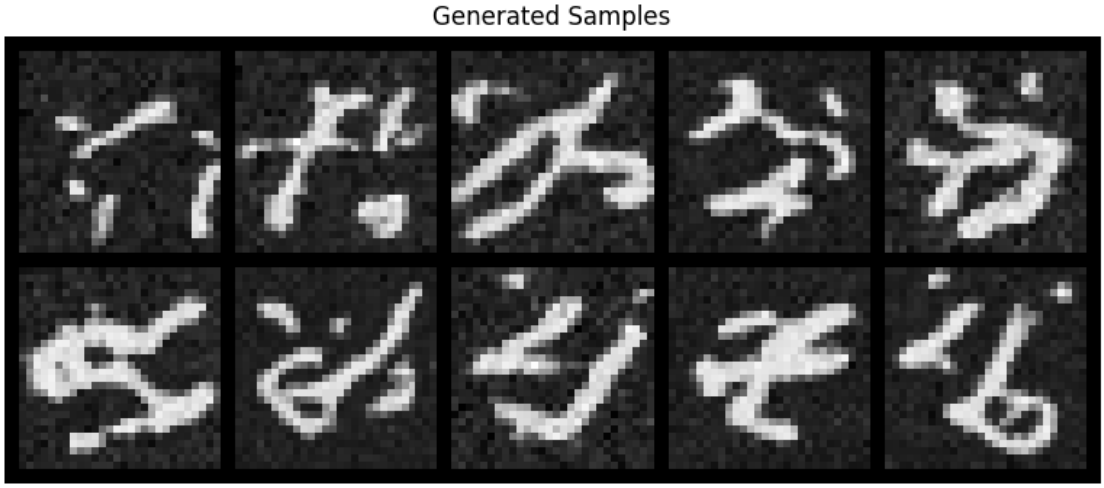

```
Epoch 17/30
--------------------
  Step 0/750, Loss: 0.0565
  Step 100/750, Loss: 0.0643
  Step 200/750, Loss: 0.0554
  Step 300/750, Loss: 0.0609
  Step 400/750, Loss: 0.0608
  Step 500/750, Loss: 0.0649
  Generating samples…
```




```
  Step 600/750, Loss: 0.0648
  Step 700/750, Loss: 0.0616
```

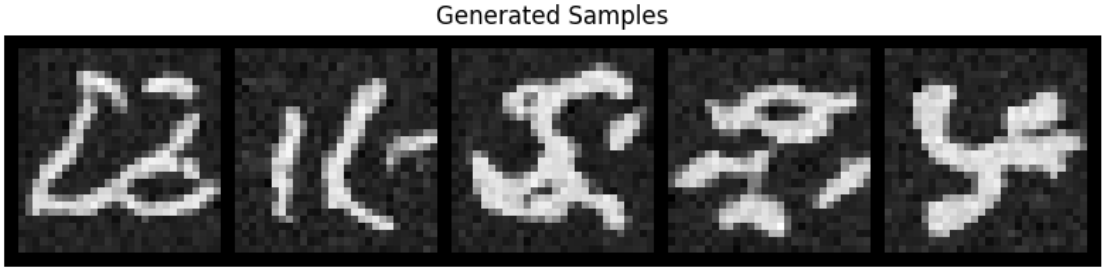
```
Training - Epoch 17 average loss: 0.0597
Running validation…
Validation - Epoch 17 average loss: 0.0596
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
No improvement for 2/10 epochs

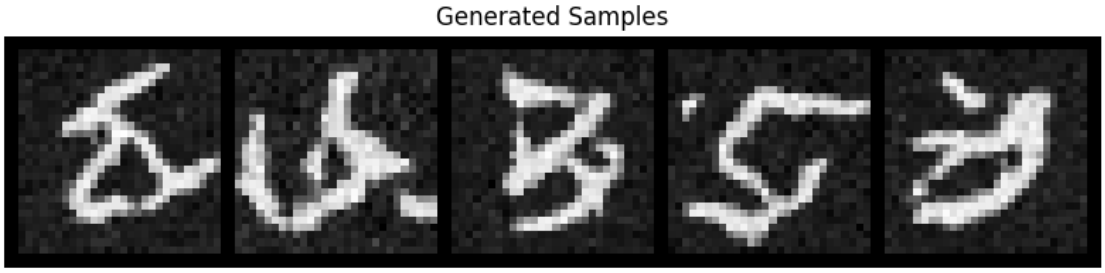
Epoch 18/30
--------------------
  Step 0/750, Loss: 0.0649
  Step 100/750, Loss: 0.0607
  Step 200/750, Loss: 0.0612
  Step 300/750, Loss: 0.0668
  Step 400/750, Loss: 0.0578
  Step 500/750, Loss: 0.0519
  Generating samples…
```



```
  Step 600/750, Loss: 0.0651
  Step 700/750, Loss: 0.0555

Training - Epoch 18 average loss: 0.0593
Running validation…
Validation - Epoch 18 average loss: 0.0582
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0582)

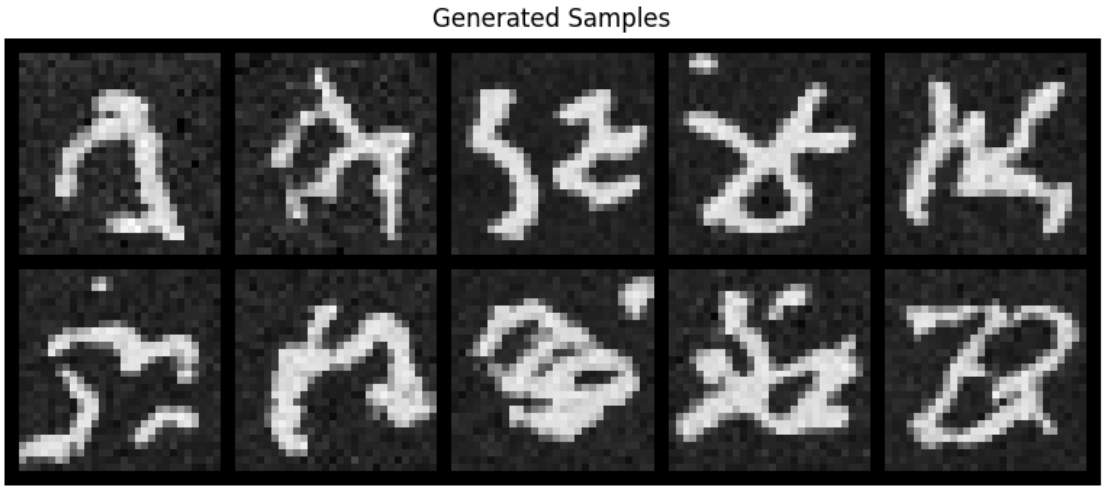
Epoch 19/30
--------------------
  Step 0/750, Loss: 0.0559
  Step 100/750, Loss: 0.0545
  Step 200/750, Loss: 0.0568
  Step 300/750, Loss: 0.0527
  Step 400/750, Loss: 0.0508
  Step 500/750, Loss: 0.0582
  Generating samples…
```




```
  Step 600/750, Loss: 0.0483
  Step 700/750, Loss: 0.0513

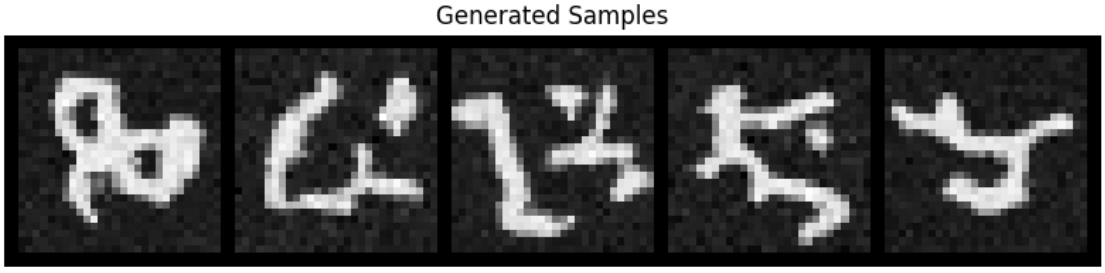
Training - Epoch 19 average loss: 0.0585
Running validation…
Validation - Epoch 19 average loss: 0.0587
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
No improvement for 1/10 epochs

Epoch 20/30
--------------------
  Step 0/750, Loss: 0.0653
  Step 100/750, Loss: 0.0532
  Step 200/750, Loss: 0.0607
  Step 300/750, Loss: 0.0606
  Step 400/750, Loss: 0.0600
  Step 500/750, Loss: 0.0538
  Generating samples…
```

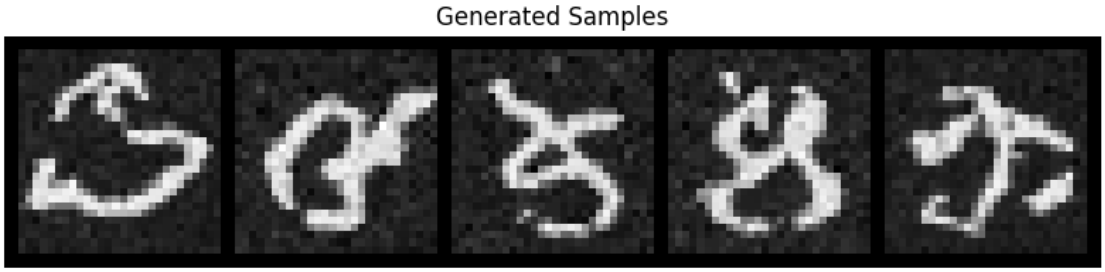



```
  Step 600/750, Loss: 0.0621
  Step 700/750, Loss: 0.0550

Training - Epoch 20 average loss: 0.0588
Running validation…
Validation - Epoch 20 average loss: 0.0587
Learning rate: 0.001000
```

```
No improvement for 2/10 epochs

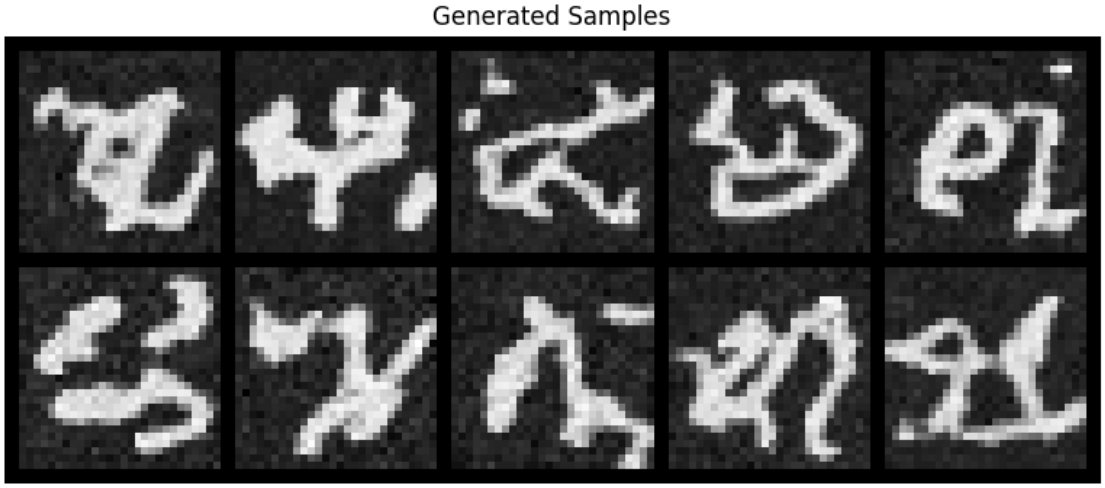
Epoch 21/30
--------------------
  Step 0/750, Loss: 0.0556
  Step 100/750, Loss: 0.0534
  Step 200/750, Loss: 0.0621
  Step 300/750, Loss: 0.0525
  Step 400/750, Loss: 0.0576
  Step 500/750, Loss: 0.0662
  Generating samples…
```



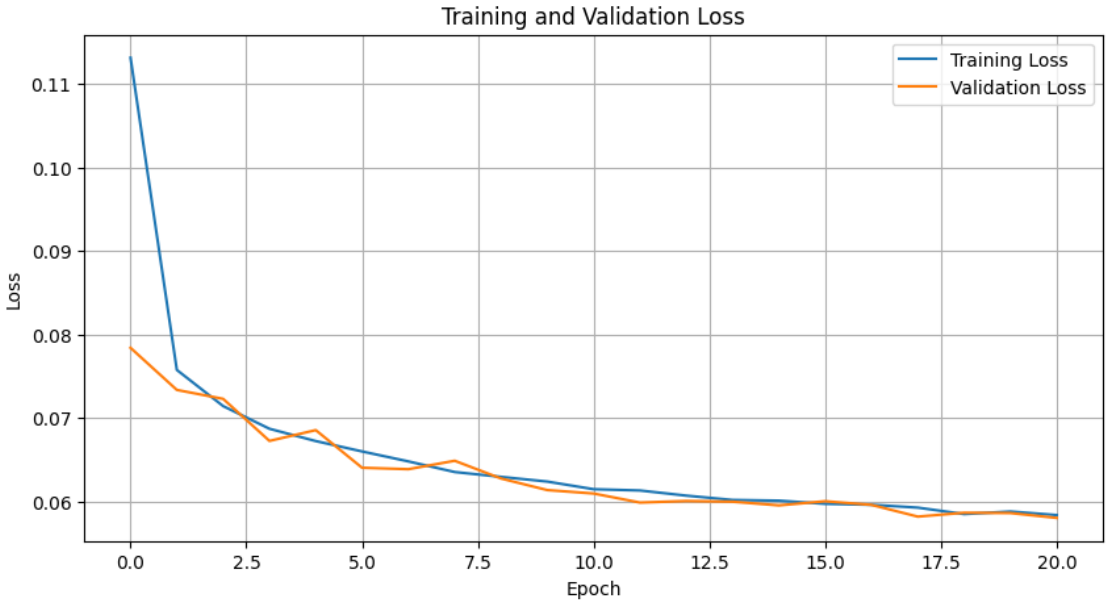

```
  Step 600/750, Loss: 0.0600
  Step 700/750, Loss: 0.0567

Training - Epoch 21 average loss: 0.0584
Running validation…
Validation - Epoch 21 average loss: 0.0581
Learning rate: 0.001000

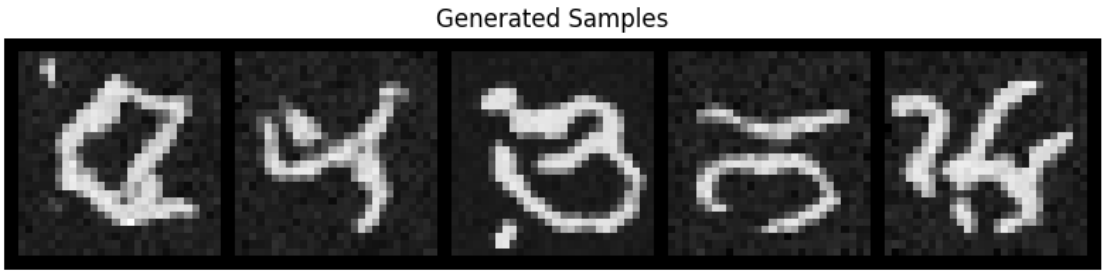
Generating samples for visual progress check…
```



```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0581)
```



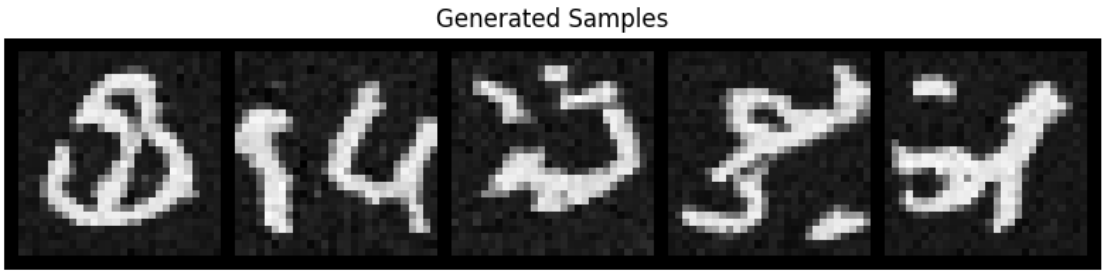

```
Epoch 22/30
--------------------
  Step 0/750, Loss: 0.0558
  Step 100/750, Loss: 0.0646
  Step 200/750, Loss: 0.0607
  Step 300/750, Loss: 0.0570
  Step 400/750, Loss: 0.0532
  Step 500/750, Loss: 0.0645
  Generating samples…
```



```
  Step 600/750, Loss: 0.0561
  Step 700/750, Loss: 0.0543

Training - Epoch 22 average loss: 0.0585
Running validation…
Validation - Epoch 22 average loss: 0.0604
Learning rate: 0.001000
No improvement for 1/10 epochs

Epoch 23/30
--------------------
  Step 0/750, Loss: 0.0673
  Step 100/750, Loss: 0.0593
  Step 200/750, Loss: 0.0595
  Step 300/750, Loss: 0.0603
  Step 400/750, Loss: 0.0553
  Step 500/750, Loss: 0.0549
  Generating samples…
```




```
  Step 600/750, Loss: 0.0602
  Step 700/750, Loss: 0.0609

Training - Epoch 23 average loss: 0.0584
Running validation…
Validation - Epoch 23 average loss: 0.0580
Learning rate: 0.001000

Generating samples for visual progress check…
```

Generated Samples



```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0580)

Epoch 24/30
--------------------
  Step 0/750, Loss: 0.0570
  Step 100/750, Loss: 0.0532
  Step 200/750, Loss: 0.0527
  Step 300/750, Loss: 0.0565
  Step 400/750, Loss: 0.0596
  Step 500/750, Loss: 0.0515
  Generating samples…
```

Generated Samples

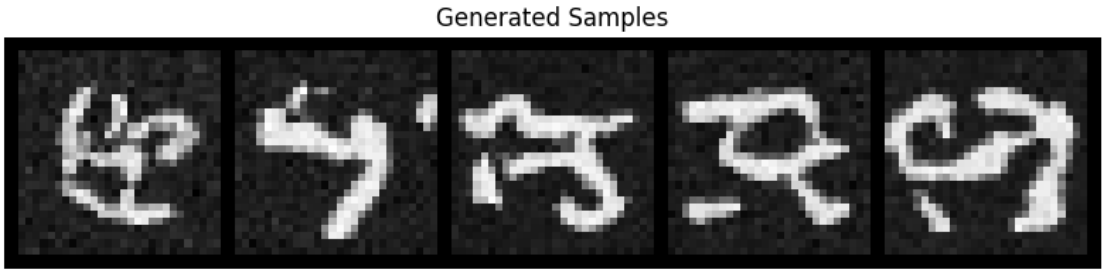


```
  Step 600/750, Loss: 0.0606
  Step 700/750, Loss: 0.0566

Training - Epoch 24 average loss: 0.0580
Running validation…
```

```
Validation - Epoch 24 average loss: 0.0578
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0578)

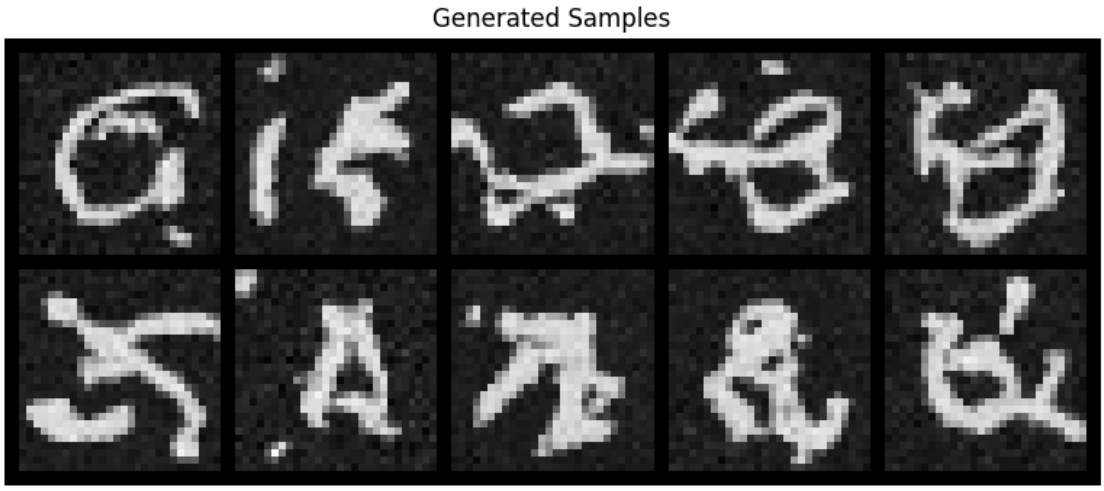
Epoch 25/30
--------------------
  Step 0/750, Loss: 0.0582
  Step 100/750, Loss: 0.0492
  Step 200/750, Loss: 0.0480
  Step 300/750, Loss: 0.0617
  Step 400/750, Loss: 0.0557
  Step 500/750, Loss: 0.0612
  Generating samples…
```




```
  Step 600/750, Loss: 0.0627
  Step 700/750, Loss: 0.0603

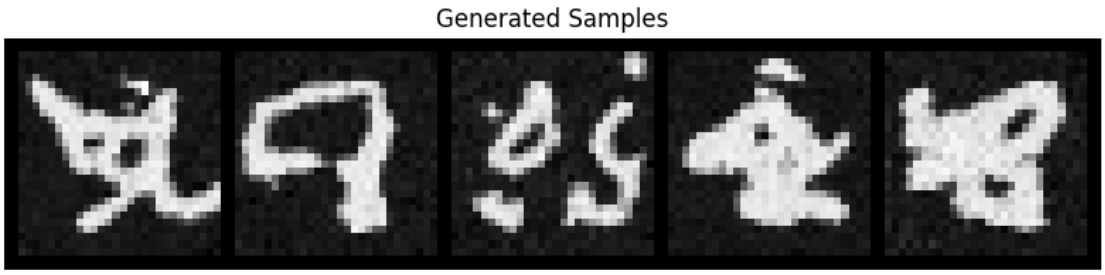
Training - Epoch 25 average loss: 0.0580
Running validation…
Validation - Epoch 25 average loss: 0.0572
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0572)

Epoch 26/30
--------------------
  Step 0/750, Loss: 0.0677
  Step 100/750, Loss: 0.0541
  Step 200/750, Loss: 0.0656
  Step 300/750, Loss: 0.0584
  Step 400/750, Loss: 0.0562
  Step 500/750, Loss: 0.0664
  Generating samples…
```



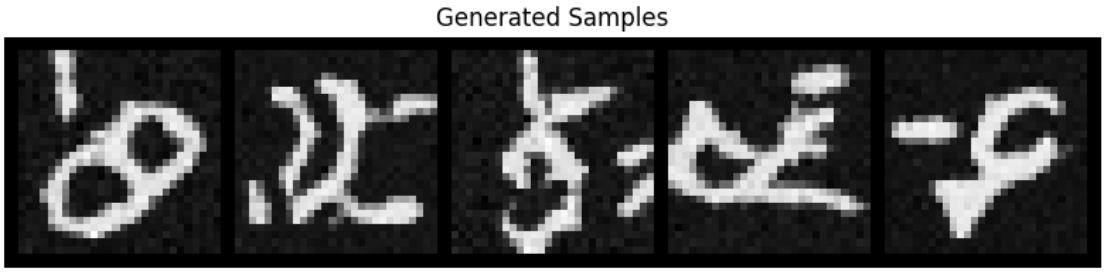

```
  Step 600/750, Loss: 0.0688
  Step 700/750, Loss: 0.0501

Training - Epoch 26 average loss: 0.0582
Running validation…
```

```
Validation - Epoch 26 average loss: 0.0570
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0570)
```



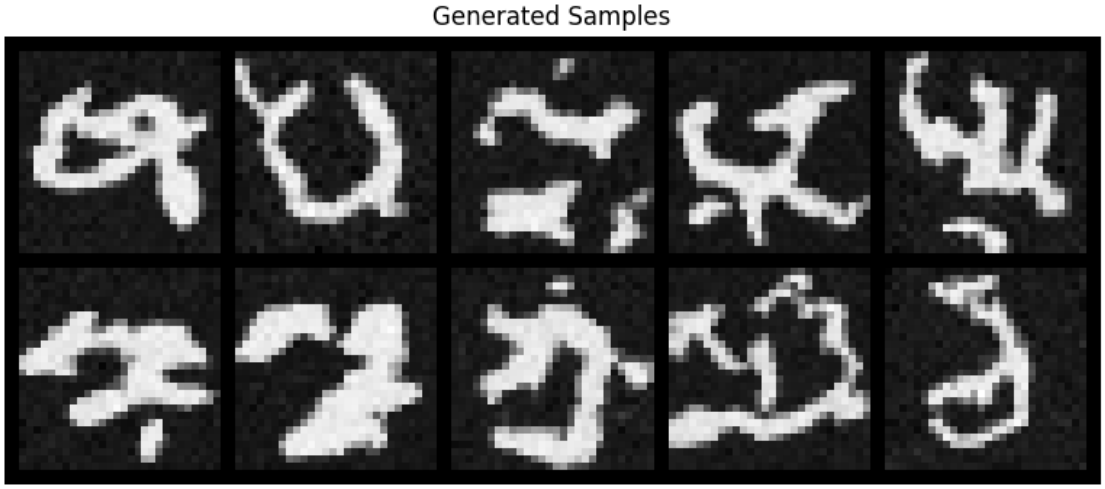

```
Epoch 27/30
--------------------
  Step 0/750, Loss: 0.0590
  Step 100/750, Loss: 0.0620
  Step 200/750, Loss: 0.0540
  Step 300/750, Loss: 0.0538
  Step 400/750, Loss: 0.0625
  Step 500/750, Loss: 0.0501
  Generating samples…
```



```
  Step 600/750, Loss: 0.0487
  Step 700/750, Loss: 0.0577

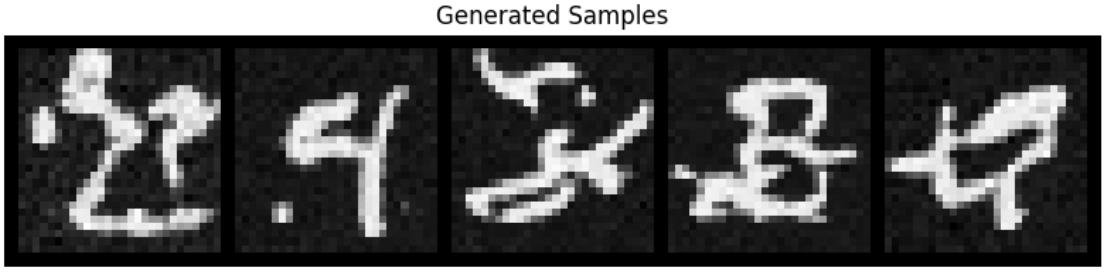
Training - Epoch 27 average loss: 0.0576
Running validation…
Validation - Epoch 27 average loss: 0.0571
Learning rate: 0.001000

Generating samples for visual progress check…
```




```
No improvement for 1/10 epochs

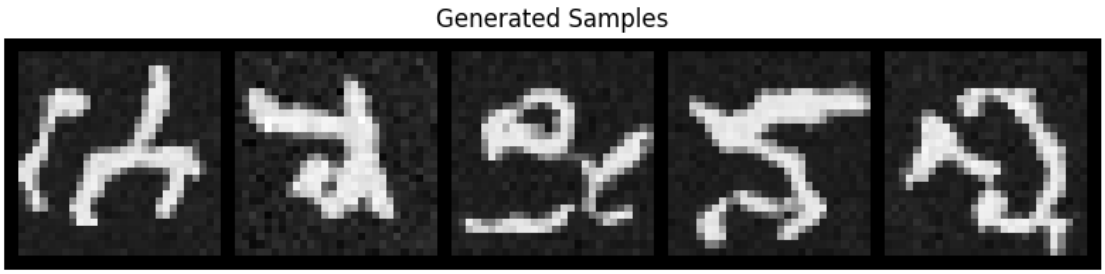
Epoch 28/30
--------------------
  Step 0/750, Loss: 0.0634
  Step 100/750, Loss: 0.0561
  Step 200/750, Loss: 0.0576
  Step 300/750, Loss: 0.0591
  Step 400/750, Loss: 0.0634
  Step 500/750, Loss: 0.0655
  Generating samples…
```



```
  Step 600/750, Loss: 0.0578
  Step 700/750, Loss: 0.0619

Training - Epoch 28 average loss: 0.0576
Running validation…
Validation - Epoch 28 average loss: 0.0569
Learning rate: 0.001000
Created backup at best_diffusion_model.pt.backup
Model successfully saved to best_diffusion_model.pt
  New best model saved! (Val Loss: 0.0569)

Epoch 29/30
--------------------
  Step 0/750, Loss: 0.0584
  Step 100/750, Loss: 0.0506
  Step 200/750, Loss: 0.0546
  Step 300/750, Loss: 0.0634
  Step 400/750, Loss: 0.0522
  Step 500/750, Loss: 0.0560
  Generating samples…
```

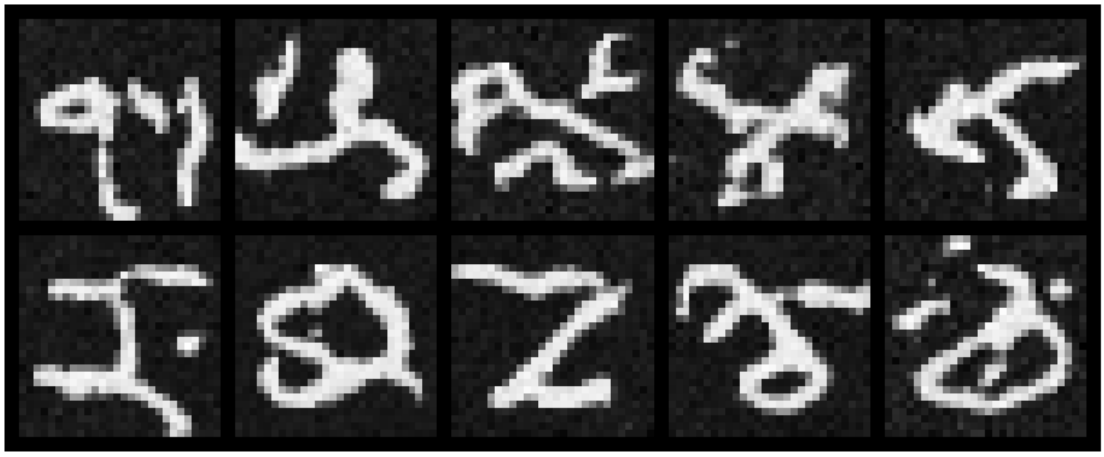



```
  Step 600/750, Loss: 0.0598
  Step 700/750, Loss: 0.0564

Training - Epoch 29 average loss: 0.0576
Running validation…
Validation - Epoch 29 average loss: 0.0573
Learning rate: 0.001000

Generating samples for visual progress check…
```

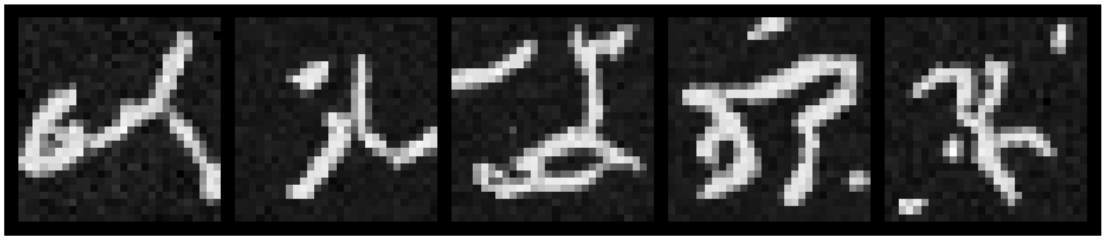
Generated Samples



```
No improvement for 1/10 epochs

Epoch 30/30
--------------------
  Step 0/750, Loss: 0.0525
  Step 100/750, Loss: 0.0607
  Step 200/750, Loss: 0.0622
  Step 300/750, Loss: 0.0614
  Step 400/750, Loss: 0.0553
  Step 500/750, Loss: 0.0622
  Generating samples…
```

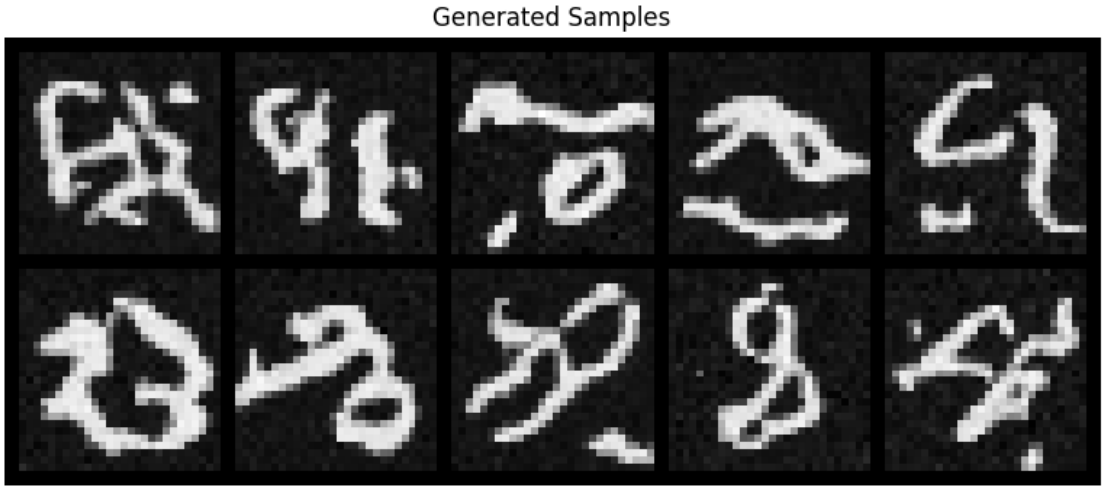
Generated Samples



```
  Step 600/750, Loss: 0.0511
  Step 700/750, Loss: 0.0529

Training - Epoch 30 average loss: 0.0576
Running validation…
Validation - Epoch 30 average loss: 0.0578
Learning rate: 0.001000
```

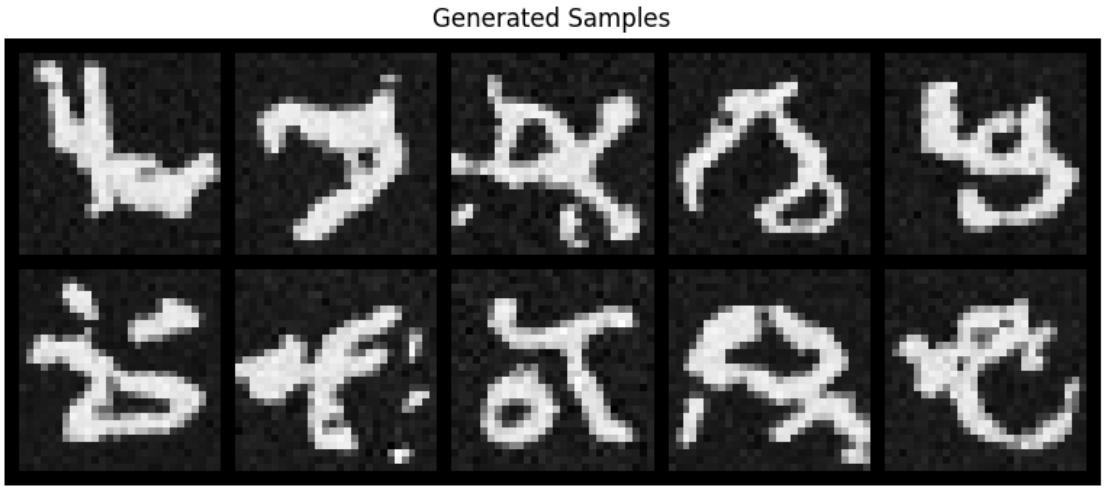
```
Generating samples for visual progress check…
```




```
No improvement for 2/10 epochs
```




```
==================================================
TRAINING COMPLETE
==================================================
Best validation loss: 0.0569
Generating final samples…
```





[ ]:
```python
# Plot training progress
plt.figure(figsize=(12, 5))

# Plot training and validation losses for comparison
plt.plot(train_losses, label='Training Loss')
if len(val_losses) > 0:  # Only plot validation if it exists
    plt.plot(val_losses, label='Validation Loss')

# Improve the plot with better labels and styling
plt.title('Diffusion Model Training Progress')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
```

```
plt.legend()
plt.grid(True)

# Add annotations for key points
if len(train_losses) > 1:
    min_train_idx = train_losses.index(min(train_losses))
    plt.annotate(f'Min: {min(train_losses):.4f}',
                 xy=(min_train_idx, min(train_losses)),
                 xytext=(min_train_idx, min(train_losses)*1.2),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)

# Add validation min point if available
if len(val_losses) > 1:
    min_val_idx = val_losses.index(min(val_losses))
    plt.annotate(f'Min: {min(val_losses):.4f}',
                 xy=(min_val_idx, min(val_losses)),
                 xytext=(min_val_idx, min(val_losses)*0.8),
                 arrowprops=dict(facecolor='black', shrink=0.05),
                 fontsize=9)

# Set y-axis to start from 0 or slightly lower than min value
plt.ylim(bottom=max(0, min(min(train_losses) if train_losses else float('inf'),
                           min(val_losses) if val_losses else float('inf'))*0.9))

plt.tight_layout()
plt.show()

# Add statistics summary for students to analyze
print("\nTraining Statistics:")
print("-" * 30)
if train_losses:
    print(f"Starting training loss:    {train_losses[0]:.4f}")
    print(f"Final training loss:       {train_losses[-1]:.4f}")
    print(f"Best training loss:        {min(train_losses):.4f}")
    print(f"Training loss improvement: {((train_losses[0] - min(train_losses)) /
 ↪ train_losses[0] * 100):.1f}%")

if val_losses:
    print("\nValidation Statistics:")
    print("-" * 30)
    print(f"Starting validation loss: {val_losses[0]:.4f}")
    print(f"Final validation loss:    {val_losses[-1]:.4f}")
    print(f"Best validation loss:     {min(val_losses):.4f}")

# STUDENT EXERCISE:
# 1. Try modifying this plot to show a smoothed version of the losses
```

```
# 2. Create a second plot showing the ratio of validation to training loss
#    (which can indicate overfitting when the ratio increases)

# --- BEGIN STUDENT WORK: ESTHER ---

# Compare only epochs that have both a training and validation result.
# Each position in these lists represents the same recorded epoch.
paired_epochs = min(len(train_losses), len(val_losses))
window = 3  # Average three consecutive epochs for smoothing

if paired_epochs < window:
    print("\nAt least 3 matching epochs are needed for these exercise plots.")

else:
    # Convert the recorded losses to numerical arrays.
    # These are copies for analysis; the original histories stay intact.
    training_values = np.array(train_losses[:paired_epochs], dtype=float)
    validation_values = np.array(val_losses[:paired_epochs], dtype=float)

    # Label epochs starting at 1, matching the training progress messages.
    epochs = np.arange(1, paired_epochs + 1)

    # Give each of the three losses equal weight: 1/3.
    # Convolution with these weights calculates the moving average.
    average_weights = np.ones(window) / window

    # "valid" uses only complete three-epoch windows.
    # The first result averages epochs 1, 2, and 3.
    smoothed_training = np.convolve(
        training_values, average_weights, mode="valid"
    )
    smoothed_validation = np.convolve(
        validation_values, average_weights, mode="valid"
    )

    # Place each average at the final epoch included in its window.
    # With a window of 3, the smoothed curves begin at epoch 3.
    smoothed_epochs = epochs[window - 1:]

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Exercise 1: Show original values faintly and smoothed values clearly.
    # Keeping both visible prevents smoothing from hiding fluctuations.
    axes[0].plot(
        epochs, training_values,
        color="tab:blue", alpha=0.3, label="Training: original"
    )
```

```
    axes[0].plot(
        epochs, validation_values,
        color="tab:orange", alpha=0.3, label="Validation: original"
    )
    axes[0].plot(
        smoothed_epochs, smoothed_training,
        color="tab:blue", label="Training: 3-epoch average"
    )
    axes[0].plot(
        smoothed_epochs, smoothed_validation,
        color="tab:orange", label="Validation: 3-epoch average"
    )
    axes[0].set_title("Original and Smoothed Losses")
    axes[0].set_xlabel("Epoch")
    axes[0].set_ylabel("Mean Squared Error")
    axes[0].legend()
    axes[0].grid(True)

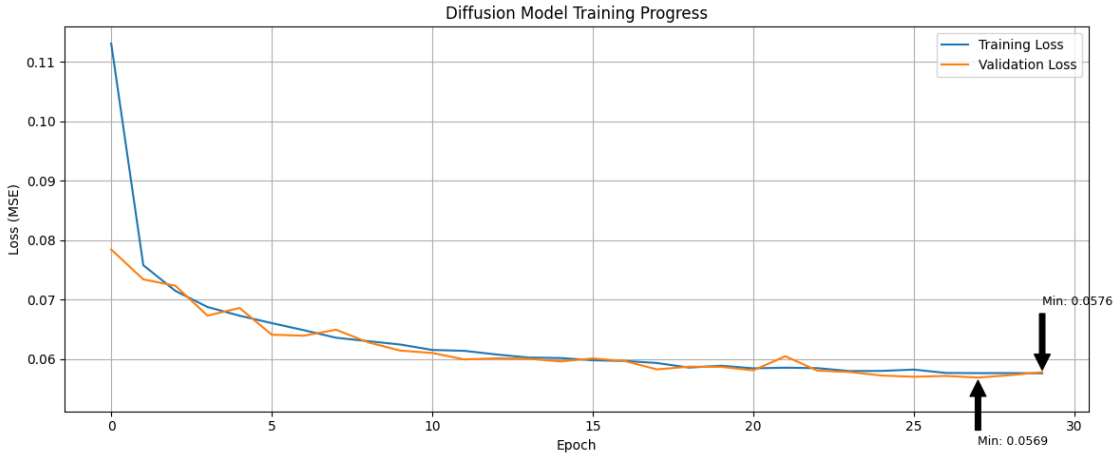
    # Exercise 2: Divide validation loss by training loss.
    # If training loss is zero, the ratio is undefined.
    # Use NaN (Not a Number) to leave a gap instead of dividing by zero.
    loss_ratio = np.divide(
        validation_values,
        training_values,
        out=np.full(paired_epochs, np.nan),
        where=training_values > 0
    )

    axes[1].plot(
        epochs, loss_ratio,
        marker="o", markersize=3, label="Validation / Training"
    )

    # A ratio of 1 means the two recorded losses are equal.
    axes[1].axhline(
        1.0, color="gray", linestyle="--", label="Equal losses"
    )
    axes[1].set_title("Validation-to-Training Loss Ratio")
    axes[1].set_xlabel("Epoch")
    axes[1].set_ylabel("Loss Ratio")
    axes[1].legend()
    axes[1].grid(True)

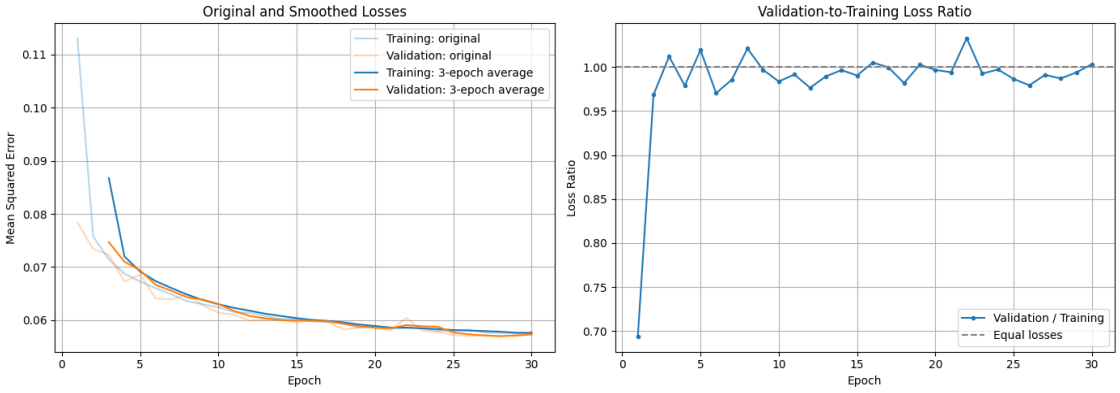
    plt.tight_layout()
    plt.show()

# --- END STUDENT WORK: ESTHER ---
```




```
Training Statistics:
--------------------------------
Starting training loss:    0.1131
Final training loss:       0.0576
Best training loss:        0.0576
Training loss improvement: 49.1%

Validation Statistics:
--------------------------------
Starting validation loss: 0.0784
Final validation loss:    0.0578
Best validation loss:     0.0569
```



1.7 Step 6: Generating New Images

Now that our model is trained, let's generate some new images! We can: 1. Generate specific numbers 2. Generate multiple versions of each number 3. See how the generation process works

step by step

[ ]:
```
def generate_number(model, number, n_samples=4):
    """
    Generate multiple versions of a requested digit.

    Args:
        model (nn.Module): The trained diffusion model
        number (int): The requested digit (0-9 for MNIST)
        n_samples (int): Number of variations to generate

    Returns:
        torch.Tensor: Generated images with shape
                      [n_samples, IMG_CH, IMG_SIZE, IMG_SIZE]
    """
    model.eval()  # Set the model to evaluation mode

    with torch.no_grad():  # Generation does not need gradient calculations

        # Start each image with its own random noise pattern.
        samples = torch.randn(
            n_samples, IMG_CH, IMG_SIZE, IMG_SIZE
        ).to(device)

        # Give every image in this batch the same requested digit.
        c = torch.full((n_samples,), number).to(device)

        # Convert digit labels into the one-hot format our model expects.
        c_one_hot = F.one_hot(c, N_CLASSES).float().to(device)

        # Shape: [n_samples, 1].
        # Ones mean that class conditioning stays active.
        c_mask = torch.ones_like(c.unsqueeze(-1)).to(device)

        print(f"Generating {n_samples} versions of number {number}...")

        # JAMES DELGADO - STUDENT WORK:
        # Report progress about five times during generation.
        # max(1, ...) keeps the interval valid for short schedules.
        progress_interval = max(1, n_steps // 5)

        # Move backward through the same schedule used during training.
        for t in range(n_steps - 1, -1, -1):

            # Every image in this batch is currently at timestep t.
            t_batch = torch.full((n_samples,), t).to(device)
```

```
            # Use the model's noise prediction to take one reverse step.
            samples = remove_noise(
                samples, t_batch, model, c_one_hot, c_mask
            )

            # JAMES DELGADO - STUDENT WORK:
            # Count completed operations from 1 through n_steps.
            completed_steps = n_steps - t

            if completed_steps % progress_interval == 0 or t == 0:
                print(
                    f"  Denoising step "
                    f"{completed_steps}/{n_steps} completed"
                )

        # Return the generated tensors before any display adjustments.
        return samples


# Generate 4 versions of each number
plt.figure(figsize=(20, 10))

for i in range(10):

    # Generate samples for the current requested digit.
    samples = generate_number(model, i, n_samples=4)

    # Display each sample.
    for j in range(4):

        # JAMES DELGADO - LEARNING NOTE:
        # This layout has 5 rows and 8 columns.
        # Each row shows two digit groups, with 4 samples per group:
        # row 1: digits 0 and 5
        # row 2: digits 1 and 6
        # row 3: digits 2 and 7
        # row 4: digits 3 and 8
        # row 5: digits 4 and 9
        plt.subplot(5, 8, (i % 5) * 8 + (i // 5) * 4 + j + 1)

        if IMG_CH == 1:  # Grayscale images

            # JAMES DELGADO - CORRECTION:
            # Use the same brightness scale for every generated image.
            # These limits affect display only, not the generated tensor.
            plt.imshow(
                samples[j, 0].cpu(),
```

```
                cmap="gray",
                vmin=-1,
                vmax=1
            )

        else:  # Color images

            # Change [C, H, W] into [H, W, C] for display.
            img = samples[j].permute(1, 2, 0).cpu()

            # JAMES DELGADO - CORRECTION:
            # Training images were normalized to [-1, 1].
            # Convert a display copy to [0, 1] and clip display extremes.
            img = ((img + 1) / 2).clamp(0, 1)
            plt.imshow(img)

        # This label identifies the request, not a verified prediction.
        plt.title(f"Requested: {i}")
        plt.axis("off")

plt.tight_layout()
plt.show()


# STUDENT ACTIVITY: Try generating the same digit with different noise seeds
# This shows the variety of styles the model can produce.
print("\nSTUDENT ACTIVITY: Generating numbers with different noise seeds")


# Helper function to generate with a seed
def generate_with_seed(number, seed_value=42, n_samples=10):

    # Reset PyTorch's random sequence before generating the images.
    torch.manual_seed(seed_value)

    return generate_number(model, number, n_samples)


# Pick an image class and show multiple variations.
# For our MNIST dataset, select a digit from 0 through 9.
# Hint 2: Use variations = generate_with_seed(...)
# Hint 3: Use plt.figure and plt.imshow to display the variations.

# Enter your code here:

# --- BEGIN STUDENT WORK: JAMES DELGADO ---
```

```
# Keep the requested digit and trained model fixed.
# Change the seed to compare results from different random sequences.
chosen_digit = 5
seed_values = [42, 123]
samples_per_seed = 4

# Create one row per seed and one column per generated variation.
fig, axes = plt.subplots(
    len(seed_values),
    samples_per_seed,
    figsize=(10, 5)
)

for row, seed_value in enumerate(seed_values):

    # The seed affects the initial noise and the random noise
    # used during reverse diffusion. Model weights stay unchanged.
    variations = generate_with_seed(
        number=chosen_digit,
        seed_value=seed_value,
        n_samples=samples_per_seed
    )

    for column in range(samples_per_seed):

        if IMG_CH == 1:

            # Select one generated image and its grayscale channel.
            image_to_show = variations[column, 0].cpu()

            axes[row, column].imshow(
                image_to_show,
                cmap="gray",
                vmin=-1,
                vmax=1
            )

        else:

            # Prepare a color image for display.
            image_to_show = variations[column].permute(1, 2, 0).cpu()
            image_to_show = ((image_to_show + 1) / 2).clamp(0, 1)

            axes[row, column].imshow(image_to_show)

        axes[row, column].set_title(
            f"Seed {seed_value}\nSample {column + 1}"
```

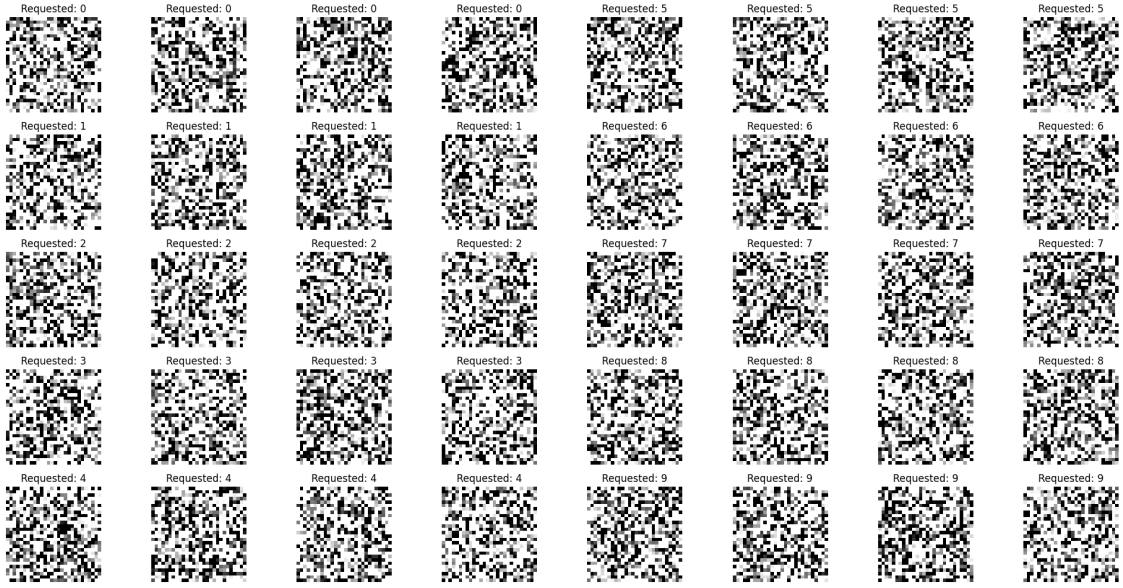
```
        )
        axes[row, column].axis("off")

fig.suptitle(f"Requested digit {chosen_digit}: comparing noise seeds")
plt.tight_layout()
plt.show()

# --- END STUDENT WORK: JAMES DELGADO ---
```

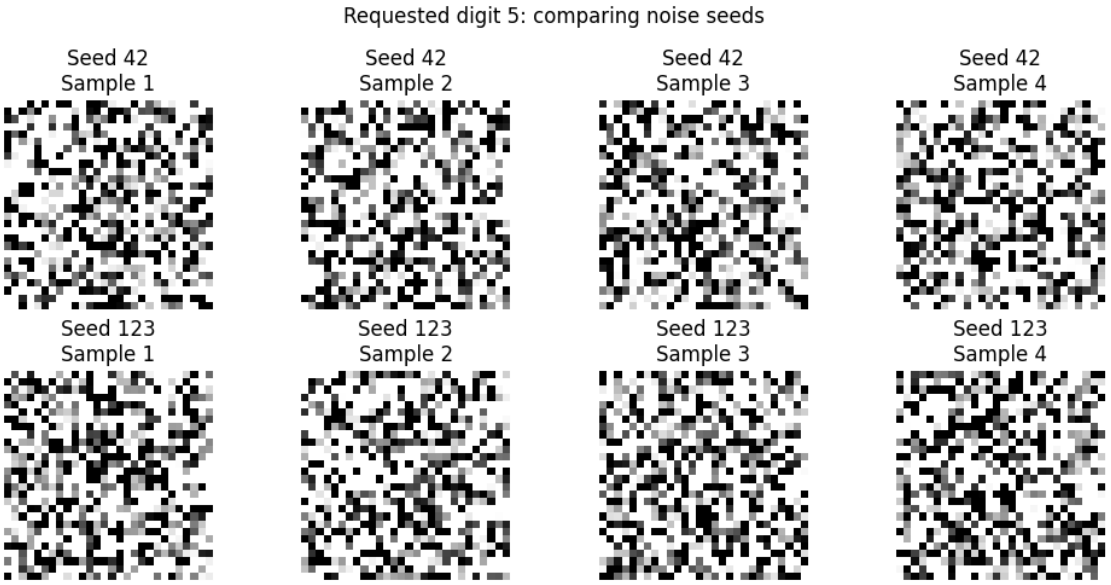
```
Generating 4 versions of number 0…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 1…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 2…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 3…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 4…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 5…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 6…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
```

```
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 7…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 8…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 9…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```




```
STUDENT ACTIVITY: Generating numbers with different noise seeds
Generating 4 versions of number 5…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
```

```
  Denoising step 80/100 completed
  Denoising step 100/100 completed
Generating 4 versions of number 5…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



1.8 Step 7: Watching the Generation Process

Let's see how our model turns random noise into clear images, step by step. This helps us understand how the diffusion process works!

[ ]:
```python
def visualize_generation_steps(model, number, n_preview_steps=10):
    """
    Show one image evolving from random noise into a generated sample.

    n_preview_steps controls how many checkpoints we display,
    in addition to the starting noise.
    """
    model.eval()

    # Check that the requested number of previews is valid.
    if not 1 <= n_preview_steps <= n_steps:
        raise ValueError(
            f"n_preview_steps must be between 1 and {n_steps}."
        )
```

```
# We are generating images, so gradient tracking is unnecessary.
with torch.no_grad():

    # Start with one random image.
    # Shape for MNIST: [1 image, 1 channel, 28 height, 28 width].
    x = torch.randn(
        1, IMG_CH, IMG_SIZE, IMG_SIZE, device=device
    )

    # Set up the digit we want to generate.
    c = torch.tensor(
        [number], dtype=torch.long, device=device
    )
    c_one_hot = F.one_hot(c, N_CLASSES).float()

    # --- BEGIN STUDENT WORK: JAMES DELGADO ---

    # CORRECTION: The class label contains 10 values, but the mask
    # needs only one value per image. Copying the one-hot shape
    # caused the earlier error when the U-Net reshaped the mask.
    # A mask value of 1 tells our model to use the digit label.
    c_mask = torch.ones((c.shape[0], 1), device=device)

    # Select checkpoints using the number of completed updates.
    # With 100 steps and 10 previews, these are 10, 20, ..., 100.
    steps_to_show = (
        torch.linspace(0, n_steps, n_preview_steps + 1)
        .round()
        .long()
        .tolist()[1:]
    )

    # Store a copy of the initial noise before denoising begins.
    images = [x[0].cpu().clone()]
    image_titles = ["Initial noise"]

    # --- END STUDENT WORK: JAMES DELGADO ---

    # Remove noise step by step.
    # Every update runs, even when we do not display its result.
    for t in range(n_steps - 1, -1, -1):
        t_batch = torch.full(
            (1,), t, dtype=torch.long, device=device
        )

        # The updated image becomes the input to the next step.
```

```
            x = remove_noise(
                x, t_batch, model, c_one_hot, c_mask
            )

            # JAMES DELGADO - LEARNING NOTE:
            # Timesteps count downward; completed updates count upward.
            # For 100 steps, t=99 is the first update and t=0 is the last.
            completed_steps = n_steps - t

            if completed_steps in steps_to_show:
                images.append(x[0].cpu().clone())
                image_titles.append(
                    f"{completed_steps}/{n_steps}\nupdates"
                )

        # Show the progression.
        plt.figure(figsize=(2 * len(images), 3))

        for i, img in enumerate(images):
            plt.subplot(1, len(images), i + 1)

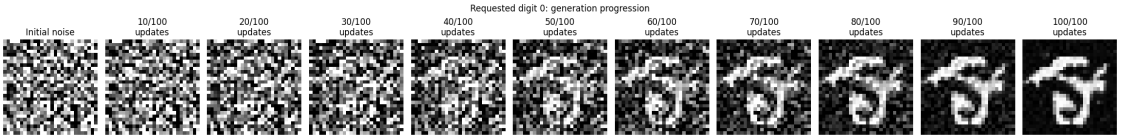
            if IMG_CH == 1:
                # JAMES DELGADO - CORRECTION:
                # Keep the same grayscale scale across all checkpoints
                # so their brightness and contrast can be compared.
                plt.imshow(
                    img[0], cmap="gray", vmin=-1, vmax=1
                )
            else:
                # Convert [C, H, W] to [H, W, C] for color display.
                # Rescaling and clipping affect only the displayed copy.
                display_image = img.permute(1, 2, 0)
                display_image = (
                    (display_image + 1) / 2
                ).clamp(0, 1)
                plt.imshow(display_image)

            plt.title(image_titles[i])
            plt.axis("off")

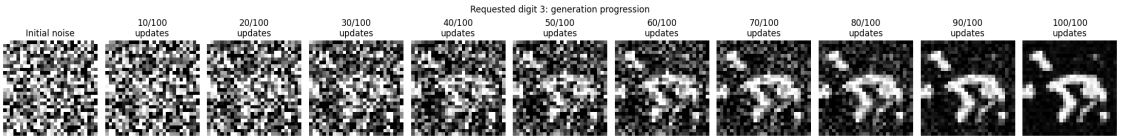
        # The title identifies the requested digit, not a verified result.
        plt.suptitle(
            f"Requested digit {number}: generation progression"
        )
        plt.tight_layout()
        plt.show()
```

```
# Show a separate generation sequence for each requested digit.
for number in [0, 3, 7]:
    print(f"\nGenerating number {number}:")
    visualize_generation_steps(model, number)
```

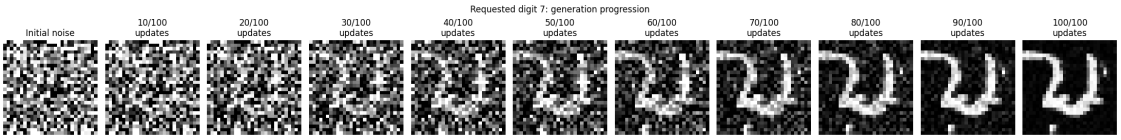
```
Generating number 0:
```




```
Generating number 3:
```




```
Generating number 7:
```



1.9 Step 8: Adding CLIP Evaluation

CLIP is a powerful AI model that can understand both images and text. We'll use it to: 1. Evaluate how realistic our generated images are 2. Score how well they match their intended numbers 3. Help guide the generation process towards better quality

[ ]:
```
## Step 8: Adding CLIP Evaluation

# CLIP (Contrastive Language-Image Pre-training) is a powerful model by OpenAI␣
 ↪that connects text and images.
# We'll use it to evaluate how recognizable our generated digits are by␣
 ↪measuring how strongly
```

```
# the CLIP model associates our generated images with text descriptions like␣
 ↪"an image of the digit 7".

# First, we need to install CLIP and its dependencies
print("Setting up CLIP (Contrastive Language-Image Pre-training) model...")

# Track installation status
clip_available = False

try:
    # Install dependencies first - these help CLIP process text and images
    print("Installing CLIP dependencies...")
    !pip install -q ftfy regex tqdm

    # Install CLIP from GitHub
    print("Installing CLIP from GitHub repository...")
    !pip install -q git+https://github.com/openai/CLIP.git

    # Import and verify CLIP is working
    print("Importing CLIP...")
    import clip

    # Test that CLIP is functioning
    models = clip.available_models()
    print(f"  CLIP installation successful! Available models: {models}")
    clip_available = True

except ImportError:
    print("  Error importing CLIP. Installation might have failed.")
    print("Try manually running: !pip install git+https://github.com/openai/
 ↪CLIP.git")
    print("If you're in a Colab notebook, try restarting the runtime after␣
 ↪installation.")

except Exception as e:
    print(f"  Error during CLIP setup: {e}")
    print("Some CLIP functionality may not work correctly.")

# Provide guidance based on installation result
if clip_available:
    print("\nCLIP is now available for evaluating your generated images!")
else:
    print("\nWARNING: CLIP installation failed. We'll skip the CLIP evaluation␣
 ↪parts.")

# Import necessary libraries
import functools
```

```
import torch.nn.functional as F
```

```
Setting up CLIP (Contrastive Language-Image Pre-training) model…
Installing CLIP dependencies…
                         44.8/44.8 kB
2.6 MB/s eta 0:00:00
Installing CLIP from GitHub repository…
  Preparing metadata (setup.py) … done
  Building wheel for clip (setup.py) … done
Importing CLIP…
 CLIP installation successful! Available models: ['RN50', 'RN101', 'RN50x4',
'RN50x16', 'RN50x64', 'ViT-B/32', 'ViT-B/16', 'ViT-L/14', 'ViT-L/14@336px']

CLIP is now available for evaluating your generated images!
```

Below we are createing a helper function to manage GPU memory when using CLIP. CLIP can be memory-intensive, so this will help prevent out-of-memory errors:

[ ]:
```
# Memory management decorator to prevent GPU OOM errors
def manage_gpu_memory(func):
    """
    Decorator that ensures proper GPU memory management.

    This wraps functions that might use large amounts of GPU memory,
    making sure memory is properly freed after function execution.
    """
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        if torch.cuda.is_available():
            # Clear cache before running function
            torch.cuda.empty_cache()
            try:
                return func(*args, **kwargs)
            finally:
                # Clear cache after running function regardless of success/
 ↪failure
                torch.cuda.empty_cache()
        return func(*args, **kwargs)
    return wrapper
```

[ ]:
```
#================================================================================
# Step 8: CLIP Model Loading and Evaluation Setup
#================================================================================

# CLIP means Contrastive Language-Image Pre-training.
# It represents images and text as numerical feature vectors, then compares␣
 ↪them.
#
```

```python
# JAMES DELGADO - LEARNING NOTE:
# Our diffusion model generates the images. CLIP is a separate pretrained model
# that compares those images with descriptions. Neither model is trained here.

import traceback
import torch
import torch.nn.functional as F
import torchvision.transforms as transforms
import matplotlib.pyplot as plt


# --- BEGIN STUDENT WORK: JAMES DELGADO ---

# Initialize this flag every time the cell runs.
# It becomes True only after CLIP loads successfully.
clip_available = False
clip_model = None
clip_preprocess = None

try:
    import clip

    # ViT means Vision Transformer.
    # clip.load returns both the model and its required image preprocessing.
    clip_model, clip_preprocess = clip.load(
        "ViT-B/32", device=device
    )

    clip_model.eval()
    clip_model.requires_grad_(False)

    clip_available = True
    print(f"CLIP loaded successfully on {device}.")

except Exception as error:
    print(
        f"CLIP could not load: "
        f"{type(error).__name__}: {error}"
    )
    print(
        "Generated images can still be displayed. "
        "CLIP scores will be marked unavailable."
    )
    print(
        "If the clip package is missing, run the notebook's "
        "CLIP installation cell, then rerun this cell."
    )
```

```python
# --- END STUDENT WORK: JAMES DELGADO ---


@torch.no_grad()
def _process_clip_batch(images, target_number):
    """
    Compare a batch of generated images with three text prompts.

    Returns:
        CPU tensor of shape [batch_size, 3].

        Each row contains relative prompt weights that sum to 1.
        These are not measured probabilities of image quality.
    """

    # Preserve the assignment's three descriptions.
    # Their wording influences the resulting scores.
    prompts = [
        f"A handwritten number {target_number}",
        f"A clear, well-written digit {target_number}",
        "A blurry or unclear number"
    ]

    # Tokenization converts text into the integer inputs CLIP expects.
    text_inputs = clip.tokenize(prompts).to(device)

    # --- BEGIN STUDENT WORK: JAMES DELGADO ---

    # Our diffusion model uses the MNIST training scale of [-1, 1].
    # Undo that normalization to obtain display values in [0, 1].
    #
    # Generated values can extend outside the training range.
    # Clamp only this evaluation copy; do not change the original samples.
    display_images = (
        (images.detach().cpu().float() + 1) / 2
    ).clamp(0, 1)

    # CLIP's preprocessing expects a PIL image.
    to_pil = transforms.ToPILImage()
    prepared_images = []

    for image in display_images:
        # RGB means red, green, and blue.
        # Converting grayscale to RGB provides three matching channels.
        pil_image = to_pil(image).convert("RGB")
```

```
        # CORRECTION: Resizing alone is insufficient.
        # Use CLIP's own resizing, cropping, and normalization.
        prepared_images.append(clip_preprocess(pil_image))

    # Combine individual image tensors into one batch.
    # For ViT-B/32, the prepared shape is [B, 3, 224, 224].
    image_inputs = torch.stack(prepared_images).to(device)

    # CLIP creates one feature vector per image and one per text prompt.
    image_features = clip_model.encode_image(image_inputs).float()
    text_features = clip_model.encode_text(text_inputs).float()

    # Normalize vectors to unit length.
    # Their dot products now measure cosine similarity:
    # how closely their directions align.
    image_features = F.normalize(image_features, dim=-1)
    text_features = F.normalize(text_features, dim=-1)

    # Compare every image with each of the three text descriptions.
    # Shape: [B, feature_count] @ [feature_count, 3] -> [B, 3].
    cosine_similarities = image_features @ text_features.T

    # Use CLIP's learned score scale.
    # This scaling controls softmax sharpness; it does not create percentages.
    logits = clip_model.logit_scale.exp() * cosine_similarities

    # Softmax produces three positive weights that sum to 1 per image.
    # A higher weight means stronger preference within THIS prompt set.
    relative_weights = logits.softmax(dim=-1)

    if not torch.isfinite(relative_weights).all().item():
        raise ValueError("CLIP produced non-finite scores.")

    # Move small result tensors to CPU so stored scores do not use GPU memory.
    return relative_weights.cpu()

    # --- END STUDENT WORK: JAMES DELGADO ---


def evaluate_with_clip(images, target_number, max_batch_size=16):
    """
    Evaluate images in manageable batches.

    Returns:
        Tensor [number_of_images, 3] when evaluation succeeds.
        None when CLIP is unavailable or evaluation fails.
    """
```

```
    # JAMES DELGADO - CORRECTION:
    # Missing evaluation results must not be replaced with made-up scores.
    if not clip_available:
        return None

    try:
        if images.ndim != 4 or images.shape[0] == 0:
            raise ValueError(
                "Expected a nonempty image batch with shape [B, C, H, W]."
            )

        if images.shape[1] not in (1, 3):
            raise ValueError("Images must have 1 or 3 channels.")

        if not torch.isfinite(images).all().item():
            raise ValueError("Generated images contain NaN or infinite values.")

        if max_batch_size < 1:
            raise ValueError("max_batch_size must be at least 1.")

        all_scores = []
        start = 0
        batch_size = min(max_batch_size, len(images))

        while start < len(images):
            end = min(start + batch_size, len(images))

            try:
                batch_scores = _process_clip_batch(
                    images[start:end], target_number
                )

            except torch.cuda.OutOfMemoryError:
                # JAMES DELGADO - LEARNING NOTE:
                # Smaller batches reduce the images processed at once.
                # If even one image does not fit, report the failure.
                if batch_size == 1:
                    raise

                batch_size = max(1, batch_size // 2)
                print(
                    f"CLIP ran out of GPU memory. "
                    f"Retrying with batch size {batch_size}."
                )
                torch.cuda.empty_cache()
                continue
```

```
            all_scores.append(batch_scores)
            start = end

        return torch.cat(all_scores, dim=0)

    except Exception as error:
        print(
            f"CLIP evaluation unavailable: "
            f"{type(error).__name__}: {error}"
        )
        return None


# --- BEGIN STUDENT WORK: JAMES DELGADO ---

def display_clip_results(samples, number, similarities, ncols=4):
    """
    Display generated samples with relative CLIP prompt weights.
    """
    n_images = len(samples)
    ncols = min(ncols, n_images)
    nrows = (n_images + ncols - 1) // ncols

    fig, axes = plt.subplots(
        nrows,
        ncols,
        figsize=(3.5 * ncols, 4 * nrows),
        squeeze=False
    )
    axes = axes.ravel()

    for i, axis in enumerate(axes):
        if i >= n_images:
            axis.axis("off")
            continue

        # Use the same image scale for every sample.
        # The conversion affects this display copy only.
        display_image = (
            (samples[i].detach().cpu().float() + 1) / 2
        ).clamp(0, 1)

        if display_image.shape[0] == 1:
            axis.imshow(
                display_image[0],
                cmap="gray",
```

```
                vmin=0,
                vmax=1
            )
        else:
            axis.imshow(display_image.permute(1, 2, 0))

        if similarities is None:
            score_text = "CLIP scores unavailable"
        else:
            handwritten, clear, blurry = similarities[i].tolist()

            # CORRECTION: These are prompt comparison weights.
            # Avoid labeling them as accuracy or pass/fail judgments.
            score_text = (
                f"Handwritten prompt: {handwritten:.2f}\n"
                f"Clear prompt: {clear:.2f}\n"
                f"Blurry prompt: {blurry:.2f}"
            )

        axis.set_title(
            f"Requested digit: {number}\n"
            f"Sample {i + 1}\n"
            f"{score_text}",
            fontsize=10
        )
        axis.axis("off")

    fig.suptitle(
        "CLIP relative prompt weights, not recognition accuracy",
        fontsize=12
    )
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    plt.show()
    plt.close(fig)

# --- END STUDENT WORK: JAMES DELGADO ---


#================================================================================
# CLIP Evaluation - Generate and Analyze Sample Digits
#================================================================================

print("\nComparing generated digits with three text prompts.")
print("The displayed weights sum to approximately 1 for each image.")
print("Compare these results with your own visual observations.")

try:
```

```
    for number in range(10):
        print(f"\nGenerating and evaluating requested digit {number}...")

        # Reuse our existing diffusion model and generation function.
        samples = generate_number(model, number, n_samples=4)

        similarities = evaluate_with_clip(
            samples,
            target_number=number,
            max_batch_size=16
        )

        display_clip_results(
            samples,
            number,
            similarities,
            ncols=4
        )

except Exception:
    # Report generation or plotting errors and stop this cell.
    # This prevents a failed run from appearing to have completed.
    print("The generation and evaluation loop stopped:")
    traceback.print_exc()
    raise


#============================================================================
# STUDENT ACTIVITY: Exploring CLIP Evaluation
#============================================================================

# --- BEGIN STUDENT WORK: JAMES DELGADO ---

# Generate more examples of one digit to examine variation.
# This creates new samples using the same trained weights.
digit = 6
activity_sample_count = 10

print(
    f"\nSTUDENT ACTIVITY: Comparing "
    f"{activity_sample_count} samples of requested digit {digit}."
)

activity_samples = generate_number(
    model,
    digit,
    n_samples=activity_sample_count
```

```
)

# Use batches of four to demonstrate chunked evaluation.
# Ten images are processed as batches of 4, 4, and 2.
activity_scores = evaluate_with_clip(
    activity_samples,
    target_number=digit,
    max_batch_size=4
)

if activity_scores is not None:
    prompt_names = [
        "Handwritten prompt",
        "Clear prompt",
        "Blurry prompt"
    ]

    mean_weights = activity_scores.mean(dim=0)
    strongest_prompts = activity_scores.argmax(dim=1)

    print("\nPrompt comparison summary:")

    for prompt_index, prompt_name in enumerate(prompt_names):
        strongest_count = (
            strongest_prompts == prompt_index
        ).sum().item()

        print(
            f"{prompt_name}: "
            f"mean weight = {mean_weights[prompt_index].item():.3f}; "
            f"highest weight for {strongest_count}/{activity_sample_count}␣
 ↪images"
        )

    # The first two descriptions overlap in meaning.
    # Adding their weights does not establish that an image is correct.
    print(
        "\nThese counts describe CLIP's prompt preferences. "
        "They do not count correctly generated digits."
    )

else:
    print("The activity images are available, but CLIP scoring failed.")

display_clip_results(
    activity_samples,
    digit,
```

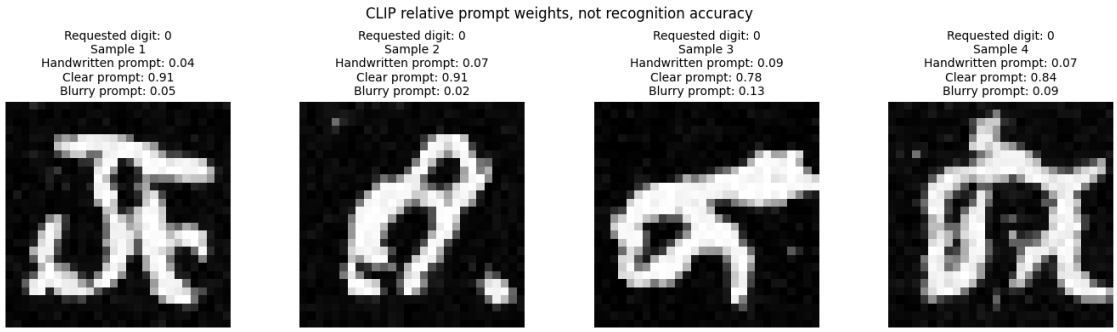
```
    activity_scores,
    ncols=5
)

# --- END STUDENT WORK: JAMES DELGADO ---
```

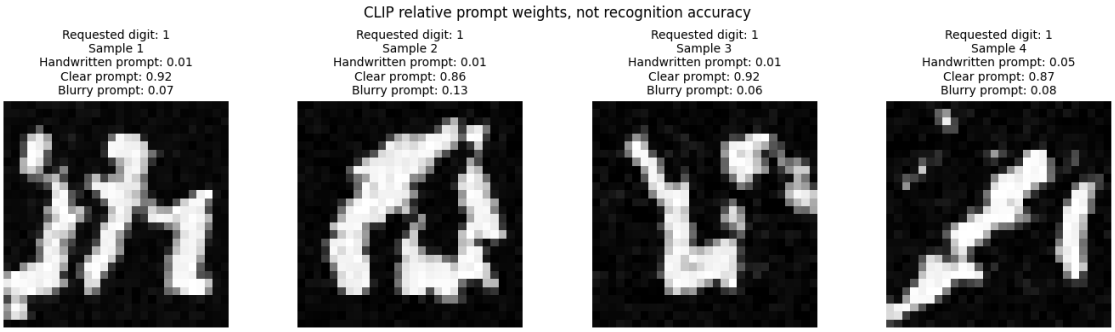
```
CLIP loaded successfully on cuda.

Comparing generated digits with three text prompts.
The displayed weights sum to approximately 1 for each image.
Compare these results with your own visual observations.

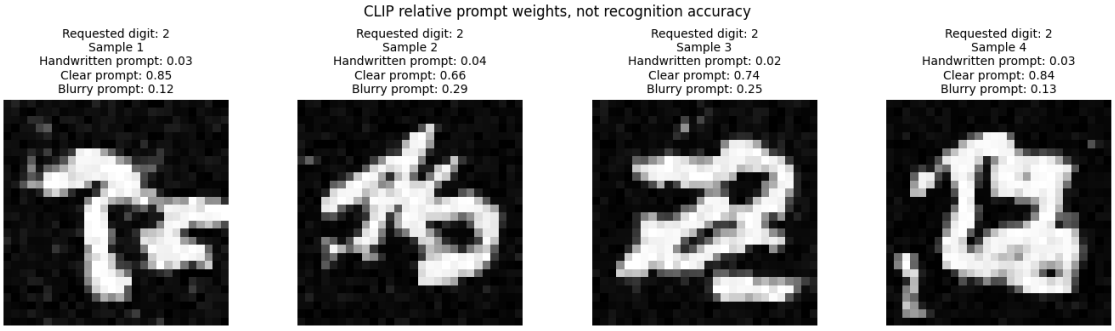
Generating and evaluating requested digit 0…
Generating 4 versions of number 0…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



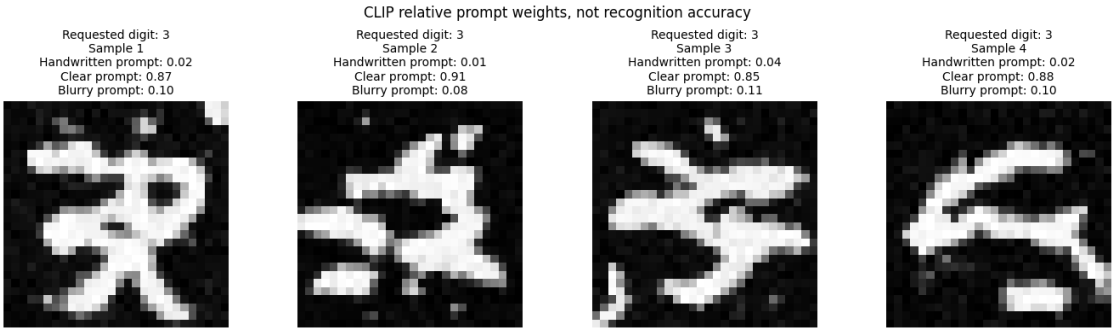

```
Generating and evaluating requested digit 1…
Generating 4 versions of number 1…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



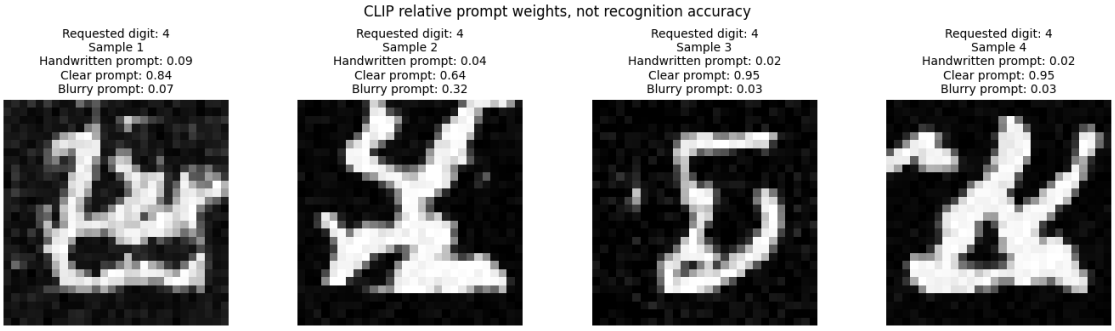

```
Generating and evaluating requested digit 2…
Generating 4 versions of number 2…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



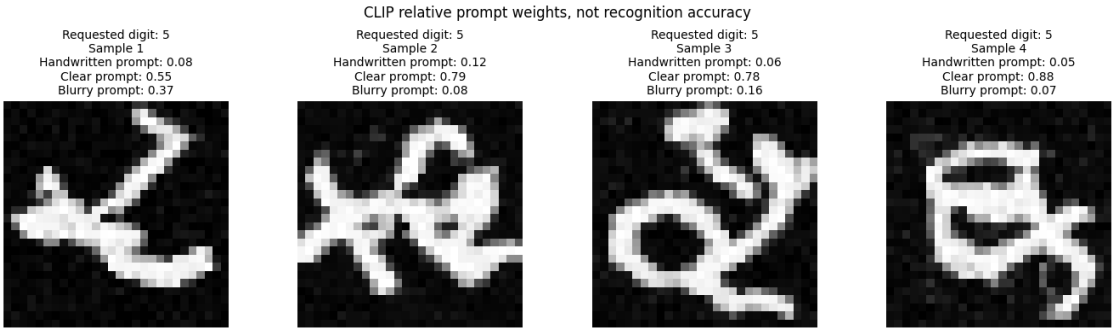

```
Generating and evaluating requested digit 3…
Generating 4 versions of number 3…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



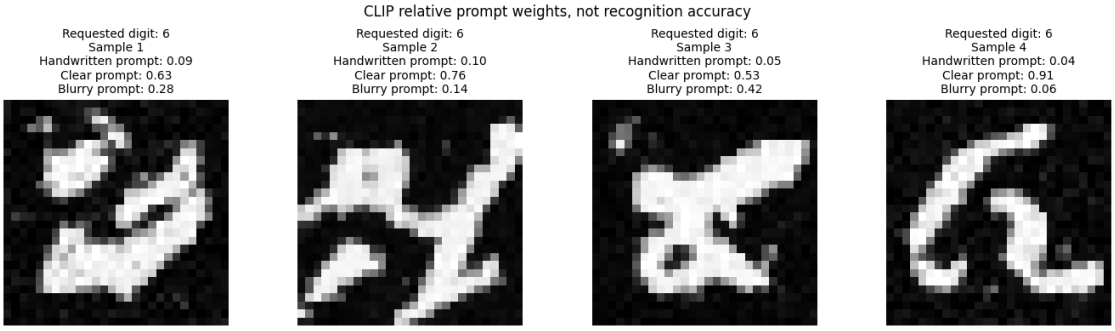

```
Generating and evaluating requested digit 4…
Generating 4 versions of number 4…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



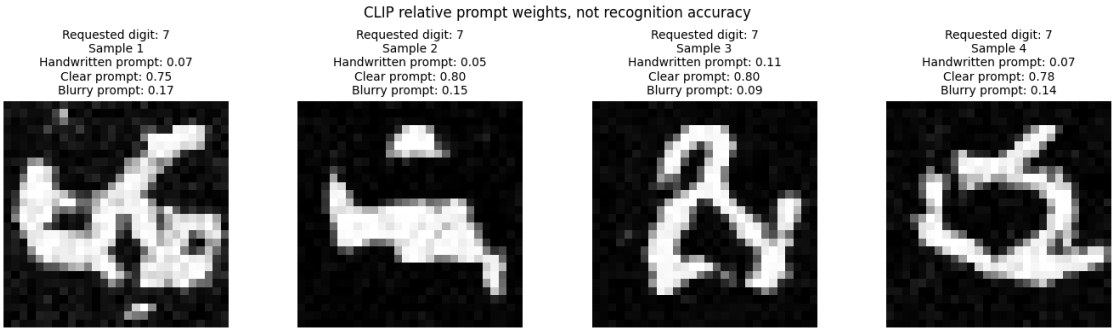

```
Generating and evaluating requested digit 5…
Generating 4 versions of number 5…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



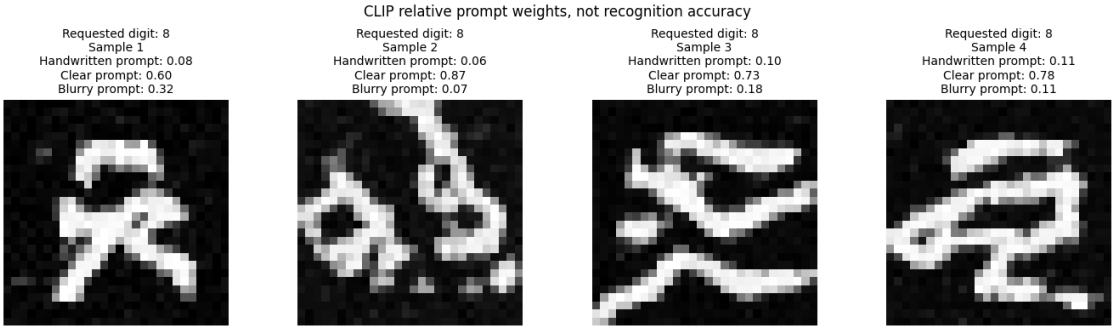

```
Generating and evaluating requested digit 6…
Generating 4 versions of number 6…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



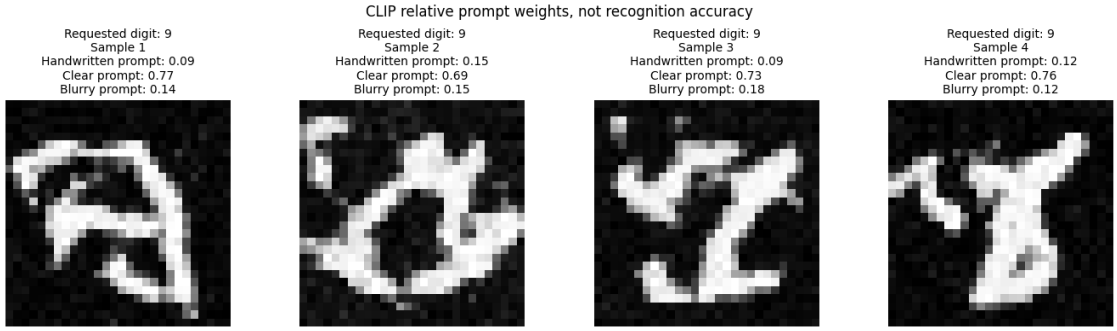

```
Generating and evaluating requested digit 7…
Generating 4 versions of number 7…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```




```
Generating and evaluating requested digit 8…
Generating 4 versions of number 8…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```



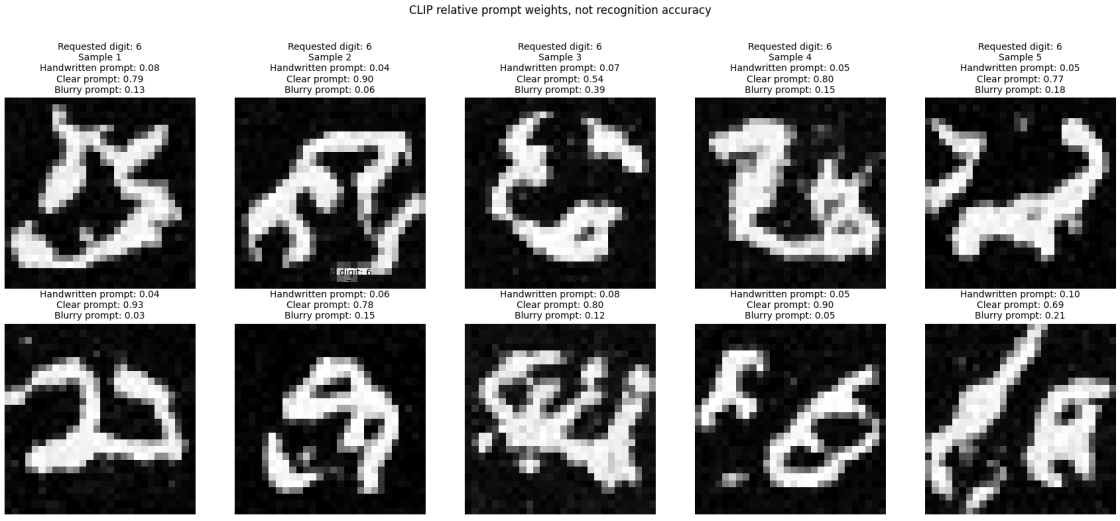

```
Generating and evaluating requested digit 9…
Generating 4 versions of number 9…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed
```




```
STUDENT ACTIVITY: Comparing 10 samples of requested digit 6.
Generating 10 versions of number 6…
  Denoising step 20/100 completed
  Denoising step 40/100 completed
  Denoising step 60/100 completed
  Denoising step 80/100 completed
  Denoising step 100/100 completed

Prompt comparison summary:
Handwritten prompt: mean weight = 0.063; highest weight for 0/10 images
Clear prompt: mean weight = 0.789; highest weight for 10/10 images
Blurry prompt: mean weight = 0.148; highest weight for 0/10 images

These counts describe CLIP's prompt preferences. They do not count correctly
generated digits.
```



1.10 Assessment Questions - 10% of ovarall grade

KING OWOUSU

1.11 Assessment Answers

1.11.1 1. Understanding Diffusion

Forward diffusion in my own words. The forward process slowly destroys an image by mixing in Gaussian noise: x_t = √ _t · x_0 + √(1− _t) · . As t grows, the share of the original image shrinks and the share of noise grows. In the noise-progression figure, the "0" is still clearly a "0" at timestep 24, has a grainy outline at 49, is mostly broken up at 74, and is only a faint ghost at 99. With running from 0.0001 to 0.02 over only 100 steps, at the last step is about 0.36, so the image keeps roughly 60% of its signal (√ 0.60). That is why a faint outline survives at step 99: the schedule does not quite reach pure noise.

Why add noise gradually? Each small step only needs to be undone a little, which is much easier than turning static into a digit in one jump. The model can learn one simple rule: predict the noise that was just added. Training picks a random timestep for every image, so the model sees every noise level. During generation, many small steps let the model fix its own mistakes and add detail progressively, with coarse layout first and fine strokes later. One giant step would force the network to guess the whole image at once, which usually gives blurry averages.

When can I first recognize the image? In the step-by-step figures (digits 0, 3, 7), the images are static through about 20–30% of the process. Stroke-like structure appears at roughly 40–50% (digit 7 by about 40–50/100, digit 0 by about 50/100, digit 3 by about 50–60/100). By about 70% the overall shape is clear, and the last 30% mostly sharpens edges and removes grain. It varies by image: the 7 committed to its shape earliest, while the 3 stayed ambiguous longer. "Recognizable" here means a coherent stroke shape; the final shapes are not clean, readable digits (see Section 5).

1.11.2 2. Model Architecture

Why a U-Net? The model must output an image the same size as its input (the predicted noise), and it needs both global context (what digit is this?) and pixel-level detail (where exactly is the noise?). The down path shrinks 28×28 → 14×14 → 7×7 so each filter sees more of the image, and the up path expands back to 28×28. Compared with a plain stack of same-resolution convolutions, this gives a large receptive field cheaply. Compared with an encoder-only network, it keeps spatial layout and produces an output of the right shape.

Skip connections. A skip connection saves a feature map from the down path and concatenates it onto the matching up-path feature map. That is why `UpBlock`'s transposed convolution takes `2 * in_chs` channels. Downsampling throws away fine spatial detail; skips hand it straight to the decoder so it doesn't have to be rebuilt from the compressed 7×7 map. They also give gradients a short path back through the network, which helps training. In our model the skips are the 64-channel 14×14 map and the 128-channel 7×7 map, saved after each `DownBlock`. One limitation: there is no skip at full 28×28 resolution (the `initial_conv` output is not saved), which likely costs fine detail.

How class conditioning works. The digit label becomes a one-hot vector (3 → [0,0,0,1,0,0,0,0,0,0]). `EmbedBlock` passes it through two linear layers with a GELU and reshapes it to [B, 128, 1, 1]. In `UNet.forward` it is multiplied by `c_mask` (1 = use the label, 0 = drop it)

and added to the bottleneck feature map, so it broadcasts over every spatial position. The time step is handled the same way (sinusoidal embedding → linear → GELU, added at the same place). The decoder is therefore influenced by "which digit" and "which step". The mask would enable classifier-free guidance, but training here always uses mask = 1.

1.11.3 3. Training Analysis

What does the loss tell us? The loss is the mean-squared error between the true noise and the predicted noise; lower means the model is better at finding the noise in a noisy image. Training loss fell from 0.0785 to 0.0577 (a 26.5% improvement) and validation loss from 0.0750 to 0.0578. The curves track each other closely, so there is no sign of overfitting. The curve flattens after about epoch 20 (0.058–0.059), so more epochs at this learning rate would give diminishing returns. A caution: the loss is averaged over all timesteps, and a small loss does not guarantee good-looking samples. Our samples show this.

How did image quality change during training? Loss improved steadily, with the biggest drop in the first ~5 epochs. By the end the model produces crisp white strokes on a clean black background with the right overall MNIST style (stroke width, centered blob), but the strokes do not form reliable digits. What did not appear is correct digit identity.

Why the time embedding? The same network must denoise at every noise level. At t 95 the right action is "pull a rough shape out of almost nothing", and at t 5 it is "clean up a few grains of noise". Without a time signal the network can't tell which situation it is in, because moderately noisy images look similar at different steps. The sinusoidal embedding gives each step a unique, smooth vector (nearby steps get similar vectors), and the network learns what to do with it, so it knows where it is in the denoising process and how strongly to correct.

1.11.4 4. CLIP Evaluation

What do the CLIP scores say? Our CLIP setup compares each image against three prompts (handwritten, clear, blurry) and reports relative weights that sum to about 1. For the 10 samples of digit 6, the "clear" prompt had a mean weight of 0.789 and won for 10/10 images; "blurry" averaged 0.148 and "handwritten" 0.063. The highest scores I saw were about 0.91–0.93 (digit 0 samples 1 and 2 at 0.91, digit 1 samples 1 and 3 at 0.92, digit 6 sample 6 at 0.93). The lowest was digit 6 sample 3 (clear 0.54, blurry 0.39), which has thin, fragmented, disconnected strokes. **Caveat:** these numbers measure how clear the picture looks, not whether it is the right digit. CLIP's own figure title says "relative prompt weights, not recognition accuracy". High scores on images I can't read as the requested digit show that the metric rewards clean, high-contrast rendering rather than correctness.

Hypothesis about easy vs. hard images. Images with thick, connected, high-contrast strokes and little speckle score higher on "clear", while broken, thin or scattered fragments score lower and drift toward "blurry". For digits, I would expect shapes with one dominant stroke (like 1) to be easier, and shapes with closed loops or several parts (6, 8, 9, 0) to be harder, because the model must get several parts right in the right relation. I haven't tested this beyond the digits shown; the next step would be to score 50+ samples per digit and compare the averages.

Using CLIP to improve generation. Technique: CLIP re-ranking (best-of-N) with digit-specific prompts. For each requested digit k, generate N = 16 candidates, score each against ten prompts ("a handwritten digit 0" … "a handwritten digit 9"), keep only candidates where prompt k has

the highest probability (zero-shot classification), and return the top few by that probability. This measures correctness directly, which the clear/blurry prompts don't. A stronger version would use CLIP's gradient to nudge x_t toward the target prompt during denoising; classifier-free guidance using the existing `c_mask` is another cheap option.

1.11.5 5. Practical Applications

Real-world uses. Image generation and editing; data augmentation (e.g. synthetic medical or handwriting images when real data is scarce); denoising and restoration (old photos, MRI/CT noise reduction); inpainting and super-resolution; and, with modified architectures, audio, video and molecule design. Class-conditional generation like ours is the simplest version of "generate X on request".

Limitations of our model. - Samples do not reliably look like the requested digit, and most of the 40-image grid is not readable as digits. - Conditioning is injected only once, at the 7×7 bottleneck, and the time embedding is only 8-dimensional, so guidance about both digit and noise level is weak. - No full-resolution skip connection, which limits fine detail. - Only 100 steps, and the schedule ends at 0.36 instead of near 0, so generation starts from pure noise that training never saw. - The model is small (1.5M parameters), trained for 30 epochs, and the loss had plateaued. - The CLIP evaluation measures clarity, not correctness, so it overstates quality, and there is no real accuracy measurement.

Three improvements I would make. 1. **Inject time and class conditioning at every resolution level** (inside each Down/Up block) and use a larger time embedding (e.g. 128-d). This most directly addresses weak digit control and weak step awareness. 2. **Fix the schedule and add a full-resolution skip:** use more steps (300–1000) or a cosine schedule so _T 0, and save the `initial_conv` output as a 28×28 skip. This removes the train/generate mismatch and restores fine detail. 3. **Add classifier-free guidance and a real evaluation:** randomly drop the label during training (using the existing mask), sample with a guidance scale, and measure correctness with a small MNIST classifier and/or digit-specific CLIP prompts, so improvements show up as accuracy rather than just clarity.